4.2.2 SE(3) 上的指数映射

下面介绍 $\mathfrak{se}(3)$ 上的指数映射。为了节省篇幅，我们不再像 $\mathfrak{so}(3)$ 那样详细推导指数映射。$\mathfrak{se}(3)$ 上的指数映射形式如下：

$$\exp\left(\boldsymbol{\xi}^{\wedge}\right)=\begin{bmatrix}\sum_{n=0}^{\infty}\frac{1}{n!}\left(\boldsymbol{\phi}^{\wedge}\right)^{n} & \sum_{n=0}^{\infty}\frac{1}{(n+1)!}\left(\boldsymbol{\phi}^{\wedge}\right)^{n}\boldsymbol{\rho}\\ \mathbf{0}^{\mathrm{T}} & 1\end{bmatrix} \tag{4.24}$$

$$\triangleq\begin{bmatrix}\boldsymbol{R} & \boldsymbol{J}\boldsymbol{\rho}\\ \mathbf{0}^{\mathrm{T}} & 1\end{bmatrix}=\boldsymbol{T}. \tag{4.25}$$

只要有一点耐心，可以照着 $\mathfrak{so}(3)$ 上的做法推导，把 exp 进行泰勒展开推导此式。令 $\boldsymbol{\phi}=\theta\boldsymbol{a}$，其中 $\boldsymbol{a}$ 为单位向量，则

$$\begin{aligned}\sum_{n=0}^{\infty}\frac{1}{(n+1)!}\left(\boldsymbol{\phi}^{\wedge}\right)^{n}&=\boldsymbol{I}+\frac{1}{2!}\theta\boldsymbol{a}^{\wedge}+\frac{1}{3!}\theta^{2}\left(\boldsymbol{a}^{\wedge}\right)^{2}+\frac{1}{4!}\theta^{3}\left(\boldsymbol{a}^{\wedge}\right)^{3}+\frac{1}{5!}\theta^{4}\left(\boldsymbol{a}^{\wedge}\right)^{4}\cdots\\&=\frac{1}{\theta}\left(\frac{1}{2!}\theta^{2}-\frac{1}{4!}\theta^{4}+\cdots\right)\left(\boldsymbol{a}^{\wedge}\right)+\frac{1}{\theta}\left(\frac{1}{3!}\theta^{3}-\frac{1}{5}\theta^{5}+\cdots\right)\left(\boldsymbol{a}^{\wedge}\right)^{2}+\boldsymbol{I}\\&=\frac{1}{\theta}\left(1-\cos\theta\right)\left(\boldsymbol{a}^{\wedge}\right)+\frac{\theta-\sin\theta}{\theta}\left(\boldsymbol{a}\boldsymbol{a}^{\mathrm{T}}-\boldsymbol{I}\right)+\boldsymbol{I}\\&=\frac{\sin\theta}{\theta}\boldsymbol{I}+\left(1-\frac{\sin\theta}{\theta}\right)\boldsymbol{a}\boldsymbol{a}^{\mathrm{T}}+\frac{1-\cos\theta}{\theta}\boldsymbol{a}^{\wedge}\overset{\text{def}}{=}\boldsymbol{J}.\end{aligned} \tag{4.26}$$

从结果上看，$\boldsymbol{\xi}$ 的指数映射左上角的 $\boldsymbol{R}$ 是我们熟知的 SO(3) 中的元素，与 $\mathfrak{se}(3)$ 中的旋转部分 $\boldsymbol{\phi}$ 对应。而右上角的 $\boldsymbol{J}$ 由上面的推导给出：

$$\boldsymbol{J}=\frac{\sin\theta}{\theta}\boldsymbol{I}+\left(1-\frac{\sin\theta}{\theta}\right)\boldsymbol{a}\boldsymbol{a}^{\mathrm{T}}+\frac{1-\cos\theta}{\theta}\boldsymbol{a}^{\wedge}. \tag{4.27}$$

该式与罗德里格斯公式有些相似，但不完全一样。我们看到，平移部分经过指数映射之后，发生了一次以 $\boldsymbol{J}$ 为系数矩阵的线性变换。请读者重视这里的 $\boldsymbol{J}$，因为后面还要用到。

同样地，虽然我们也可以类比推得对数映射，不过根据变换矩阵 $\boldsymbol{T}$ 求 $\mathfrak{so}(3)$ 上的对应向量也有更省事的方式：从左上角的 $\boldsymbol{R}$ 计算旋转向量，而右上角的 $\boldsymbol{t}$ 满足：

$$\boldsymbol{t}=\boldsymbol{J}\boldsymbol{\rho}. \tag{4.28}$$

由于 $\boldsymbol{J}$ 可以由 $\boldsymbol{\phi}$ 得到，所以这里的 $\boldsymbol{\rho}$ 也可由此线性方程解得。现在，我们已经弄清了李群、

李代数的定义与相互的转换关系，总结如图 4-1 所示。如果读者有哪里不明白，可以翻回去看看公式推导。

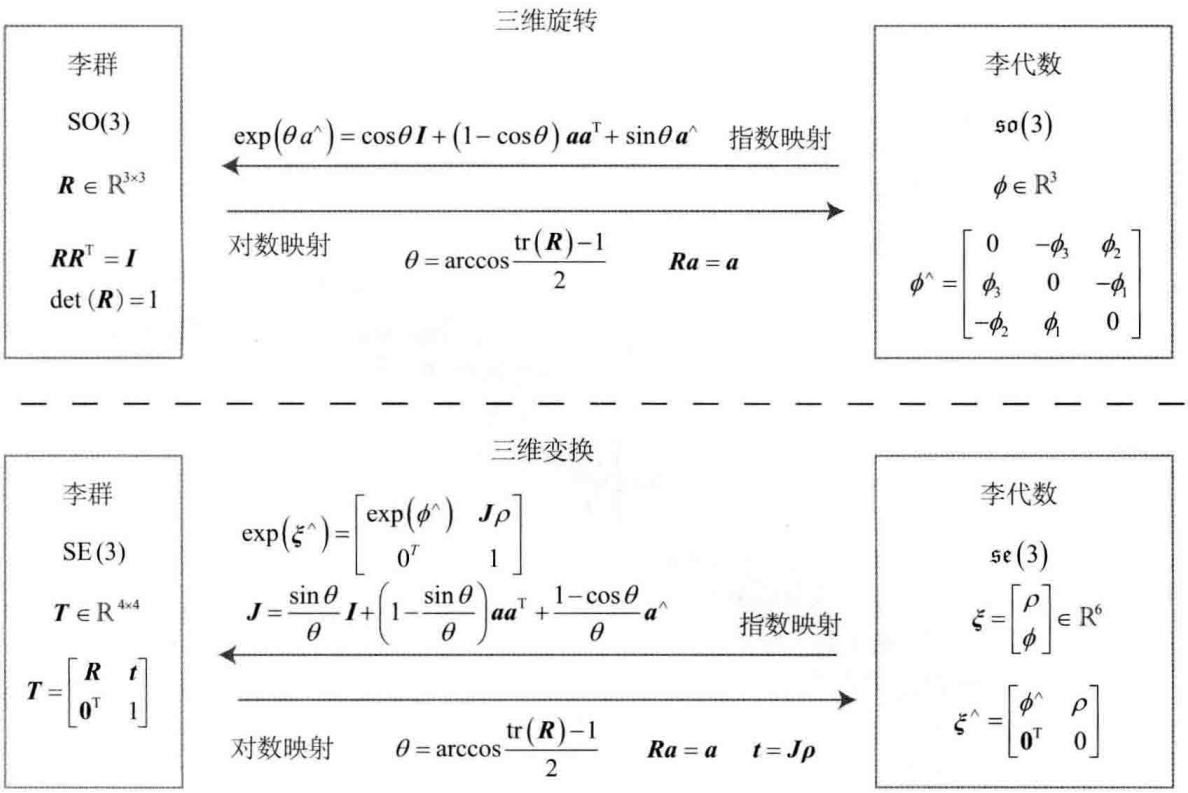


图 4-1　$\mathrm{SO}(3), \mathrm{SE}(3), \mathfrak{so}(3), \mathfrak{se}(3)$ 的对应关系

4.3　李代数求导与扰动模型

4.3.1　BCH 公式与近似形式

使用李代数的一大动机是进行优化，而在优化过程中导数是非常必要的信息（我们会在第 6 讲详细介绍）。下面来考虑一个问题。虽然我们已经清楚了 SO(3) 和 SE(3) 上的李群与李代数关系，但是，当在 SO(3) 中完成两个矩阵乘法时，李代数中 $\mathfrak{so}(3)$ 上发生了什么改变呢？反过来说，当 $\mathfrak{so}(3)$ 上做两个李代数的加法时，SO(3) 上是否对应着两个矩阵的乘积？如果成立，相当于：

$$\exp(\boldsymbol{\phi}_1^{\wedge})\exp(\boldsymbol{\phi}_2^{\wedge}) = \exp\left((\boldsymbol{\phi}_1 + \boldsymbol{\phi}_2)^{\wedge}\right)?$$

如果 $\boldsymbol{\phi}_1, \boldsymbol{\phi}_2$ 为标量，那么显然该式成立；但此处我们计算的是**矩阵**的指数函数，而非标量的

指数。换言之，我们在研究下式是否成立：

$$\ln\left(\exp\left(\boldsymbol{A}\right)\exp\left(\boldsymbol{B}\right)\right)=\boldsymbol{A}+\boldsymbol{B}\ ?$$

很遗憾，该式在矩阵时并不成立。两个李代数指数映射乘积的完整形式，由 Baker-Campbell-Hausdorff 公式（BCH 公式）[①]给出。由于其完整形式较复杂，我们只给出其展开式的前几项：

$$\ln\left(\exp\left(\boldsymbol{A}\right)\exp\left(\boldsymbol{B}\right)\right)=\boldsymbol{A}+\boldsymbol{B}+\frac{1}{2}\left[\boldsymbol{A},\boldsymbol{B}\right]+\frac{1}{12}\left[\boldsymbol{A},\left[\boldsymbol{A},\boldsymbol{B}\right]\right]-\frac{1}{12}\left[\boldsymbol{B},\left[\boldsymbol{A},\boldsymbol{B}\right]\right]+\cdots \tag{4.29}$$

其中 [ ] 为李括号。BCH 公式告诉我们，当处理两个矩阵指数之积时，它们会产生一些由李括号组成的余项。特别地，考虑 SO(3) 上的李代数 $\ln\left(\exp\left(\boldsymbol{\phi}_1^\wedge\right)\exp\left(\boldsymbol{\phi}_2^\wedge\right)\right)^\vee$，当 $\boldsymbol{\phi_1}$ 或 $\boldsymbol{\phi_2}$ 为小量时，小量二次以上的项都可以被忽略。此时，BCH 拥有线性近似表达[②]：

$$\ln\left(\exp\left(\boldsymbol{\phi}_1^\wedge\right)\exp\left(\boldsymbol{\phi}_2^\wedge\right)\right)^\vee\approx\begin{cases}\boldsymbol{J}_l\left(\boldsymbol{\phi}_2\right)^{-1}\boldsymbol{\phi}_1+\boldsymbol{\phi}_2 & \text{当}\boldsymbol{\phi}_1\text{为小量,}\\ \boldsymbol{J}_r\left(\boldsymbol{\phi}_1\right)^{-1}\boldsymbol{\phi}_2+\boldsymbol{\phi}_1 & \text{当}\boldsymbol{\phi}_2\text{为小量.}\end{cases} \tag{4.30}$$

以第一个近似为例。该式告诉我们，当对一个旋转矩阵 $\boldsymbol{R}_2$（李代数为 $\boldsymbol{\phi}_2$）左乘一个微小旋转矩阵 $\boldsymbol{R}_1$（李代数为 $\boldsymbol{\phi}_1$）时，可以近似地看作，在原有的李代数 $\boldsymbol{\phi}_2$ 上加上了一项 $\boldsymbol{J}_l(\boldsymbol{\phi}_2)^{-1}\boldsymbol{\phi}_1$。同理，第二个近似描述了右乘一个微小位移的情况。于是，李代数在 BCH 近似下，分成了左乘近似和右乘近似两种，在使用时我们须注意使用的是左乘模型还是右乘模型。

本书以左乘为例。左乘 BCH 近似雅可比 $\boldsymbol{J}_l$ 事实上就是式 (4.27) 的内容：

$$\boldsymbol{J}_l=\boldsymbol{J}=\frac{\sin\theta}{\theta}\boldsymbol{I}+\left(1-\frac{\sin\theta}{\theta}\right)\boldsymbol{a}\boldsymbol{a}^{\mathrm{T}}+\frac{1-\cos\theta}{\theta}\boldsymbol{a}^\wedge. \tag{4.31}$$

它的逆为

$$\boldsymbol{J}_l^{-1}=\frac{\theta}{2}\cot\frac{\theta}{2}\boldsymbol{I}+\left(1-\frac{\theta}{2}\cot\frac{\theta}{2}\right)\boldsymbol{a}\boldsymbol{a}^{\mathrm{T}}-\frac{\theta}{2}\boldsymbol{a}^\wedge. \tag{4.32}$$

而右乘雅可比仅需要对自变量取负号即可：

$$\boldsymbol{J}_r(\boldsymbol{\phi})=\boldsymbol{J}_l(-\boldsymbol{\phi}). \tag{4.33}$$

这样，我们就可以谈论李群乘法与李代数加法的关系了。

为了方便读者理解，我们重新叙述 BCH 近似的意义。假定对某个旋转 $\boldsymbol{R}$，对应的李代数为

---

①参见`https://en.wikipedia.org/wiki/Baker-Campbell-Hausdorff_formula`。

②对 BCH 具体形式和近似表达的具体推导，本书不作讨论，请参考文献 [6]。

$\boldsymbol{\phi}$。我们给它左乘一个微小旋转，记作 $\Delta\boldsymbol{R}$，对应的李代数为 $\Delta\boldsymbol{\phi}$。那么，在李群上，得到的结果就是 $\Delta\boldsymbol{R}\cdot\boldsymbol{R}$，而在李代数上，根据 BCH 近似，为 $\boldsymbol{J}_l^{-1}(\boldsymbol{\phi})\Delta\boldsymbol{\phi}+\boldsymbol{\phi}$。合并起来，可以简单地写成：

$$\exp\left(\Delta\boldsymbol{\phi}^{\wedge}\right)\exp\left(\boldsymbol{\phi}^{\wedge}\right)=\exp\left(\left(\boldsymbol{\phi}+\boldsymbol{J}_l^{-1}\left(\boldsymbol{\phi}\right)\Delta\boldsymbol{\phi}\right)^{\wedge}\right). \tag{4.34}$$

反之，如果我们在李代数上进行加法，让一个 $\boldsymbol{\phi}$ 加上 $\Delta\boldsymbol{\phi}$，那么可以近似为李群上带左右雅可比的乘法：

$$\exp\left(\left(\boldsymbol{\phi}+\Delta\boldsymbol{\phi}\right)^{\wedge}\right)=\exp\left(\left(\boldsymbol{J}_l\Delta\boldsymbol{\phi}\right)^{\wedge}\right)\exp\left(\boldsymbol{\phi}^{\wedge}\right)=\exp\left(\boldsymbol{\phi}^{\wedge}\right)\exp\left(\left(\boldsymbol{J}_r\Delta\boldsymbol{\phi}\right)^{\wedge}\right). \tag{4.35}$$

这就为之后李代数上做微积分提供了理论基础。同样地，对于 SE(3)，也有类似的 BCH 近似：

$$\exp\left(\Delta\boldsymbol{\xi}^{\wedge}\right)\exp\left(\boldsymbol{\xi}^{\wedge}\right)\approx\exp\left(\left(\boldsymbol{\mathcal{J}}_l^{-1}\Delta\boldsymbol{\xi}+\boldsymbol{\xi}\right)^{\wedge}\right), \tag{4.36}$$

$$\exp\left(\boldsymbol{\xi}^{\wedge}\right)\exp\left(\Delta\boldsymbol{\xi}^{\wedge}\right)\approx\exp\left(\left(\boldsymbol{\mathcal{J}}_r^{-1}\Delta\boldsymbol{\xi}+\boldsymbol{\xi}\right)^{\wedge}\right). \tag{4.37}$$

这里 $\boldsymbol{\mathcal{J}}_l$ 形式比较复杂，它是一个 6×6 的矩阵，读者可以参考文献 [6] 中式 (7.82) 和 (7.83) 的内容。由于我们在计算中没有用到该雅可比，故这里略去它的实际形式。

4.3.2　SO(3) 上的李代数求导

下面来讨论一个带有李代数的函数，以及关于该李代数求导的问题。该问题有很强的实际背景。在 SLAM 中，我们要估计一个相机的位置和姿态，该位姿是由 SO(3) 上的旋转矩阵或 SE(3) 上的变换矩阵描述的。不妨设某个时刻小萝卜的位姿为 $\boldsymbol{T}$。它观察到了一个世界坐标位于 $\boldsymbol{p}$ 的点，产生了一个观测数据 $\boldsymbol{z}$。那么，由坐标变换关系知：

$$\boldsymbol{z}=\boldsymbol{T}\boldsymbol{p}+\boldsymbol{w}. \tag{4.38}$$

其中 $\boldsymbol{w}$ 为随机噪声。由于它的存在，$\boldsymbol{z}$ 往往不可能精确地满足 $\boldsymbol{z}=\boldsymbol{T}\boldsymbol{p}$ 的关系。所以，我们通常会计算理想的观测与实际数据的误差：

$$\boldsymbol{e}=\boldsymbol{z}-\boldsymbol{T}\boldsymbol{p}. \tag{4.39}$$

假设一共有 N 个这样的路标点和观测，于是就有 N 个上式。那么，对小萝卜进行位姿估计，相当于寻找一个最优的 $\boldsymbol{T}$，使得整体误差最小化：

$$\min_{\boldsymbol{T}}J(\boldsymbol{T})=\sum_{i=1}^{N}\left\|\boldsymbol{z}_i-\boldsymbol{T}\boldsymbol{p}_i\right\|_2^2. \tag{4.40}$$

求解此问题，需要计算目标函数 J 关于变换矩阵 $\boldsymbol{T}$ 的导数。我们把具体的算法留到后面再讲。这里的重点是，**我们经常会构建与位姿有关的函数，然后讨论该函数关于位姿的导数，以调整当前的估计值**。然而，SO(3), SE(3) 上并没有良好定义的加法，它们只是群。如果我们把 $\boldsymbol{T}$ 当成一个普通矩阵来处理优化，就必须对它加以约束。而从李代数角度来说，由于李代数由向量组成，具有良好的加法运算，因此，使用李代数解决求导问题的思路分为两种：

1. 用李代数表示姿态，然后根据李代数加法对**李代数求导**。
2. 对李群**左乘**或**右乘**微小扰动，然后对该**扰动求导**，称为左扰动和右扰动模型。

第一种方式对应到李代数的求导模型，而第二种方式则对应到扰动模型。下面讨论这两种思路的异同。

4.3.3 李代数求导

首先，考虑 SO(3) 上的情况。假设我们对一个空间点 $\boldsymbol{p}$ 进行了旋转，得到了 $\boldsymbol{Rp}$。现在，要计算旋转之后点的坐标相对于旋转的导数，我们非正式地记为[①]

$$\frac{\partial(\boldsymbol{Rp})}{\partial \boldsymbol{R}}.$$

由于 SO(3) 没有加法，所以该导数无法按照导数的定义进行计算。设 $\boldsymbol{R}$ 对应的李代数为 $\boldsymbol{\phi}$，我们转而计算[②]：

$$\frac{\partial(\exp(\boldsymbol{\phi}^\wedge)\boldsymbol{p})}{\partial \boldsymbol{\phi}}.$$

按照导数的定义，有

$$\begin{aligned}
\frac{\partial(\exp(\boldsymbol{\phi}^\wedge)\boldsymbol{p})}{\partial \boldsymbol{\phi}} &= \lim_{\delta\boldsymbol{\phi}\to\boldsymbol{0}} \frac{\exp\left((\boldsymbol{\phi}+\delta\boldsymbol{\phi})^\wedge\right)\boldsymbol{p} - \exp(\boldsymbol{\phi}^\wedge)\boldsymbol{p}}{\delta\boldsymbol{\phi}} \\
&= \lim_{\delta\boldsymbol{\phi}\to\boldsymbol{0}} \frac{\exp\left((\boldsymbol{J}_l\delta\boldsymbol{\phi})^\wedge\right)\exp(\boldsymbol{\phi}^\wedge)\boldsymbol{p} - \exp(\boldsymbol{\phi}^\wedge)\boldsymbol{p}}{\delta\boldsymbol{\phi}} \\
&= \lim_{\delta\boldsymbol{\phi}\to\boldsymbol{0}} \frac{\left(\boldsymbol{I}+(\boldsymbol{J}_l\delta\boldsymbol{\phi})^\wedge\right)\exp(\boldsymbol{\phi}^\wedge)\boldsymbol{p} - \exp(\boldsymbol{\phi}^\wedge)\boldsymbol{p}}{\delta\boldsymbol{\phi}} \\
&= \lim_{\delta\boldsymbol{\phi}\to\boldsymbol{0}} \frac{(\boldsymbol{J}_l\delta\boldsymbol{\phi})^\wedge\exp(\boldsymbol{\phi}^\wedge)\boldsymbol{p}}{\delta\boldsymbol{\phi}} \\
&= \lim_{\delta\boldsymbol{\phi}\to\boldsymbol{0}} \frac{-(\exp(\boldsymbol{\phi}^\wedge)\boldsymbol{p})^\wedge\boldsymbol{J}_l\delta\boldsymbol{\phi}}{\delta\boldsymbol{\phi}} = -(\boldsymbol{Rp})^\wedge\boldsymbol{J}_l.
\end{aligned}$$

①请注意这里并不能按照矩阵微分来定义导数，这只是一个记号。

②严格说来，在矩阵微分中，只能求行向量关于列向量的导数，所得结果是一个矩阵。但本书写成列向量对列向量的导数，读者可以认为先对分子进行转置，再对最后结果进行转置。这使得式子变得简洁，不然我们就不得不给每一行的分子加一个转置符号。在这种意义下，可以认为 $\mathrm{d}(\boldsymbol{Ax})/\mathrm{d}\boldsymbol{x} = \boldsymbol{A}$。

第 2 行的近似为 BCH 线性近似，第 3 行为泰勒展开舍去高阶项后的近似（由于取了极限，可以写等号），第 4 行至第 5 行将反对称符号看作叉积，交换之后变号。于是，我们推导出了旋转后的点相对于李代数的导数：

$$\frac{\partial (\boldsymbol{Rp})}{\partial \boldsymbol{\phi}} = (-\boldsymbol{Rp})^{\wedge} \boldsymbol{J}_l. \tag{4.41}$$

不过，由于这里仍然含有形式比较复杂的 $\boldsymbol{J}_l$，我们不太希望计算它。而下面要讲的扰动模型则提供了更简单的导数计算方式。

4.3.4　扰动模型（左乘）

另一种求导方式是对 $\boldsymbol{R}$ 进行一次扰动 $\Delta \boldsymbol{R}$，看结果相对于扰动的变化率。这个扰动可以乘在左边也可以乘在右边，最后结果会有一点儿微小的差异，我们以左扰动为例。设左扰动 $\Delta \boldsymbol{R}$ 对应的李代数为 $\boldsymbol{\varphi}$。然后，对 $\boldsymbol{\varphi}$ 求导，即

$$\frac{\partial (\boldsymbol{Rp})}{\partial \boldsymbol{\varphi}} = \lim_{\boldsymbol{\varphi} \to \boldsymbol{0}} \frac{\exp(\boldsymbol{\varphi}^{\wedge}) \exp(\boldsymbol{\phi}^{\wedge}) \boldsymbol{p} - \exp(\boldsymbol{\phi}^{\wedge}) \boldsymbol{p}}{\boldsymbol{\varphi}}. \tag{4.42}$$

该式的求导比上面更简单：

$$\begin{aligned} \frac{\partial (\boldsymbol{Rp})}{\partial \boldsymbol{\varphi}} &= \lim_{\boldsymbol{\varphi} \to \boldsymbol{0}} \frac{\exp(\boldsymbol{\varphi}^{\wedge}) \exp(\boldsymbol{\phi}^{\wedge}) \boldsymbol{p} - \exp(\boldsymbol{\phi}^{\wedge}) \boldsymbol{p}}{\boldsymbol{\varphi}} \\ &= \lim_{\boldsymbol{\varphi} \to \boldsymbol{0}} \frac{(\boldsymbol{I} + \boldsymbol{\varphi}^{\wedge}) \exp(\boldsymbol{\phi}^{\wedge}) \boldsymbol{p} - \exp(\boldsymbol{\phi}^{\wedge}) \boldsymbol{p}}{\boldsymbol{\varphi}} \\ &= \lim_{\boldsymbol{\varphi} \to \boldsymbol{0}} \frac{\boldsymbol{\varphi}^{\wedge} \boldsymbol{Rp}}{\boldsymbol{\varphi}} = \lim_{\boldsymbol{\varphi} \to \boldsymbol{0}} \frac{-(\boldsymbol{Rp})^{\wedge} \boldsymbol{\varphi}}{\boldsymbol{\varphi}} = -(\boldsymbol{Rp})^{\wedge}. \end{aligned}$$

可见，相比于直接对李代数求导，省去了一个雅可比 $\boldsymbol{J}_l$ 的计算。这使得扰动模型更为实用。请读者务必理解这里的求导运算，这在位姿估计中具有重要的意义。

4.3.5　SE(3) 上的李代数求导

最后，我们给出 SE(3) 上的扰动模型，而直接李代数上的求导就不再介绍了。假设某空间点 $\boldsymbol{p}$ 经过一次变换 $\boldsymbol{T}$（对应李代数为 $\boldsymbol{\xi}$），得到 $\boldsymbol{Tp}$[①]。现在，给 $\boldsymbol{T}$ 左乘一个扰动 $\Delta \boldsymbol{T} = \exp(\delta \boldsymbol{\xi}^{\wedge})$，我们设扰动项的李代数为 $\delta \boldsymbol{\xi} = [\delta \boldsymbol{\rho}, \delta \boldsymbol{\phi}]^{\mathrm{T}}$，那么：

$$\frac{\partial (\boldsymbol{Tp})}{\partial \delta \boldsymbol{\xi}} = \lim_{\delta \boldsymbol{\xi} \to \boldsymbol{0}} \frac{\exp(\delta \boldsymbol{\xi}^{\wedge}) \exp(\boldsymbol{\xi}^{\wedge}) \boldsymbol{p} - \exp(\boldsymbol{\xi}^{\wedge}) \boldsymbol{p}}{\delta \boldsymbol{\xi}}$$

①请注意为了使乘法成立，$\boldsymbol{p}$ 必须使用齐次坐标。

$$
\begin{aligned}
&= \lim_{\delta\boldsymbol{\xi}\to\mathbf{0}} \frac{(\boldsymbol{I}+\delta\boldsymbol{\xi}^\wedge)\exp(\boldsymbol{\xi}^\wedge)\boldsymbol{p} - \exp(\boldsymbol{\xi}^\wedge)\boldsymbol{p}}{\delta\boldsymbol{\xi}} \\
&= \lim_{\delta\boldsymbol{\xi}\to\mathbf{0}} \frac{\delta\boldsymbol{\xi}^\wedge \exp(\boldsymbol{\xi}^\wedge)\boldsymbol{p}}{\delta\boldsymbol{\xi}} \\
&= \lim_{\delta\boldsymbol{\xi}\to\mathbf{0}} \frac{\begin{bmatrix}\delta\boldsymbol{\phi}^\wedge & \delta\boldsymbol{\rho}\\ \mathbf{0}^\mathrm{T} & 0\end{bmatrix}\begin{bmatrix}\boldsymbol{R}\boldsymbol{p}+\boldsymbol{t}\\ 1\end{bmatrix}}{\delta\boldsymbol{\xi}} \\
&= \lim_{\delta\boldsymbol{\xi}\to\mathbf{0}} \frac{\begin{bmatrix}\delta\boldsymbol{\phi}^\wedge(\boldsymbol{R}\boldsymbol{p}+\boldsymbol{t})+\delta\boldsymbol{\rho}\\ \mathbf{0}^\mathrm{T}\end{bmatrix}}{[\delta\boldsymbol{\rho},\delta\boldsymbol{\phi}]^\mathrm{T}} = \begin{bmatrix}\boldsymbol{I} & -(\boldsymbol{R}\boldsymbol{p}+\boldsymbol{t})^\wedge\\ \mathbf{0}^\mathrm{T} & \mathbf{0}^\mathrm{T}\end{bmatrix} \overset{\text{def}}{=} (\boldsymbol{T}\boldsymbol{p})^\odot.
\end{aligned}
$$

我们把最后的结果定义成一个算符 $\odot$[①]，它把一个齐次坐标的空间点变换成一个 4×6 的矩阵。此式稍微需要解释的是矩阵求导方面的顺序，假设 $\boldsymbol{a},\boldsymbol{b},\boldsymbol{x},\boldsymbol{y}$ 都是列向量，那么在我们的符号写法下，有如下的规则：

$$
\frac{\mathrm{d}\begin{bmatrix}\boldsymbol{a}\\ \boldsymbol{b}\end{bmatrix}}{\mathrm{d}\begin{bmatrix}\boldsymbol{x}\\ \boldsymbol{y}\end{bmatrix}} = \left(\frac{\mathrm{d}[\boldsymbol{a},\boldsymbol{b}]^\mathrm{T}}{\mathrm{d}\begin{bmatrix}\boldsymbol{x}\\ \boldsymbol{y}\end{bmatrix}}\right)^\mathrm{T} = \begin{bmatrix}\frac{\mathrm{d}\boldsymbol{a}}{\mathrm{d}\boldsymbol{x}} & \frac{\mathrm{d}\boldsymbol{b}}{\mathrm{d}\boldsymbol{x}}\\ \frac{\mathrm{d}\boldsymbol{a}}{\mathrm{d}\boldsymbol{y}} & \frac{\mathrm{d}\boldsymbol{b}}{\mathrm{d}\boldsymbol{y}}\end{bmatrix}^\mathrm{T} = \begin{bmatrix}\frac{\mathrm{d}\boldsymbol{a}}{\mathrm{d}\boldsymbol{x}} & \frac{\mathrm{d}\boldsymbol{a}}{\mathrm{d}\boldsymbol{y}}\\ \frac{\mathrm{d}\boldsymbol{b}}{\mathrm{d}\boldsymbol{x}} & \frac{\mathrm{d}\boldsymbol{b}}{\mathrm{d}\boldsymbol{y}}\end{bmatrix} \tag{4.43}
$$

至此，我们已经介绍了李群与李代数上的微分运算。之后的章节中，我们将应用这些知识解决实际问题。关于李群与李代数的某些重要数学性质，我们作为习题留给读者。

4.4 实践：Sophus

4.4.1 Sophus 的基本使用方法

我们已经介绍了李代数的入门知识，现在是通过实践演练并巩固所学知识的时候了。我们来讨论如何在程序中操作李代数。在第 3 讲中，我们看到 Eigen 提供了几何模块，但没有提供李代数的支持。一个较好的李代数库是 Strasdat 维护的 Sophus 库[②]。Sophus 库支持本章主要讨论的 SO(3) 和 SE(3)，此外，还含有二维运动 SO(2), SE(2) 及相似变换 Sim(3) 的内容。它是直接在 Eigen 基

①笔者会读作“咚”，像一个石子掉在井里的声音。

②最早提出李代数的是 Sophus Lie，这个库就以他的名字命名了。

础上开发的，我们不需要安装额外的依赖库。读者可以直接从 GitHub 上获取 Sophus，在本书的代码目录 slambook/3rdparty 下也提供了 Sophus 源代码。由于历史原因，Sophus 早期版本只提供了双精度的李群/李代数类。后续版本改写成了模板类。模板类的 Sophus 中可以使用不同精度的李群/李代数，但同时增加了使用难度。在本书中，我们使用**带模板**的 Sophus 库。本书的 3rdparty 中提供的 Sophus 是**模板**版本，它应该在你下载本书代码的时候就已经复制下来了。Sophus 本身也是一个 cmake 工程。想必你已经了解如何编译 cmake 工程了，这里不再赘述。Sophus 库只需编译即可，无须安装。

下面来演示 Sophus 库中的 SO(3) 和 SE(3) 运算：

slambook/ch4/useSophus.cpp

```
#include <iostream>
#include <cmath>
#include <Eigen/Core>
#include <Eigen/Geometry>
#include "sophus/se3.hpp"

using namespace std;
using namespace Eigen;

/// 本程序演示sophus的基本用法
int main(int argc, char **argv) {
    // 沿Z轴转90度的旋转矩阵
    Matrix3d R = AngleAxisd(M_PI / 2, Vector3d(0, 0, 1)).toRotationMatrix();
    // 或者四元数
    Quaterniond q(R);
    Sophus::SO3d SO3_R(R);              // Sophus::SO3d可以直接从旋转矩阵构造
    Sophus::SO3d SO3_q(q);              // 也可以通过四元数构造
    // 二者是等价的
    cout << "SO(3) from matrix:\n" << SO3_R.matrix() << endl;
    cout << "SO(3) from quaternion:\n" << SO3_q.matrix() << endl;
    cout << "they are equal" << endl;

    // 使用对数映射获得它的李代数
    Vector3d so3 = SO3_R.log();
    cout << "so3 = " << so3.transpose() << endl;
    // hat为向量到反对称矩阵
    cout << "so3 hat=\n" << Sophus::SO3d::hat(so3) << endl;
    // 相对的，vee为反对称到向量
    cout << "so3 hat vee= " << Sophus::SO3d::vee(Sophus::SO3d::hat(so3)).transpose() << endl;

    // 增量扰动模型的更新
```

```
    Vector3d update_so3(1e-4, 0, 0); //假设更新量为这么多
    Sophus::SO3d SO3_updated = Sophus::SO3d::exp(update_so3) * SO3_R;
    cout << "SO3 updated = \n" << SO3_updated.matrix() << endl;

    cout << "*******************************" << endl;
    // 对SE(3)操作大同小异
    Vector3d t(1, 0, 0);           // 沿X轴平移1
    Sophus::SE3d SE3_Rt(R, t);           // 从R,t构造SE(3)
    Sophus::SE3d SE3_qt(q, t);            // 从q,t构造SE(3)
    cout << "SE3 from R,t= \n" << SE3_Rt.matrix() << endl;
    cout << "SE3 from q,t= \n" << SE3_qt.matrix() << endl;
    // 李代数se(3)是一个六维向量，方便起见先typedef一下
    typedef Eigen::Matrix<double, 6, 1> Vector6d;
    Vector6d se3 = SE3_Rt.log();
    cout << "se3 = " << se3.transpose() << endl;
    // 观察输出，会发现在Sophus中，se(3)的平移在前，旋转在后.
    // 同样地，有hat和vee两个算符
    cout << "se3 hat = \n" << Sophus::SE3d::hat(se3) << endl;
    cout << "se3 hat vee = " << Sophus::SE3d::vee(Sophus::SE3d::hat(se3)).transpose() << endl
    ;

    // 最后，演示更新
    Vector6d update_se3; //更新量
    update_se3.setZero();
    update_se3(0, 0) = 1e-4d;
    Sophus::SE3d SE3_updated = Sophus::SE3d::exp(update_se3) * SE3_Rt;
    cout << "SE3 updated = " << endl << SE3_updated.matrix() << endl;

    return 0;
}
```

该演示程序分为两部分。前半部分介绍 SO(3) 上的操作，后半部分则为 SE(3)。我们演示了如何构造 SO(3), SE(3) 对象，对它们进行指数、对数映射，以及当知道更新量后，如何对李群元素进行更新。如果读者切实理解了本讲内容，那么这个程序对你来说应该没有什么难度。为了编译它，请在 CMakeLists.txt 里添加以下几行代码：

slambook/ch4/useSophus/CMakeLists.txt

```
# 为使用sophus，需要使用find_package命令找到它
find_package( Sophus REQUIRED )
include_directories( ${Sophus_INCLUDE_DIRS} )

add_executable( useSophus useSophus.cpp )
```

find_package 命令是 cmake 提供的寻找某个库的头文件与库文件的指令。如果 cmake 能够找到它，就会提供头文件和库文件所在的目录的变量。在 Sophus 这个例子中，就是 Sophus_INCLUDE_DIRS。基于模板的 Sophus 库和 Eigen 一样，是仅含头文件而没有源文件的。根据它们，我们就能将 Sophus 库引入自己的 cmake 工程。请读者自行查看此程序的输出信息，它与我们之前的推导是一致的。

4.4.2　例子：评估轨迹的误差

在实际工程中，我们经常需要评估一个算法的估计轨迹与真实轨迹的差异来评价算法的精度。真实轨迹往往通过某些更高精度的系统获得，而估计轨迹则是由待评价的算法计算得到的。第 3 讲我们演示了如何显示存储在文件中的某条轨迹，本节我们考虑如何计算两条轨迹的误差。考虑一条估计轨迹 $\boldsymbol{T}_{\text{esti},i}$ 和真实轨迹 $\boldsymbol{T}_{\text{gt},i}$，其中 $i=1,\cdots,N$，那么我们可以定义一些误差指标来描述它们之间的差别。

误差指标可以有很多种，常见的有**绝对轨迹误差**（Absolute Trajectory Error，ATE），形如：

$$\text{ATE}_{\text{all}}=\sqrt{\frac{1}{N}\sum_{i=1}^{N}\|\log(\boldsymbol{T}_{\text{gt},i}^{-1}\boldsymbol{T}_{\text{esti},i})^{\vee}\|_2^2}, \tag{4.44}$$

这实际上是每个位姿李代数的**均方根误差**（Root-Mean-Squared Error，RMSE）。这种误差可以刻画两条轨迹的旋转和平移误差。同时，也有的文献仅考虑平移误差[23]，从而可以定义**绝对平移误差**（Average Translational Error）：

$$\text{ATE}_{\text{trans}}=\sqrt{\frac{1}{N}\sum_{i=1}^{N}\|\text{trans}(\boldsymbol{T}_{\text{gt},i}^{-1}\boldsymbol{T}_{\text{esti},i})\|_2^2}, \tag{4.45}$$

其中 trans 表示取括号内部变量的平移部分。因为从整条轨迹上看，旋转出现误差后，随后的轨迹在平移上也会出现误差，所以两种指标在实际中都适用。

除此之外，也可以定义相对的误差。例如，考虑 i 时刻到 $i+\Delta t$ 时刻的运动，那么相对位姿误差（Relative Pose Error，RPE）可定义为

$$\text{RPE}_{\text{all}}=\sqrt{\frac{1}{N-\Delta t}\sum_{i=1}^{N-\Delta t}\|\log\left(\left(\boldsymbol{T}_{\text{gt},i}^{-1}\boldsymbol{T}_{\text{gt},i+\Delta t}\right)^{-1}\left(\boldsymbol{T}_{\text{esti},i}^{-1}\boldsymbol{T}_{\text{esti},i+\Delta t}\right)\right)^{\vee}\|_2^2}, \tag{4.46}$$

同样地，也可只取平移部分：

$$\mathrm{RPE}_{\mathrm{trans}} = \sqrt{\frac{1}{N-\Delta t}\sum_{i=1}^{N-\Delta t} \left\| \mathrm{trans}\left(\left(\boldsymbol{T}_{\mathrm{gt},i}^{-1} \boldsymbol{T}_{\mathrm{gt},i+\Delta t} \right)^{-1} \left(\boldsymbol{T}_{\mathrm{esti},i}^{-1} \boldsymbol{T}_{\mathrm{esti},i+\Delta t} \right) \right) \right\|_2^2}. \tag{4.47}$$

利用 Sophus 库，很容易实现这部分计算。下面我们演示绝对轨迹误差的计算。在这个例子中，我们有 groundtruth.txt 和 estimated.txt 两条轨迹，下面的代码将读取这两条轨迹，计算误差，然后显示到 3D 窗口中。为简洁起见，省略了画轨迹部分的代码，在第 3 讲中我们已经做过类似的工作。

slambook/ch4/example/trajectoryError.cpp（部分）

```
#include <iostream>
#include <fstream>
#include <unistd.h>
#include <pangolin/pangolin.h>
#include <sophus/se3.hpp>

using namespace Sophus;
using namespace std;

string groundtruth_file = "./example/groundtruth.txt";
string estimated_file = "./example/estimated.txt";

typedef vector<Sophus::SE3d, Eigen::aligned_allocator<Sophus::SE3d>> TrajectoryType;

void DrawTrajectory(const TrajectoryType &gt, const TrajectoryType &esti);

TrajectoryType ReadTrajectory(const string &path);

int main(int argc, char **argv) {
    TrajectoryType groundtruth = ReadTrajectory(groundtruth_file);
    TrajectoryType estimated = ReadTrajectory(estimated_file);
    assert(!groundtruth.empty() && !estimated.empty());
    assert(groundtruth.size() == estimated.size());

    // compute rmse
    double rmse = 0;
    for (size_t i = 0; i < estimated.size(); i++) {
        Sophus::SE3d p1 = estimated[i], p2 = groundtruth[i];
        double error = (p2.inverse() * p1).log().norm();
        rmse += error * error;
    }
    rmse = rmse / double(estimated.size());
    rmse = sqrt(rmse);
```

```
    cout << "RMSE = " << rmse << endl;

    DrawTrajectory(groundtruth, estimated);
    return 0;
}

TrajectoryType ReadTrajectory(const string &path) {
    ifstream fin(path);
    TrajectoryType trajectory;
    if (!fin) {
        cerr << "trajectory " << path << " not found." << endl;
        return trajectory;
    }

    while (!fin.eof()) {
        double time, tx, ty, tz, qx, qy, qz, qw;
        fin >> time >> tx >> ty >> tz >> qx >> qy >> qz >> qw;
        Sophus::SE3d p1(Eigen::Quaterniond(qx, qy, qz, qw), Eigen::Vector3d(tx, ty, tz));
        trajectory.push_back(p1);
    }
    return trajectory;
}
```

该程序输出的结果为 2.207，图像如图 4-2 所示。读者也可以尝试将旋转部分去掉，仅计算平移部分的误差。就这个例子来说，我们事实上已经帮助读者做了一些预处理任务，包括轨迹的时间对齐、外参预估，这些内容现在还没有讲到，我们将在以后的学习中谈论。

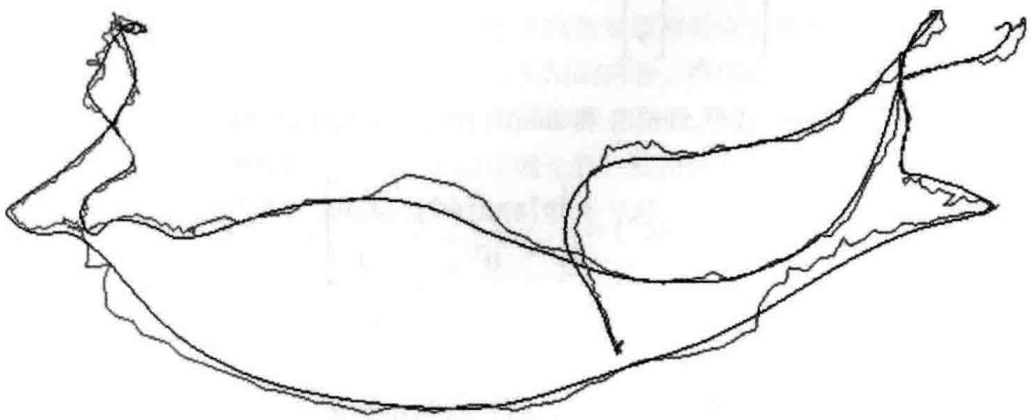


图 4-2　计算估计轨迹与真实轨迹之间的误差

4.5 * 相似变换群与李代数

最后，我们要提一下在单目视觉中使用的相似变换群 Sim(3)，以及对应的李代数 $\mathfrak{sim}(3)$。如果你只对双目 SLAM 或 RGB-D SLAM 感兴趣，可以跳过本节。

我们已经介绍过单目的尺度不确定性。如果在单目 SLAM 中使用 SE(3) 表示位姿，那么由于尺度不确定性与尺度漂移，整个 SLAM 过程中的尺度会发生变化，这在 SE(3) 中未能体现出来。因此，在单目情况下我们一般会显式地把尺度因子表达出来。用数学语言来说，对于位于空间的点 $\boldsymbol{p}$，在相机坐标系下要经过一个**相似变换**，而非欧氏变换：

$$\boldsymbol{p}' = \begin{bmatrix} s\boldsymbol{R} & \boldsymbol{t} \\ \boldsymbol{0}^{\mathrm{T}} & 1 \end{bmatrix} \boldsymbol{p} = s\boldsymbol{R}\boldsymbol{p} + \boldsymbol{t}. \tag{4.48}$$

在相似变换中，我们把尺度 s 表达出来了。它同时作用在 $\boldsymbol{p}$ 的 3 个坐标之上，对 $\boldsymbol{p}$ 进行了一次缩放。与 SO(3)、SE(3) 相似，相似变换也对矩阵乘法构成群，称为相似变换群 Sim(3)：

$$\mathrm{Sim}(3) = \left\{ \boldsymbol{S} = \begin{bmatrix} s\boldsymbol{R} & \boldsymbol{t} \\ \boldsymbol{0}^{\mathrm{T}} & 1 \end{bmatrix} \in \mathbb{R}^{4\times 4} \right\}. \tag{4.49}$$

同样地，Sim(3) 也有对应的李代数、指数映射、对数映射等。李代数 $\mathfrak{sim}(3)$ 元素是一个 7 维向量 $\boldsymbol{\zeta}$。它的前 6 维与 $\mathfrak{se}(3)$ 相同，最后多了一项 σ。

$$\mathfrak{sim}(3) = \left\{ \boldsymbol{\zeta} | \boldsymbol{\zeta} = \begin{bmatrix} \boldsymbol{\rho} \\ \boldsymbol{\phi} \\ \sigma \end{bmatrix} \in \mathbb{R}^7, \boldsymbol{\zeta}^\wedge = \begin{bmatrix} \sigma\boldsymbol{I} + \boldsymbol{\phi}^\wedge & \boldsymbol{\rho} \\ \boldsymbol{0}^{\mathrm{T}} & 0 \end{bmatrix} \in \mathbb{R}^{4\times 4} \right\}. \tag{4.50}$$

它比 $\mathfrak{se}(3)$ 多了一项 σ。关联 Sim(3) 和 $\mathfrak{sim}(3)$ 的仍是指数映射和对数映射。指数映射为

$$\exp\left(\boldsymbol{\zeta}^\wedge\right) = \begin{bmatrix} \mathrm{e}^\sigma \exp\left(\boldsymbol{\phi}^\wedge\right) & \boldsymbol{J}_s\boldsymbol{\rho} \\ \boldsymbol{0}^{\mathrm{T}} & 1 \end{bmatrix}. \tag{4.51}$$

其中，$\boldsymbol{J}_s$ 的形式为

$$\begin{aligned} \boldsymbol{J}_s = & \frac{\mathrm{e}^\sigma - 1}{\sigma}\boldsymbol{I} + \frac{\sigma\mathrm{e}^\sigma \sin\theta + (1 - \mathrm{e}^\sigma \cos\theta)\,\theta}{\sigma^2 + \theta^2}\boldsymbol{a}^\wedge \\ & + \left(\frac{\mathrm{e}^\sigma - 1}{\sigma} - \frac{(\mathrm{e}^\sigma \cos\theta - 1)\,\sigma + (\mathrm{e}^\sigma \sin\theta)\theta}{\sigma^2 + \theta^2} \right) \boldsymbol{a}^\wedge \boldsymbol{a}^\wedge. \end{aligned}$$

通过指数映射，我们能够找到李代数与李群的关系。对于李代数 $\boldsymbol{\zeta}$，它与李群的对应关系为

$$s = \mathrm{e}^{\sigma},\ \boldsymbol{R} = \exp(\boldsymbol{\phi}^{\wedge}),\ \boldsymbol{t} = \boldsymbol{J}_s\boldsymbol{\rho}. \tag{4.52}$$

旋转部分和 SO(3) 是一致的。平移部分，在 $\mathfrak{se}(3)$ 中需要乘一个雅可比 $\boldsymbol{\mathcal{J}}$，而相似变换的雅可比更复杂。对于尺度因子，可以看到李群中的 s 即为李代数中 σ 的指数函数。

Sim(3) 的 BCH 近似与 SE(3) 是类似的。我们可以讨论一个点 $\boldsymbol{p}$ 经过相似变换 $\boldsymbol{Sp}$ 后，相对于 $\boldsymbol{S}$ 的导数。同样地，存在微分模型和扰动模型两种方式，而扰动模型较为简单。我们省略推导过程，直接给出扰动模型的结果。设给予 $\boldsymbol{Sp}$ 左侧一个小扰动 $\exp(\boldsymbol{\zeta}^{\wedge})$，并求 $\boldsymbol{Sp}$ 对于扰动的导数。因为 $\boldsymbol{Sp}$ 是 4 维的齐次坐标，$\boldsymbol{\zeta}$ 是 7 维向量，所以该导数应该是 4×7 的雅可比。方便起见，记 $\boldsymbol{Sp}$ 的前 3 维组成向量为 $\boldsymbol{q}$，那么：

$$\frac{\partial \boldsymbol{Sp}}{\partial \boldsymbol{\zeta}} = \begin{bmatrix} \boldsymbol{I} & -\boldsymbol{q}^{\wedge} & \boldsymbol{q} \\ \boldsymbol{0}^{\mathrm{T}} & \boldsymbol{0}^{\mathrm{T}} & 0 \end{bmatrix}. \tag{4.53}$$

关于 Sim(3)，我们就介绍到这里。更详细的关于 Sim(3) 的资料，建议读者参见文献 [24]。

4.6　小结

本讲引入了李群 SO(3) 和 SE(3)，以及它们对应的李代数 $\mathfrak{so}(3)$ 和 $\mathfrak{se}(3)$。我们介绍了位姿在它们上面的表达和转换，然后通过 BCH 的线性近似，就可以对位姿进行扰动并求导。这给之后讲解位姿的优化打下了理论基础，因为我们需要经常对某一个位姿的估计值进行调整，使它对应的误差减小。只有在弄清楚如何对位姿进行调整和更新之后，我们才能继续下一步的内容。

本讲的内容可能比较偏理论，毕竟它不像计算机视觉那样经常有好看的图片展示。与讲解李群与李代数的数学教科书相比，由于我们只关心实用的内容，所以讲的过程非常精简，速度也相对快。本讲内容是解决后续许多问题的基础，请读者务必理解，特别是位姿估计部分。

值得一提的是，除了李代数，同样也可以用四元数、欧拉角等方式表示旋转，只是后续的处理要麻烦一些。在实际应用中，也可以使用 SO(3) 加上平移的方式来代替 SE(3)，从而回避一些雅可比的计算。

习题

1. 验证 SO(3)、SE(3) 和 Sim(3) 关于乘法成群。
2. 验证 $(\mathbb{R}^3, \mathbb{R}, \times)$ 构成李代数。

3. 验证 $\mathfrak{so}(3)$ 和 $\mathfrak{se}(3)$ 满足李代数要求的性质。
4. 验证性质 (4.20) 和 (4.21)。
5. 证明：

$$\boldsymbol{R}\boldsymbol{p}^{\wedge}\boldsymbol{R}^{\mathrm{T}} = (\boldsymbol{R}\boldsymbol{p})^{\wedge}.$$

6. 证明：

$$\boldsymbol{R}\exp(\boldsymbol{p}^{\wedge})\boldsymbol{R}^{\mathrm{T}} = \exp((\boldsymbol{R}\boldsymbol{p})^{\wedge}).$$

该式称为 SO(3) 上的**伴随**性质。同样地，在 SE(3) 上也有伴随性质：

$$\boldsymbol{T}\exp(\boldsymbol{\xi}^{\wedge})\boldsymbol{T}^{-1} = \exp\left((\mathrm{Ad}(\boldsymbol{T})\boldsymbol{\xi})^{\wedge}\right), \tag{4.54}$$

其中：

$$\mathrm{Ad}(\boldsymbol{T}) = \begin{bmatrix} \boldsymbol{R} & \boldsymbol{t}^{\wedge}\boldsymbol{R} \\ \boldsymbol{0} & \boldsymbol{R} \end{bmatrix}. \tag{4.55}$$

7. 仿照左扰动的推导，推导 SO(3) 和 SE(3) 在右扰动下的导数。
8. 搜索 cmake 的 find_package 指令是如何运作的。它有哪些可选的参数？为了让 cmake 找到某个库，需要哪些先决条件？

第5讲 相机与图像

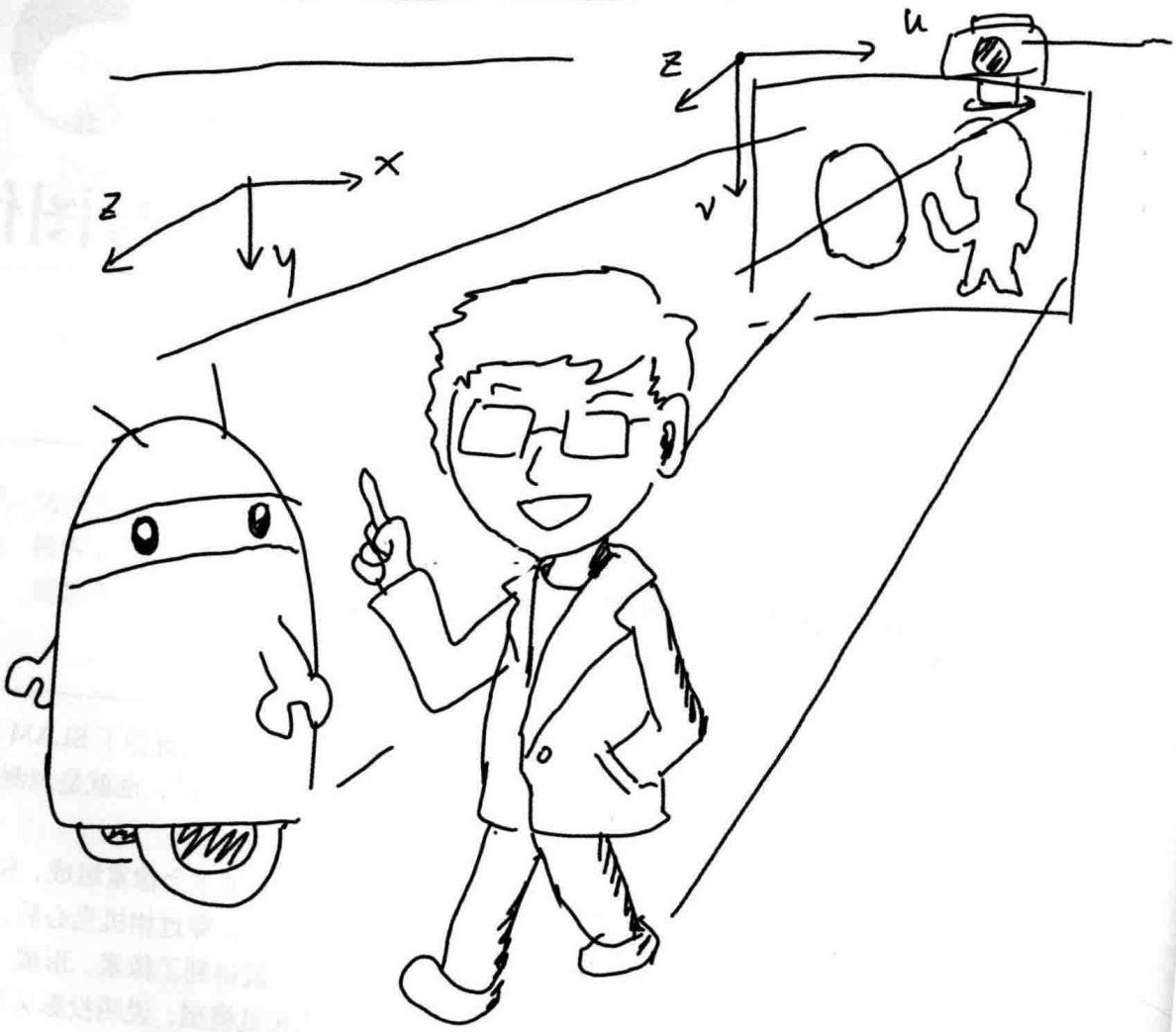
主要目标

1. 理解针孔相机的模型、内参与径向畸变参数。
2. 理解一个空间点是如何投影到相机成像平面的。
3. 掌握 OpenCV 的图像存储与表达方式。
4. 学会基本的摄像头标定方法。

前面两讲中，我们介绍了“机器人如何表示自身位姿”的问题，部分地解释了 SLAM 经典模型中变量的含义和运动方程部分。本讲将讨论“机器人如何观测外部世界”，也就是观测方程部分。而在以相机为主的视觉 SLAM 中，观测主要是指**相机成像**的过程。

我们在现实生活中能看到大量的照片。在计算机中，一张照片由很多个像素组成，每个像素记录了色彩或亮度的信息。三维世界中的一个物体反射或发出的光线，穿过相机光心后，投影在相机的成像平面上。相机的感光器件接收到光线后，产生测量值，就得到了像素，形成了我们见到的照片。这个过程能否用数学原理来描述呢？本讲将首先讨论相机模型，说明投影关系具体如何描述，相机的内参是什么。同时，简单介绍双目成像与 RGB-D 相机的原理。然后，介绍二维照片像素的基本操作。最后，根据内外参数的含义，演示一个点云拼接的实验。

$$s\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K(Rp+t)$$





5.1　相机模型

相机将三维世界中的坐标点（单位为米）映射到二维图像平面（单位为像素）的过程能够用一个几何模型进行描述。这个模型有很多种，其中最简单的称为**针孔模型**。针孔模型是很常用而且有效的模型，它描述了一束光线通过针孔之后，在针孔背面投影成像的关系。在本书中我们用一个简单的针孔相机模型对这种映射关系进行建模。同时，由于相机镜头上的透镜的存在，使得光线投影到成像平面的过程中会产生**畸变**。因此，我们使用针孔和畸变两个模型来描述整个投影过程。

本节先给出相机的针孔模型，再对透镜的畸变模型进行讲解。这两个模型能够把外部的三维点投影到相机内部成像平面，构成相机的**内参数**（Intrinsics）。

5.1.1　针孔相机模型

在初中物理课堂上，我们可能都见过一个蜡烛投影实验：在一个暗箱的前方放着一支点燃的蜡烛，蜡烛的光透过暗箱上的一个小孔投影在暗箱的后方平面上，并在这个平面上形成一个倒立的蜡烛图像。在这个过程中，小孔模型能够把三维世界中的蜡烛投影到一个二维成像平面。同理，我们可以用这个简单的模型来解释相机的成像过程，如图 5-1 所示。

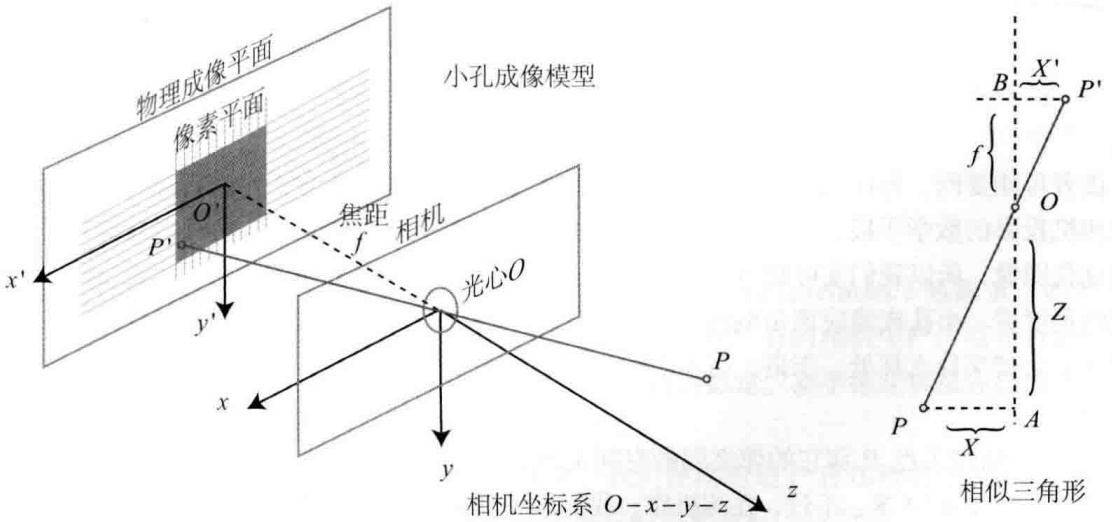


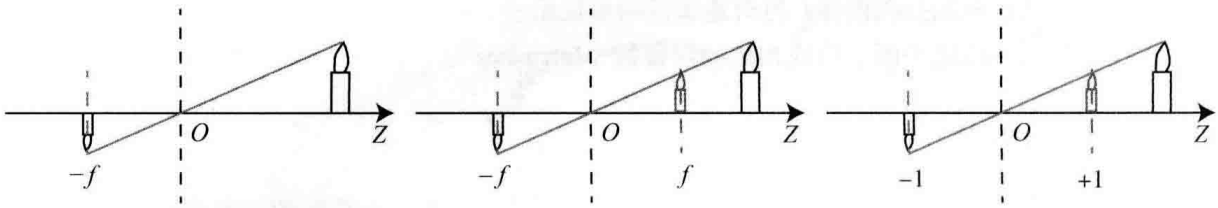
图 5-1　针孔相机模型

现在对这个简单的针孔模型进行几何建模。设 $O-x-y-z$ 为相机坐标系，习惯上我们让 z 轴指向相机前方，x 轴向右，y 轴向下（此图我们应该站在左侧看右侧）。O 为摄像机的**光心**，也是针孔模型中的针孔。现实世界的空间点 P，经过小孔 O 投影之后，落在物理成像平面 $O'-x'-y'$ 上，成像点为 P'。设 P 的坐标为 $[X,Y,Z]^{\mathrm{T}}$，P' 为 $[X',Y',Z']^{\mathrm{T}}$，并且设物理成像平面到小孔的距离为 f（焦距）。那么，根据三角形相似关系，有

$$\frac{Z}{f} = -\frac{X}{X'} = -\frac{Y}{Y'}. \tag{5.1}$$

其中负号表示成的像是倒立的。不过，实际相机得到的图像并不是倒像（否则相机的使用会非常不方便）。为了让模型更符合实际，我们可以等价地把成像平面对称地放到相机前方，和三维空间点一起放在摄像机坐标系的同一侧，如图 5-2 所示。这样做可以把公式中的负号去掉，使式子更加简洁：

$$\frac{Z}{f} = \frac{X}{X'} = \frac{Y}{Y'}. \tag{5.2}$$



真实成像平面　　对称的成像平面　　归一化成像平面

图 5-2　真实成像平面，对称的成像平面，归一化成像平面的图示

把 X', Y' 放到等式左侧，整理得

$$\begin{aligned} X' &= f\frac{X}{Z} \\ Y' &= f\frac{Y}{Z} \end{aligned}. \tag{5.3}$$

读者可能要问，为什么我们可以看似随意地把成像平面挪到前方呢？这只是我们处理真实世界与相机投影的数学手段，并且，大多数相机输出的图像并不是倒像——相机自身的软件会帮你翻转这张图像，所以我们实际得到的是正像，也就是对称的成像平面上的像。所以，尽管从物理原理的角度看，小孔成像应该是倒像，但由于我们对图像做了预处理，所以理解成在对称平面上的像并不会带来什么坏处。于是，在不引起歧义的情况下，我们也不加限制地称后一种情况为针孔模型。

式 (5.3) 描述了点 P 和它的像之间的空间关系，这里所有点的单位都可理解成米，比如焦距是 0.2 米，X' 是 0.14 米。不过，在相机中，我们最终获得的是一个个的像素，这还需要在成像平面上对像进行采样和量化。为了描述传感器将感受到的光线转换成图像像素的过程，我们设在物理成像平面上固定着一个像素平面 $o-u-v$。我们在像素平面得到了 P' 的**像素坐标**：$[u, v]^{\mathrm{T}}$。

像素坐标系[①]通常的定义方式是：原点 o' 位于图像的左上角，u 轴向右与 x 轴平行，v 轴向下与 y 轴平行。像素坐标系与成像平面之间，相差了一个**缩放**和一个**原点的平移**。我们设像素坐标在 u 轴上缩放了 α 倍，在 v 轴上缩放了 β 倍。同时，原点平移了 $[c_x, c_y]^{\mathrm{T}}$。那么，P' 的坐标与

①或图像坐标系，见 5.2 节。

像素坐标 $[u,v]^{\mathrm{T}}$ 的关系为

$$\begin{cases} u = \alpha X' + c_x \\ v = \beta Y' + c_y \end{cases}. \tag{5.4}$$

代入式 (5.3) 并把 αf 合并成 f_x，把 βf 合并成 f_y，得

$$\begin{cases} u = f_x \dfrac{X}{Z} + c_x \\ v = f_y \dfrac{Y}{Z} + c_y \end{cases}. \tag{5.5}$$

其中，f 的单位为米，α, β 的单位为像素/米，所以 f_x, f_y 和 c_x, c_y 的单位为像素。把该式写成矩阵形式会更加简洁，不过左侧需要用到齐次坐标，右侧则是非齐次坐标：

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \frac{1}{Z} \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \end{pmatrix} \overset{\text{def}}{=} \frac{1}{Z} \boldsymbol{K} \boldsymbol{P}. \tag{5.6}$$

我们习惯性地把 Z 挪到左侧：

$$Z \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \end{pmatrix} \overset{\text{def}}{=} \boldsymbol{K} \boldsymbol{P}. \tag{5.7}$$

该式中，我们把中间的量组成的矩阵称为**相机的内参数（Camera Intrinsics）矩阵** $\boldsymbol{K}$。通常认为，相机的内参在出厂之后是固定的，不会在使用过程中发生变化。有的相机生产厂商会告诉你相机的内参，而有时需要你自己确定相机的内参，也就是所谓的**标定**。鉴于标定算法业已成熟（如著名的单目棋盘格张正友标定法[25]），这里就不介绍了。

有内参，自然也有相对的外参。在式 (5.6) 中，我们使用的是 P 在相机坐标系下的坐标，但实际上由于相机在运动，所以 P 的相机坐标应该是它的世界坐标（记为 $\boldsymbol{P}_{\mathrm{w}}$）根据相机的当前位姿变换到相机坐标系下的结果。相机的位姿由它的旋转矩阵 $\boldsymbol{R}$ 和平移向量 $\boldsymbol{t}$ 来描述。那么有

$$Z \boldsymbol{P}_{uv} = Z \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \boldsymbol{K} (\boldsymbol{R} \boldsymbol{P}_{\mathrm{w}} + \boldsymbol{t}) = \boldsymbol{K} \boldsymbol{T} \boldsymbol{P}_{\mathrm{w}}. \tag{5.8}$$

注意后一个式子隐含了一次齐次坐标到非齐次坐标的转换（你能看出来吗？）[①]。它描述了 P 的世界坐标到像素坐标的投影关系。其中，相机的位姿 $\boldsymbol{R}, \boldsymbol{t}$ 又称为**相机的外参数**（Camera Extrinsics）[②]。相比于不变的内参，外参会随着相机运动发生改变，也是 SLAM 中待估计的目标，代表着机器人的轨迹。

投影过程还可以从另一个角度来看。式 (5.8) 表明，我们可以把一个世界坐标点先转换到相机坐标系，再除掉它最后一维的数值（即该点距离相机成像平面的深度），这相当于把最后一维进行**归一化处理**，得到点 P 在相机**归一化平面**上的投影：

$$(\boldsymbol{R}\boldsymbol{P}_{\mathrm{w}} + \boldsymbol{t}) = \underbrace{[X, Y, Z]^{\mathrm{T}}}_{\text{相机坐标}} \rightarrow \underbrace{[X/Z, Y/Z, 1]^{\mathrm{T}}}_{\text{归一化坐标}}. \tag{5.9}$$

归一化坐标可看成相机前方[③] $z = 1$ 处的平面上的一个点，这个 $z = 1$ 平面也称为**归一化平面**。归一化坐标再左乘内参就得到了像素坐标，所以我们可以把像素坐标 $[u, v]^{\mathrm{T}}$ 看成对归一化平面上的点进行量化测量的结果。从这个模型中也可以看出，如果对相机坐标同时乘以任意非零常数，归一化坐标都是一样的，这说明**点的深度在投影过程中被丢失了**，所以单目视觉中没法得到像素点的深度值。

5.1.2 畸变模型

为了获得好的成像效果，我们在相机的前方加了透镜。透镜的加入会对成像过程中光线的传播产生新的影响：一是透镜自身的形状对光线传播的影响；二是在机械组装过程中，透镜和成像平面不可能完全平行，这也会使光线穿过透镜投影到成像面时的位置发生变化。

由透镜形状引起的**畸变**（Distortion，也叫失真）称为**径向畸变**。在针孔模型中，一条直线投影到像素平面上还是一条直线。可是，在实际拍摄的照片中，摄像机的透镜往往使得真实环境中的一条直线在图片中变成了曲线[④]。越靠近图像的边缘，这种现象越明显。由于实际加工制作的透镜往往是中心对称的，这使得不规则的畸变通常径向对称。它们主要分为两大类：**桶形畸变**和**枕形畸变**，如图 5-3 所示。

桶形畸变图像放大率随着与光轴之间的距离增加而减小，而枕形畸变则恰好相反。在这两种畸变中，穿过图像中心和光轴有交点的直线还能保持形状不变。

除了透镜的形状会引入径向畸变，由于在相机的组装过程中不能使透镜和成像面严格平行，

①即，在 $\boldsymbol{TP}$ 中使用齐次坐标，再转化为非齐次坐标，再与 $\boldsymbol{K}$ 相乘。

②在机器人或自动驾驶车辆中，外参有时也解释成相机坐标系到机器人本体坐标系之间的变换，描述“相机安装在什么地方”。

③注意，在实际计算中需要检查 Z 是否为正，因为负数 Z 也可以通过这个方式得到归一化平面上的一个点，但相机并不会拍到成像平面后方的景物。

④是的，它不再直了，而是变成弯的。如果往里弯，则称为桶形畸变；往外弯则称为枕形畸变。

所以也会引入**切向畸变**，如图 5-4 所示。

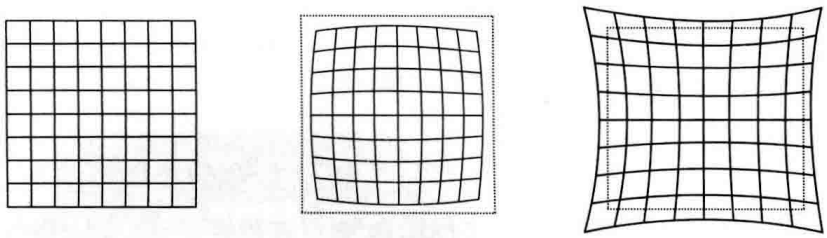


图 5-3　径向畸变的两种类型

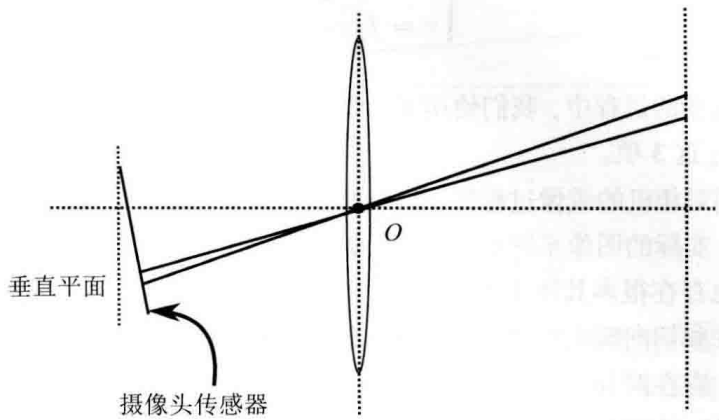


图 5-4　切向畸变来源示意图

为了更好地理解径向畸变和切向畸变，我们用更严格的数学形式对两者进行描述。考虑**归一化平面**上的任意一点 $\boldsymbol{p}$，它的坐标为 $[x,y]^{\mathrm{T}}$，也可写成极坐标的形式 $[r,\theta]^{\mathrm{T}}$，其中 r 表示点 $\boldsymbol{p}$ 与坐标系原点之间的距离，θ 表示与水平轴的夹角。径向畸变可以看成坐标点沿着长度方向发生了变化，也就是其距离原点的长度发生了变化。切向畸变可以看成坐标点沿着切线方向发生了变化，也就是水平夹角发生了变化。通常假设这些畸变呈多项式关系，即

$$\begin{aligned} x_{\text{distorted}} &= x(1+k_1r^2+k_2r^4+k_3r^6) \\ y_{\text{distorted}} &= y(1+k_1r^2+k_2r^4+k_3r^6) \end{aligned} \tag{5.10}$$

其中，$[x_{\text{distorted}},y_{\text{distorted}}]^{\mathrm{T}}$ 是畸变后点的**归一化坐标**。另外，对于**切向畸变**，可以使用另外两个参数 p_1,p_2 进行纠正：

$$\begin{aligned} x_{\text{distorted}} &= x+2p_1xy+p_2(r^2+2x^2) \\ y_{\text{distorted}} &= y+p_1(r^2+2y^2)+2p_2xy \end{aligned} \tag{5.11}$$

因此，联合式 (5.10) 和式 (5.11)，对于相机坐标系中的一点 P，我们能够通过 5 个畸变系数找到这个点在像素平面上的正确位置：

1. 将三维空间点投影到归一化图像平面。设它的归一化坐标为 $[x, y]^{\mathrm{T}}$。
2. 对归一化平面上的点计算径向畸变和切向畸变。

$$\begin{cases} x_{\text{distorted}} = x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + 2p_1 xy + p_2(r^2 + 2x^2) \\ y_{\text{distorted}} = y(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1(r^2 + 2y^2) + 2p_2 xy \end{cases}. \tag{5.12}$$

3. 将畸变后的点通过内参数矩阵投影到像素平面，得到该点在图像上的正确位置。

$$\begin{cases} u = f_x x_{\text{distorted}} + c_x \\ v = f_y y_{\text{distorted}} + c_y \end{cases}. \tag{5.13}$$

在上面的纠正畸变的过程中，我们使用了 5 个畸变项。实际应用中，可以灵活选择纠正模型，比如只选择 k_1, p_1, p_2 这 3 项。

在本节中，我们对相机的成像过程使用针孔模型进行了建模，也对透镜引起的径向畸变和切向畸变进行了描述。实际的图像系统中，学者们提出了很多其他的模型，比如相机的仿射模型和透视模型等，同时也存在很多其他类型的畸变。考虑到视觉 SLAM 中一般都使用普通的摄像头，针孔模型及径向畸变和切向畸变模型已经足够，因此，我们不再对其他模型进行描述。

值得一提的是，存在两种去畸变处理（Undistort，或称畸变校正）做法。我们可以选择先对整张图像进行去畸变，得到去畸变后的图像，然后讨论此图像上的点的空间位置。或者，也可以从畸变图像上的某个点出发，按照畸变方程，讨论其畸变前的空间位置。二者都是可行的，不过前者在视觉 SLAM 中似乎更常见。所以，当一个图像去畸变之后，我们就可以直接用针孔模型建立投影关系，而不用考虑畸变。因此，在后文的讨论中，我们可以直接假设图像已经进行了去畸变处理。

最后，我们总结单目相机的成像过程：

1. 世界坐标系下有一个固定的点 P，世界坐标为 $\boldsymbol{P}_{\mathrm{w}}$。
2. 由于相机在运动，它的运动由 $\boldsymbol{R}, \boldsymbol{t}$ 或变换矩阵 $\boldsymbol{T} \in \mathrm{SE}(3)$ 描述。P 的相机坐标为 $\tilde{\boldsymbol{P}}_{\mathrm{c}} = \boldsymbol{R}\boldsymbol{P}_{\mathrm{w}} + \boldsymbol{t}$。
3. 这时的 $\tilde{\boldsymbol{P}}_{\mathrm{c}}$ 的分量为 X, Y, Z，把它们投影到归一化平面 $Z = 1$ 上，得到 P 的归一化坐标：$\boldsymbol{P}_{\mathrm{c}} = [X/Z, Y/Z, 1]^{\mathrm{T}}$[①]。
4. 有畸变时，根据畸变参数计算 $\boldsymbol{P}_{\mathrm{c}}$ 发生畸变后的坐标。
5. P 的归一化坐标经过内参后，对应到它的像素坐标：$\boldsymbol{P}_{uv} = \boldsymbol{K}\boldsymbol{P}_{\mathrm{c}}$。

①注意：Z 可能小于 1，说明该点位于归一化平面后面，它可能不会在相机平面上成像，实践中要检查一次。

综上所述，我们一共谈到了四种坐标：世界坐标、相机坐标、归一化坐标和像素坐标。请读者厘清它们的关系，它反映了整个成像的过程。

5.1.3　双目相机模型

针孔相机模型描述了单个相机的成像模型。然而，仅根据一个像素，我们无法确定这个空间点的具体位置。这是因为，从相机光心到归一化平面连线上的所有点，都可以投影至该像素上。只有当 P 的深度确定时（比如通过双目或 RGB-D 相机），我们才能确切地知道它的空间位置，如图 5-5 所示。

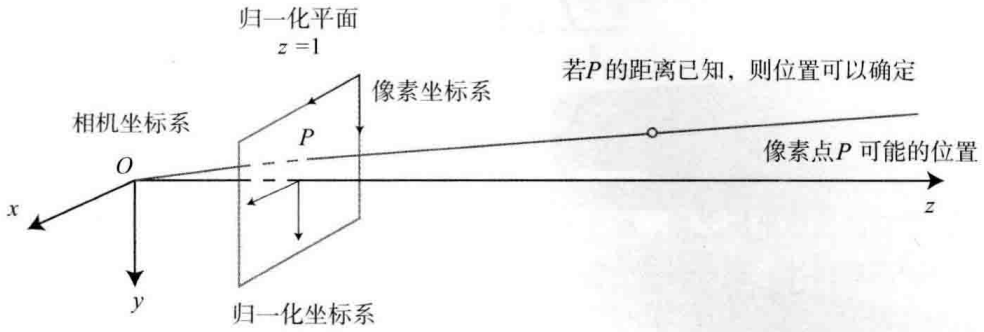


图 5-5　像素点可能存在的位置

测量像素距离（或深度）的方式有很多种，比如人眼就可以根据左右眼看到的景物差异（或称视差）判断物体与我们的距离。双目相机的原理亦是如此：通过同步采集左右相机的图像，计算图像间视差，以便估计每一个像素的深度。下面简单介绍双目相机的成像原理（如图 5-6 所示）。

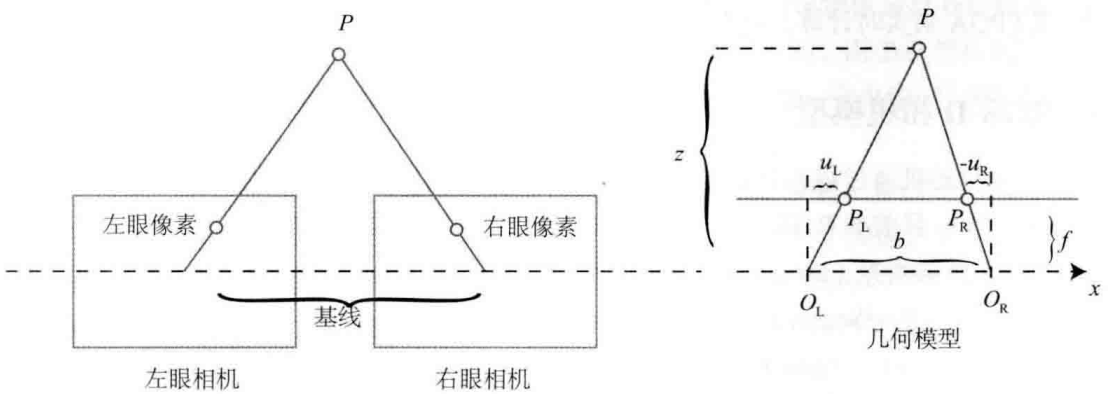


图 5-6　双目相机的成像模型。O_L, O_R 为左右光圈中心，方框为成像平面，f 为焦距。u_L 和 u_R 为成像平面的坐标。请注意，按照图中坐标定义，u_R 应该是负数，所以图中标出的距离为 $-u_R$

双目相机一般由左眼相机和右眼相机两个水平放置的相机组成。当然也可以做成上下两个目[①]，不过我们见到的主流双目都是做成左右形式的。在左右双目相机中，我们可以把两个相机都看作针孔相机。它们是水平放置的，意味着两个相机的光圈中心都位于 x 轴上。两者之间的距离称为双目相机的**基线**（记作 b），是双目相机的重要参数。

现在，考虑一个空间点 P，它在左眼相机和右眼相机各成一像，记作 $P_{\mathrm{L}}, P_{\mathrm{R}}$。由于相机基线的存在，这两个成像位置是不同的。理想情况下，由于左右相机只在 x 轴上有位移，所以 P 的像也只在 x 轴（对应图像的 u 轴）上有差异。记它的左侧坐标为 u_{L}，右侧坐标为 u_{R}，几何关系如图 5-6 右侧所示。根据 $\triangle PP_{\mathrm{L}}P_{\mathrm{R}}$ 和 $\triangle PO_{\mathrm{L}}O_{\mathrm{R}}$ 的相似关系，有

$$\frac{z-f}{z} = \frac{b-u_{\mathrm{L}}+u_{\mathrm{R}}}{b}. \tag{5.14}$$

稍加整理，得

$$z = \frac{fb}{d}, \quad d \overset{\mathrm{def}}{=} u_{\mathrm{L}} - u_{\mathrm{R}}. \tag{5.15}$$

其中 d 定义为左右图的横坐标之差，称为**视差**。根据视差，我们可以估计一个像素与相机之间的距离。视差与距离成反比：视差越大，距离越近[②]。同时，由于视差最小为一个像素，于是双目的深度存在一个理论上的最大值，由 fb 确定。我们看到，基线越长，双目能测到的最大距离就越远；反之，小型双目器件则只能测量很近的距离。相似地，我们人眼在看非常远的物体时（如很远的飞机），通常不能准确判断它的距离。

虽然由视差计算深度的公式很简洁，但视差 d 本身的计算却比较困难。我们需要确切地知道左眼图像的某个像素出现在右眼图像的哪一个位置（即对应关系），这件事也属于“人类觉得容易而计算机觉得困难”的任务。当我们想计算每个像素的深度时，其计算量与精度都将成为问题，而且只有在图像纹理变化丰富的地方才能计算视差。由于计算量的原因，双目深度估计仍需要使用 GPU 或 FPGA 来实时计算。这将在第 13 讲中介绍。

5.1.4 RGB-D 相机模型

相比于双目相机通过视差计算深度的方式，RGB-D 相机的做法更“主动”，它能够主动测量每个像素的深度。目前的 RGB-D 相机按原理可分为两大类（如图 5-7 所示）：

1. 通过**红外结构光**（Structured Light）原理测量像素距离。例子有 Kinect 1 代、Project Tango 1 代、Intel RealSense 等。
2. 通过**飞行时间**（Time-of-Flight，ToF）原理测量像素距离。例子有 Kinect 2 代和一些现有的 ToF 传感器等。

①那样的话外观会有些奇特。

②读者可以自己用眼睛模拟。

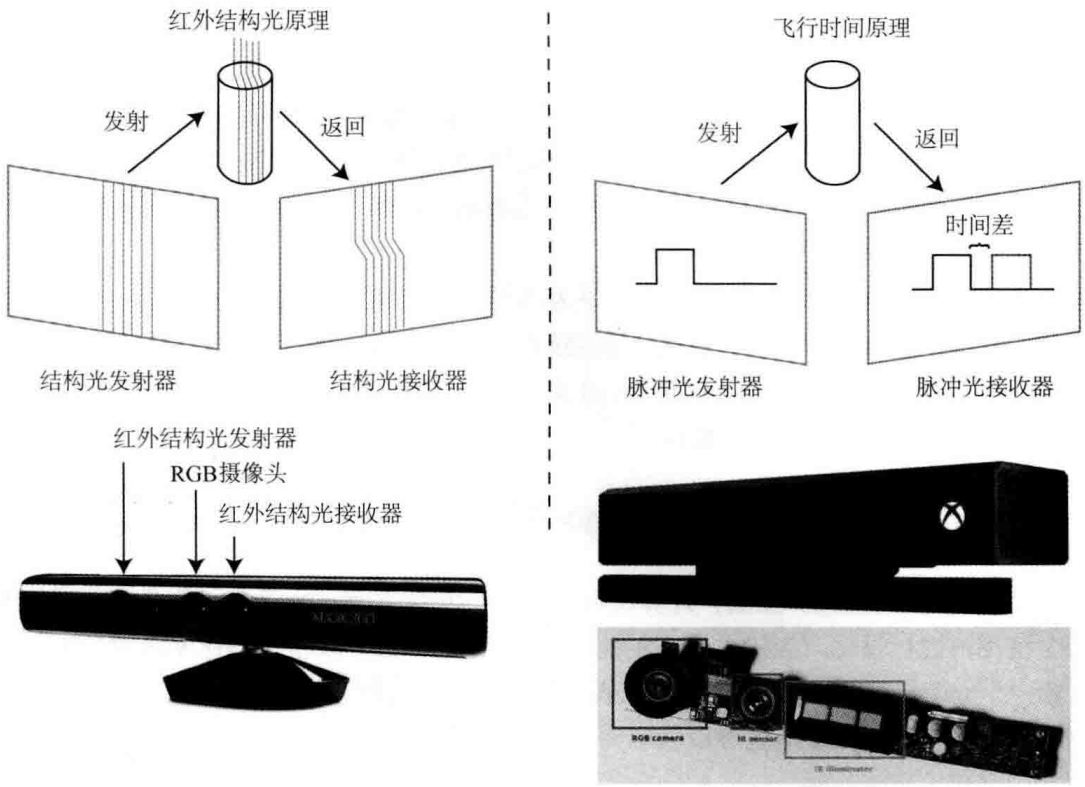


图 5-7　RGB-D 相机原理示意图

无论是哪种类型，RGB-D 相机都需要向探测目标发射一束光线（通常是红外光）。在红外结构光原理中，相机根据返回的结构光图案，计算物体与自身之间的距离。而在 ToF 原理中，相机向目标发射脉冲光，然后根据发送到返回之间的光束飞行时间，确定物体与自身的距离。ToF 的原理和激光传感器十分相似，只不过激光是通过逐点扫描获取距离的，而 ToF 相机则可以获得整个图像的像素深度，这也正是 RGB-D 相机的特点。所以，如果你把一个 RGB-D 相机拆开，通常会发现除了普通的摄像头，至少会有一个发射器和一个接收器。

在测量深度之后，RGB-D 相机通常按照生产时的各相机摆放位置，自己完成深度与彩色图像素之间的配对，输出一一对应的彩色图和深度图。我们可以在同一个图像位置，读取到色彩信息和距离信息，计算像素的 3D 相机坐标，生成点云（Point Cloud）。既可以在图像层面对 RGB-D 数据进行处理，也可在点云层面处理。本讲的第二个实验将演示 RGB-D 相机的点云构建过程。

RGB-D 相机能够实时地测量每个像素点的距离。但是，由于使用这种发射 - 接收的测量方式，其使用范围比较受限。用红外光进行深度值测量的 RGB-D 相机，容易受到日光或其他传感器发射的红外光干扰，因此不能在室外使用。在没有调制的情况下，同时使用多个 RGB-D 相机时也会相互干扰。对于透射材质的物体，因为接收不到反射光，所以无法测量这些点的位置。此外，RGB-D 相机在成本、功耗方面，都有一些劣势。

5.2 图像

相机加上镜头，把三维世界中的信息转换成了一张由像素组成的照片，随后存储在计算机中，作为后续处理的数据来源。在数学中，图像可以用一个矩阵来描述；而在计算机中，它们占据一段连续的磁盘或内存空间，可以用二维数组来表示。这样一来，程序就不必区别它们处理的是一个数值矩阵，还是有实际意义的图像了。

本节，我们将介绍计算机图像处理的一些基本操作。特别地，通过 OpenCV 中图像数据的处理，理解计算机中处理图像的常见步骤，为后续章节打下基础。我们从最简单的图像——灰度图说起。在一张灰度图中，每个像素位置 (x, y) 对应一个灰度值 I，所以，一张宽度为 w、高度为 h 的图像，数学上可以记为一个函数：

$$I(x, y) : \mathbb{R}^2 \mapsto \mathbb{R}.$$

其中，(x, y) 是像素的坐标。然而，计算机并不能表达实数空间，所以我们需要对下标和图像读数在某个范围内进行量化。例如，x, y 通常是从 0 开始的整数。在常见的灰度图中，用 0~255 的整数（即一个 unsigned char，1 个字节）来表达图像的灰度读数。那么，一张宽度为 640 像素、高度为 480 像素分辨率的灰度图就可以表示为：

二维数组表达图像

```
unsigned char image[480][640];
```

为什么这里的二维数组是 480×640 呢？因为在程序中，图像以二维数组形式存储。它的第一个下标是指数组的行，而第二个下标则是列。在图像中，数组的行数对应图像的高度，而列数对应图像的宽度。

下面考察这幅图像的内容。图像自然是由像素组成的。当访问某一个像素时，需要指明它所处的坐标，如图 5-8 所示。该图左边显示了传统像素坐标系的定义方式。像素坐标系原点位于图像的左上角，X 轴向右，Y 轴向下（也就是前面所说的 u, v 坐标）。如果它还有第三个轴——Z 轴，那么根据右手法则，Z 轴应该是向前的。这种定义方式是与相机坐标系一致的。我们平时说的图像的宽度或列数，对应着 X 轴；而图像的行数或高度，则对应着它的 Y 轴。

根据这种定义方式，如果我们讨论一个位于 x, y 处的像素，那么它在程序中的访问方式应该是：

访问图像像素

```
unsigned char pixel = image[y][x];
```

它对应着灰度值 $I(x, y)$ 的读数。请注意这里的 x 和 y 的顺序。虽然我们不厌其烦地讨论坐标系的问题，但是像这种下标顺序的错误，仍会是新手在调试过程中经常碰到的，且具有一定隐

蔽性的错误之一。如果你在写程序时不慎调换了 x, y 的坐标，编译器无法提供任何信息，而你所能看到的只是程序运行中的一个越界错误而已。

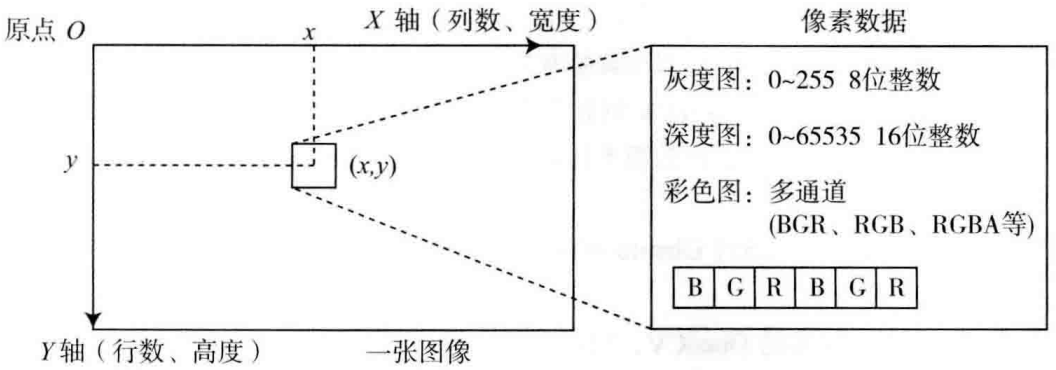


图 5-8　图像坐标示意图

一个像素的灰度可以用 8 位整数记录，也就是一个 0~255 的值。当我们要记录的信息更多时，一个字节恐怕就不够了。例如，在 RGB-D 相机的深度图中，记录了各个像素与相机之间的距离。这个距离通常以毫米为单位，而 RGB-D 相机的量程通常在十几米左右，超过了 255。这时，人们会采用 16 位整数（C++ 中的 unsigned short）来记录深度图的信息，也就是位于 0~65535 的值。换算成米的话，最大可以表示 65 米，足够 RGB-D 相机使用了。

彩色图像的表示则需要通道（channel）的概念。在计算机中，我们用红色、绿色和蓝色这三种颜色的组合来表达任意一种色彩。于是对于每一个像素，就要记录其 R、G、B 三个数值，每一个数值就称为一个通道。例如，最常见的彩色图像有三个通道，每个通道都由 8 位整数表示。在这种规定下，一个像素占据 24 位空间。

通道的数量、顺序都是可以自由定义的。在 OpenCV 的彩色图像中，通道的默认顺序是 B、G、R。也就是说，当我们得到一个 24 位的像素时，前 8 位表示蓝色数值，中间 8 位为绿色数值，最后 8 位为红色数值。同理，也可使用 R、G、B 的顺序表示一个彩色图。如果还想表达图像的透明度，就使用 R、G、B、A 四个通道。

5.3　实践：计算机中的图像

5.3.1　OpenCV 的基本使用方法

下面通过一个演示程序来理解：在 OpenCV 中图像是如何存取，我们又是如何访问其中的像素的。

安装 OpenCV

OpenCV[①]提供了大量的开源图像算法，是计算机视觉中使用极广的图像处理算法库。本书也使用 OpenCV 做基本的图像处理。在使用之前，建议读者选择“从源代码安装”的方式安装它。在 Ubuntu 系统下，有**从源代码安装**和**只安装库文件**两种方式可以选择：

1. 从源代码安装，是指从 OpenCV 网站下载所有的 OpenCV 源代码，并在机器上编译安装，以便使用。好处是可以选择的版本比较丰富，而且能看到源代码，不过需要花费一些编译时间。
2. 只安装库文件，是指通过 Ubuntu 安装由 Ubuntu 社区人员已经编译好的库文件，这样就无须重新编译一遍。

因为我们使用较新版本的 OpenCV，所以必须选择从源代码安装的方式安装它。一来，可以调整一些编译选项，匹配编程环境（例如，需不需要 GPU 加速等）；再者，可以使用一些额外的功能。OpenCV 目前维护了三个主要版本，分为 OpenCV 2.4 系列、OpenCV 3 系列和 OpenCV 4 系列。本书使用 OpenCV **3** 系列。

由于 OpenCV 工程比较大，就不放在本书的 3rdparty 目录下了。请读者从 `http://opencv.org/downloads.html` 下载，选择 OpenCV for Linux 版本即可。你会获得一个像 opencv-3.1.0.zip 这样的压缩包。将它解压到任意目录下，我们发现 OpenCV 也是一个 cmake 工程。

在编译之前，先来安装 OpenCV 的依赖项：

终端输入：

```
sudo apt-get install build-essential libgtk2.0-dev libvtk5-dev libjpeg-dev libtiff4-dev
libjasper-dev libopenexr-dev libtbb-dev
```

事实上，OpenCV 的依赖项很多，缺少某些编译项会影响它的部分功能（不过我们也不会用到所有功能）。OpenCV 会在 cmake 阶段检查依赖项是否会安装，并调整自己的功能。如果你的电脑上有 GPU 并且安装了相关依赖项，OpenCV 也会把 GPU 加速打开。不过对于本书，上面那些依赖项就足够了。

随后的编译安装和普通的 cmake 工程一样，请在 make 之后，调用 sudo make install 将 OpenCV 安装到你的机器上（而不是仅仅编译它）。视机器配置，这个编译过程大概需要二十分钟到一个小时不等。如果你的 CPU 比较强，可以使用“make -j4”这样的命令，调用多个线程进行编译（-j 后面的参数就是使用的线程数量）。安装后，OpenCV 默认存储在/usr/local 目录下。你可以寻找 OpenCV 头文件与库文件的安装位置，看看它们都在哪里。另外，如果之前已经安装了 OpenCV 2 系列，那么建议你把 OpenCV 3 安装到别的地方（想想这应该如何操作）。

①官方主页：`http://opencv.org`。

操作 OpenCV 图像

接下来，通过一个例程熟悉 OpenCV 对图像的操作。

slambook/ch5/imageBasics/imageBasics.cpp

```
#include <iostream>
#include <chrono>

using namespace std;

#include <opencv2/core/core.hpp>
#include <opencv2/highgui/highgui.hpp>

int main(int argc, char **argv) {
    // 读取argv[1]指定的图像
    cv::Mat image;
    image = cv::imread(argv[1]); //cv::imread函数读取指定路径下的图像

    // 判断图像文件是否正确读取
    if (image.data == nullptr) { //数据不存在,可能是文件不存在
        cerr << "文件" << argv[1] << "不存在." << endl;
        return 0;
    }

    // 文件顺利读取, 首先输出一些基本信息
    cout << "图像宽为" << image.cols << ",高为" << image.rows
         << ",通道数为" << image.channels() << endl;
    cv::imshow("image", image);      // 用cv::imshow显示图像
    cv::waitKey(0);                  // 暂停程序,等待一个按键输入

    // 判断image的类型
    if (image.type() != CV_8UC1 && image.type() != CV_8UC3) {
        // 图像类型不符合要求
        cout << "请输入一张彩色图或灰度图." << endl;
        return 0;
    }

    // 遍历图像, 请注意以下遍历方式也可使用于随机像素访问
    // 使用std::chrono给算法计时
    chrono::steady_clock::time_point t1 = chrono::steady_clock::now();
    for (size_t y = 0; y < image.rows; y++) {
        // 用cv::Mat::ptr获得图像的行指针
        unsigned char *row_ptr = image.ptr<unsigned char>(y);  // row_ptr是第y行的头指针
        for (size_t x = 0; x < image.cols; x++) {
```

```
            // 访问位于x,y处的像素
            unsigned char *data_ptr = &row_ptr[x * image.channels()]; // data_ptr指向待访问的
            像素数据
            // 输出该像素的每个通道,如果是灰度图就只有一个通道
            for (int c = 0; c != image.channels(); c++) {
                unsigned char data = data_ptr[c]; // data为I(x,y)第c个通道的值
            }
        }
    }
    chrono::steady_clock::time_point t2 = chrono::steady_clock::now();
    chrono::duration<double> time_used = chrono::duration_cast < chrono::duration < double >>
     (t2 - t1);
    cout << "遍历图像用时：" << time_used.count() << " 秒。" << endl;

    // 关于cv::Mat的拷贝
    // 直接赋值并不会拷贝数据
    cv::Mat image_another = image;
    // 修改image_another会导致image发生变化
    image_another(cv::Rect(0, 0, 100, 100)).setTo(0); // 将左上角100*100的块置零
    cv::imshow("image", image);
    cv::waitKey(0);

    // 使用clone函数拷贝数据
    cv::Mat image_clone = image.clone();
    image_clone(cv::Rect(0, 0, 100, 100)).setTo(255);
    cv::imshow("image", image);
    cv::imshow("image_clone", image_clone);
    cv::waitKey(0);

    // 对于图像还有很多基本的操作，如剪切、旋转、缩放等，限于篇幅就不一一介绍了，请参看OpenCV
    官方文档查询每个函数的调用方法
    cv::destroyAllWindows();
    return 0;
}
```

在该例程中，我们演示了如下几个操作：图像读取、显示、像素遍历、复制、赋值等。大部分的注解已写在代码里。编译该程序时，你需要在 CMakeLists.txt 中添加 OpenCV 的头文件，然后把程序链接到库文件上。同时，由于使用了 C++ 11 标准（如 nullptr 和 chrono），还需要设置编译器：

slambook/ch5/imageBasics/CMakeLists.txt

```
# 添加C++ 11标准支持
set( CMAKE_CXX_FLAGS "-std=c++11" )
```

```
3
4  # 寻找OpenCV库
5  find_package( OpenCV REQUIRED )
6  # 添加头文件
7  include_directories( ${OpenCV_INCLUDE_DIRS} )
8
9  add_executable( imageBasics imageBasics.cpp )
10 # 链接OpenCV库
11 target_link_libraries( imageBasics ${OpenCV_LIBS} )
```

关于代码，我们给出几点说明：

1. 程序从 argv[1]，也就是命令行的第一个参数中读取图像位置。我们为读者准备了一张图像（ubuntu.png，一张 Ubuntu 的壁纸，希望你喜欢）供测试使用。因此，编译之后，使用如下命令调用此程序：

   终端输入：

   ```
   1 build/imageBasics ubuntu.png
   ```

   如果在 IDE 中调用此程序，则请务必确保把参数同时给它。这可以在启动项中配置。
2. 在程序的 10~18 行，使用 cv::imread 函数读取图像，并把图像和基本信息显示出来。
3. 在程序的 35~47 行，遍历了图像中的所有像素，并计算了整个循环所用的时间。请注意像素的遍历方式并不是唯一的，而且例程给出的方式也不是最高效的。OpenCV 提供了迭代器，你可以通过迭代器遍历图像的像素。或者，cv::Mat::data 提供了指向图像数据开头的指针，你可以直接通过该指针自行计算偏移量，然后得到像素的实际内存位置。例程所用的方式是为了便于读者理解图像的结构。

   在笔者的机器上（虚拟机），遍历这张图像用时大约 12.74 毫秒。你可以与自己机器上的速度对比。不过，我们使用的是 cmake 默认的 debug 模式，如果使用 release 模式会快很多。
4. OpenCV 提供了许多对图像进行操作的函数，我们在此不一一列举，否则本书就会变成 OpenCV 操作手册。例程给出了较为常见的读取、显示操作，以及复制图像中可能陷入的深拷贝误区。在编程过程中，读者还会碰到图像的旋转、插值等操作，这时你应该自行查阅函数对应的文档，以了解它们的原理与使用方式。

应该指出，OpenCV 并不是唯一的图像库，它只是许多图像库里使用范围较广泛的一个。不过，多数图像库对图像的表达是大同小异的。我们希望读者了解了 OpenCV 对图像的表示后，能够理解其他库中图像的表达，从而在需要数据格式时能够自己处理。另外，由于 cv::Mat 也是矩阵类，除了表示图像，我们也可以用它来存储位姿等矩阵数据。一般认为，Eigen 对于固定大小的矩阵使用起来效率更高。

5.3.2 图像去畸变

在理论部分我们介绍了径向和切向畸变，下面来演示一个去畸变过程。OpenCV 提供了去畸变函数 cv::Undistort()，但本例我们从公式出发计算畸变前后的图像坐标。

slambook/ch5/imageBasics/undistortImage.cpp

```
#include <opencv2/opencv.hpp>
#include <string>
using namespace std;
string image_file = "./distorted.png";   // 请确保路径正确

int main(int argc, char **argv) {
    // 本程序实现去畸变部分的代码。尽管我们可以调用OpenCV的去畸变，但自己实现一遍有助于理解。
    // 畸变参数
    double k1 = -0.28340811, k2 = 0.07395907, p1 = 0.00019359, p2 = 1.76187114e-05;
    // 内参
    double fx = 458.654, fy = 457.296, cx = 367.215, cy = 248.375;

    cv::Mat image = cv::imread(image_file, 0);   // 图像是灰度图，CV_8UC1
    int rows = image.rows, cols = image.cols;
    cv::Mat image_undistort = cv::Mat(rows, cols, CV_8UC1);   // 去畸变以后的图

    // 计算去畸变后图像的内容
    for (int v = 0; v < rows; v++) {
        for (int u = 0; u < cols; u++) {
            // 按照公式，计算点(u,v)对应到畸变图像中的坐标(u_distorted, v_distorted)
            double x = (u - cx) / fx, y = (v - cy) / fy;
            double r = sqrt(x * x + y * y);
            double x_distorted = x * (1 + k1 * r * r + k2 * r * r * r * r * r) + 2 * p1 * x * y +
             p2 * (r * r + 2 * x * x);
            double y_distorted = y * (1 + k1 * r * r + k2 * r * r * r * r * r) + p1 * (r * r + 2
            * y * y) + 2 * p2 * x * y;
            double u_distorted = fx * x_distorted + cx;
            double v_distorted = fy * y_distorted + cy;

            // 赋值（最近邻插值）
            if (u_distorted >= 0 && v_distorted >= 0 && u_distorted < cols && v_distorted <
            rows) {
                image_undistort.at<uchar>(v, u) = image.at<uchar>((int) v_distorted, (int)
                u_distorted);
            } else {
                image_undistort.at<uchar>(v, u) = 0;
            }
```

```
        }
    }

    // 画出去畸变后图像
    cv::imshow("distorted", image);
    cv::imshow("undistorted", image_undistort);
    cv::waitKey();
    return 0;
}
```

5.4　实践：3D 视觉

5.4.1　双目视觉

我们已经介绍了双目视觉的成像原理。现在我们从双目视觉的左右图像出发，计算图像对应的视差图，然后计算各像素在相机坐标系下的坐标，它们将构成**点云**。在第 5 讲代码目录的 stereo 文件夹中，我们准备了双目视觉的图像，如图 5-9 所示。下面的代码演示了计算视差图和点云部分：

slambook/ch5/stereoVision/stereoVision.cpp（部分）

```
int main(int argc, char **argv) {
    // 内参
    double fx = 718.856, fy = 718.856, cx = 607.1928, cy = 185.2157;
    // 基线
    double b = 0.573;

    // 读取图像
    cv::Mat left = cv::imread(left_file, 0);
    cv::Mat right = cv::imread(right_file, 0);
    cv::Ptr<cv::StereoSGBM> sgbm = cv::StereoSGBM::create(
        0, 96, 9, 8 * 9 * 9, 32 * 9 * 9, 1, 63, 10, 100, 32);     // 神奇的参数
    cv::Mat disparity_sgbm, disparity;
    sgbm->compute(left, right, disparity_sgbm);
    disparity_sgbm.convertTo(disparity, CV_32F, 1.0 / 16.0f);

    // 生成点云
    vector<Vector4d, Eigen::aligned_allocator<Vector4d>> pointcloud;

    // 如果你的机器慢，请把后面的v++和u++改成v+=2, u+=2
    for (int v = 0; v < left.rows; v++)
```

```
        for (int u = 0; u < left.cols; u++) {
            if (disparity.at<float>(v, u) <= 10.0 || disparity.at<float>(v, u) >= 96.0)
            continue;

            Vector4d point(0, 0, 0, left.at<uchar>(v, u) / 255.0); //前三维为xyz,第四维为颜色

            // 根据双目模型计算point的位置
            double x = (u - cx) / fx;
            double y = (v - cy) / fy;
            double depth = fx * b / (disparity.at<float>(v, u));
            point[0] = x * depth;
            point[1] = y * depth;
            point[2] = depth;

            pointcloud.push_back(point);
    }

    cv::imshow("disparity", disparity / 96.0);
    cv::waitKey(0);
    // 画出点云
    showPointCloud(pointcloud);
    return 0;
}
```

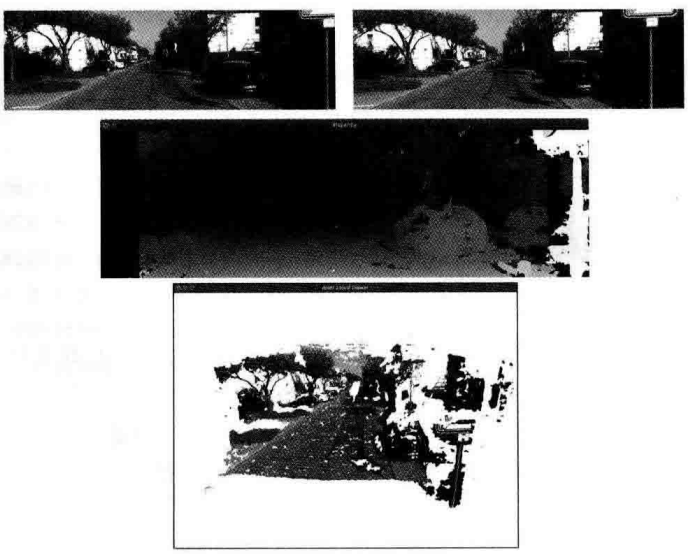


图 5-9 双目视觉的例子，左上：左眼图像，右上：右眼图像，中间：SGBM 的视差图，下方：点云图。注意，因为左侧相机看到了一部分右侧相机没有看到的内容，所以对应的视差是空的

这个例子中我们调用了 OpenCV 实现的 SGBM（Semi-Global Batch Matching）[26] 算法计算左右图像的视差，然后通过双目相机的几何模型把它变换到相机的 3D 空间中。SGBM 使用了来自网络的经典参数配置，我们主要调整了最大和最小视差。视差数据结合相机的内参、基线，即能确定各点在三维空间中的位置。为了节省版面，我们省略了显示点云相关的代码。

本书不准备展开介绍双目相机的视差计算算法，感兴趣的读者可以阅读相关的参考文献 [27, 28]。除了 OpenCV 实现的双目算法，还有许多其他的库专注于实现高效的视差计算。它是一个复杂又实用的课题。

5.4.2　RGB-D 视觉

最后，我们演示一个 RGB-D 视觉的例子。RGB-D 相机的方便之处在于能通过物理方法获得像素深度信息。如果已知相机的内外参，我们就可以计算任何一个像素在世界坐标系下的位置，从而建立一张点云地图。现在我们就来演示。

我们准备了 5 对图像，位于 slambook/ch5/rgbd 文件夹中。在 color/下有 1.png 到 5.png 共 5 张 RGB 图，而在 depth/下有 5 张对应的深度图。同时，pose.txt 文件给出了 5 张图像的相机外参位姿（以 $\boldsymbol{T}_{\mathrm{wc}}$ 形式）。位姿记录的形式和之前一样，为平移向量加旋转四元数：

$$[x, y, z, q_x, q_y, q_z, q_w],$$

其中，q_w 是四元数的实部。例如，第一对图的外参为：

$$[-0.228993, 0.00645704, 0.0287837, -0.0004327, -0.113131, -0.0326832, 0.993042].$$

下面我们写一段程序，完成两件事：

（1）根据内参计算一对 RGB-D 图像对应的点云。

（2）根据各张图的相机位姿（也就是外参），把点云加起来，组成地图。

slambook/ch5/rgbd/jointMap.cpp（部分）

```
int main(int argc, char **argv) {
    vector<cv::Mat> colorImgs, depthImgs;    // 彩色图和深度图
    TrajectoryType poses;         // 相机位姿

    ifstream fin("./pose.txt");
    if (!fin) {
        cerr << "请在有pose.txt的目录下运行此程序" << endl;
        return 1;
    }

```

```
    for (int i = 0; i < 5; i++) {
        boost::format fmt("./%s/%d.%s"); //图像文件格式
        colorImgs.push_back(cv::imread((fmt % "color" % (i + 1) % "png").str()));
        depthImgs.push_back(cv::imread((fmt % "depth" % (i + 1) % "pgm").str(), -1)); // 使用
        -1读取原始图像

        double data[7] = {0};
        for (auto &d:data) fin >> d;
        Sophus::SE3d pose(Eigen::Quaterniond(data[6], data[3], data[4], data[5]),
        Eigen::Vector3d(data[0], data[1], data[2]));
        poses.push_back(pose);
    }

    // 计算点云并拼接
    // 相机内参
    double cx = 325.5;
    double cy = 253.5;
    double fx = 518.0;
    double fy = 519.0;
    double depthScale = 1000.0;
    vector<Vector6d, Eigen::aligned_allocator<Vector6d>> pointcloud;
    pointcloud.reserve(1000000);

    for (int i = 0; i < 5; i++) {
        cout << "转换图像中: " << i + 1 << endl;
        cv::Mat color = colorImgs[i];
        cv::Mat depth = depthImgs[i];
        Sophus::SE3d T = poses[i];
        for (int v = 0; v < color.rows; v++)
            for (int u = 0; u < color.cols; u++) {
                unsigned int d = depth.ptr<unsigned short>(v)[u]; // 深度值
                if (d == 0) continue; // 为0表示没有测量到
                Eigen::Vector3d point;
                point[2] = double(d) / depthScale;
                point[0] = (u - cx) * point[2] / fx;
                point[1] = (v - cy) * point[2] / fy;
                Eigen::Vector3d pointWorld = T * point;

                Vector6d p;
                p.head<3>() = pointWorld;
                p[5] = color.data[v * color.step + u * color.channels()];   // blue
                p[4] = color.data[v * color.step + u * color.channels() + 1]; // green
                p[3] = color.data[v * color.step + u * color.channels() + 2]; // red
```

```
                pointcloud.push_back(p);
            }
        }

        cout << "点云共有" << pointcloud.size() << "个点." << endl;
        showPointCloud(pointcloud);
        return 0;
}
```

运行程序后即可在 Pangolin 窗口中看到拼合的点云地图（如图 5-10 所示）。你可以拖动鼠标查看。

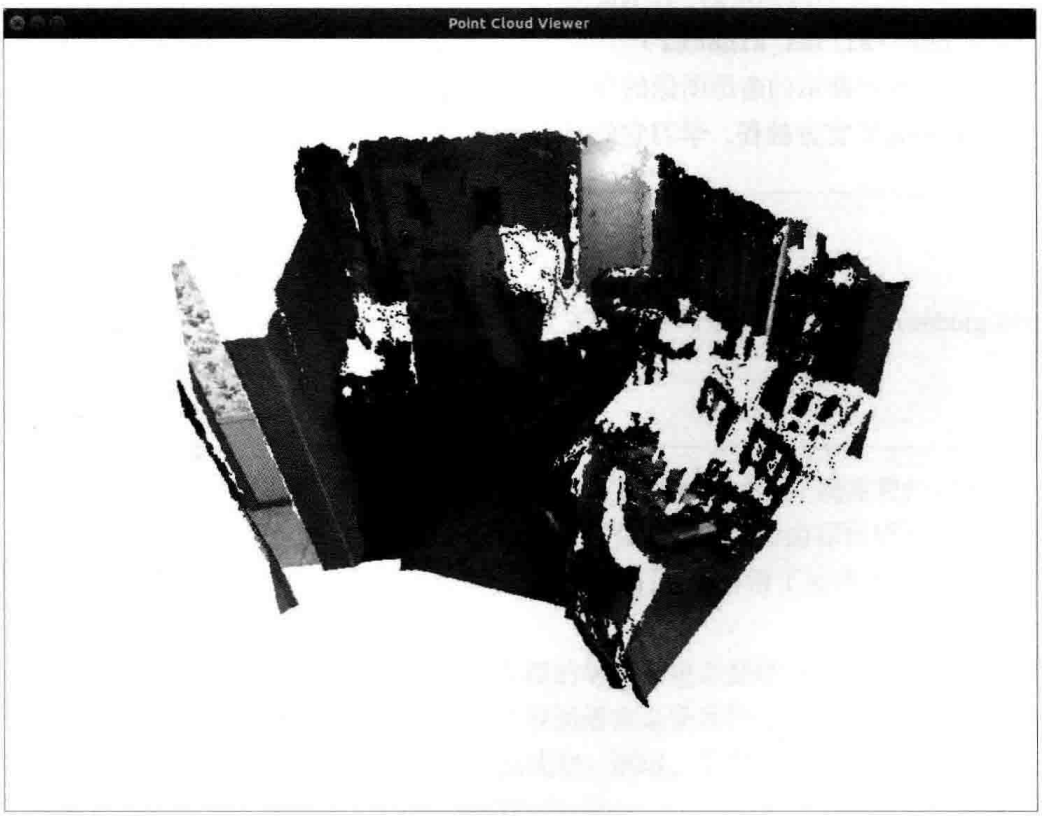


图 5-10　拼合的点云地图（见彩插）

通过这些例子，我们演示了计算机视觉中一些常见的单目、双目和深度相机算法。希望读者通过这些简单的例子，体会相机内外参、畸变参数的含义。

习题

1.* 寻找一部相机（你的手机或笔记本的摄像头即可），标定它的内参。你可能会用到标定板，或者自己打印一张标定用的棋盘格。
2. 叙述相机内参的物理意义。如果一部相机的分辨率变为原来的两倍而其他地方不变，那么它的内参将如何变化？
3. 搜索特殊相机（鱼眼或全景相机）的标定方法。它们与普通的针孔模型有何不同？
4. 调研全局快门（global shutter）相机和卷帘快门（rolling shutter）相机的异同。它们在 SLAM 中有何优缺点？
5. RGB-D 相机是如何标定的？以 Kinect 为例，需要标定哪些参数？（参照`https://github.com/code-iai/iai_kinect2`）
6. 除了示例程序演示的遍历图像的方式，你还能举出哪些遍历图像的方法？
7.* 阅读 OpenCV 官方教程，学习它的基本用法。

第 6 讲

非线性优化

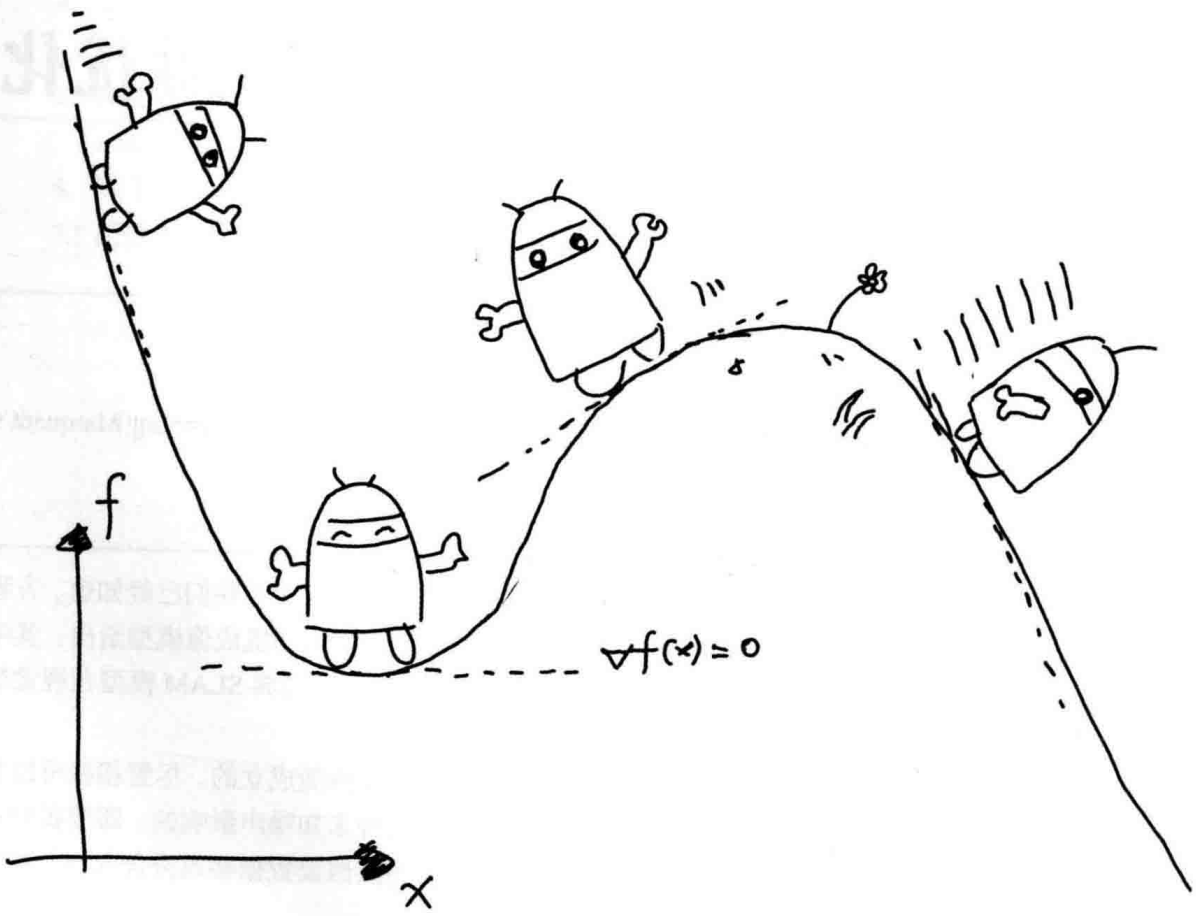
主要目标

1. 理解最小二乘法的含义和处理方式。
2. 理解高斯牛顿法（Gauss-Newton's method）、列文伯格—马夸尔特方法（Levenburg-Marquadt's method）等下降策略。
3. 学习 Ceres 库和 g2o 库的基本使用方法。

在前面几讲，我们介绍了经典 SLAM 模型的运动方程和观测方程。现在我们已经知道，方程中的位姿可以由变换矩阵来描述，然后用李代数进行优化。观测方程由相机成像模型给出，其中内参是随相机固定的，而外参则是相机的位姿。于是，我们已经弄清了经典 SLAM 模型在视觉情况下的具体表达。

然而，由于噪声的存在，运动方程和观测方程的等式必定不是精确成立的。尽管相机可以非常好地符合针孔模型，但遗憾的是，我们得到的数据通常是受各种未知噪声影响的。即使我们有高精度的相机，运动方程和观测方程也只能近似成立。所以，与其假设数据必须符合方程，不如讨论如何在有噪声的数据中进行准确的状态估计。

解决状态估计问题需要一定程度的最优化背景知识。本节将介绍基本的无约束非线性优化方法，同时介绍优化库 g2o 和 Ceres 的使用方式。



$$f(x+\Delta x) \approx f(x) + \nabla f(x)\Delta x$$

$$+ \frac{1}{2} \Delta x^T H(x) \Delta x$$

$$+ \cdots$$

6.1 状态估计问题

6.1.1 批量状态估计与最大后验估计

接着前面几讲的内容，我们回顾第 2 讲讨论的经典 SLAM 模型。它由一个运动方程和一个观测方程构成，如式 (2.5) 所示：

$$\begin{cases} \boldsymbol{x}_k = f\left(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k\right) + \boldsymbol{w}_k \\ \boldsymbol{z}_{k,j} = h\left(\boldsymbol{y}_j, \boldsymbol{x}_k\right) + \boldsymbol{v}_{k,j} \end{cases}. \tag{6.1}$$

通过第 4 讲的知识，我们了解到这里的 $\boldsymbol{x}_k$ 乃是相机的位姿，可以用 SE(3) 来描述。至于观测方程，第 5 讲已经说明，即针孔相机模型。为了让读者对它们有更深的印象，我们不妨讨论其具体参数化形式。首先，位姿变量 $\boldsymbol{x}_k$ 可以由 $\boldsymbol{T}_k \in \mathrm{SE}(3)$ 表达。其次，运动方程与输入的具体形式有关，但在视觉 SLAM 中没有特殊性（和普通的机器人、车辆的情况一样），我们暂且不谈。观测方程则由针孔模型给定。假设在 $\boldsymbol{x}_k$ 处对路标 $\boldsymbol{y}_j$ 进行了一次观测，对应到图像上的像素位置 $\boldsymbol{z}_{k,j}$，那么，观测方程可以表示成

$$s\boldsymbol{z}_{k,j} = \boldsymbol{K}(\boldsymbol{R}_k\boldsymbol{y}_j + \boldsymbol{t}_k). \tag{6.2}$$

其中 $\boldsymbol{K}$ 为相机内参，s 为像素点的距离，也是 $(\boldsymbol{R}_k\boldsymbol{y}_j + \boldsymbol{t}_k)$ 的第三个分量。如果使用变换矩阵 $\boldsymbol{T}_k$ 描述位姿，那么路标点 $\boldsymbol{y}_j$ 必须以齐次坐标来描述，计算完成后要转换为非齐次坐标。如果你还不熟悉这个过程，请回顾第 5 讲。

现在，考虑数据受噪声影响后会发生什么改变。在运动和观测方程中，我们**通常**假设两个噪声项 $\boldsymbol{w}_k, \boldsymbol{v}_{k,j}$ 满足零均值的高斯分布，像这样：

$$\boldsymbol{w}_k \sim \mathcal{N}\left(\boldsymbol{0}, \boldsymbol{R}_k\right), \boldsymbol{v}_k \sim \mathcal{N}\left(\boldsymbol{0}, \boldsymbol{Q}_{k,j}\right). \tag{6.3}$$

其中 $\mathcal{N}$ 表示高斯分布，$\boldsymbol{0}$ 表示零均值，$\boldsymbol{R}_k, \boldsymbol{Q}_{k,j}$ 为协方差矩阵。在这些噪声的影响下，我们希望通过带噪声的数据 $\boldsymbol{z}$ 和 $\boldsymbol{u}$ 推断位姿 $\boldsymbol{x}$ 和地图 $\boldsymbol{y}$（以及它们的概率分布），这构成了一个状态估计问题。

处理这个状态估计问题的方法大致分成两种。由于在 SLAM 过程中，这些数据是随时间逐渐到来的，所以，我们应该持有一个当前时刻的估计状态，然后用新的数据来更新它。这种方式称为**增量/渐进**（incremental）的方法，或者叫**滤波器**。在历史上很长一段时间内，研究者们使用滤波器，尤其是扩展卡尔曼滤波器及其衍生方法求解它。另一种方式，则是把数据“攒”起来一并处理，这种方式称为**批量**（batch）的方法。例如，我们可以把 0 到 k 时刻所有的输入和观测数

据都放在一起，问，在这样的输入和观测下，如何估计整个 0 到 k 时刻的轨迹与地图呢？

这两种不同的处理方式引出了很多不同的估计手段。大体来说，增量方法仅关心**当前时刻**的状态估计 $\boldsymbol{x}_k$，而对之前的状态则不多考虑；相对地，批量方法可以在**更大的范围**达到最优化，被认为优于传统的滤波器[13]，而成为当前视觉 SLAM 的主流方法。极端情况下，我们可以让机器人或无人机收集所有时刻的数据，再带回计算中心统一处理，这也正是 SfM（Structure from Motion）的主流做法。当然，这种极端情况显然是不**实时**的，不符合 SLAM 的运用场景。所以在 SLAM 中，实用的方法通常是一些折衷的手段。例如，我们固定一些历史轨迹，仅对当前时刻附近的一些轨迹进行优化，这是后面要讲到的**滑动窗口估计**法。

理论上，批量方法更容易介绍。同时，理解了批量方法也更容易理解增量的方法。所以，本节我们重点介绍以非线性优化为主的批量优化方法，将卡尔曼滤波器及更深入的知识留到介绍后端的章节再进行讨论。由于讨论的是批量方法，考虑从 1 到 N 的所有时刻，并假设有 M 个路标点。定义所有时刻的机器人位姿和路标点坐标为

$$\boldsymbol{x} = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\}, \quad \boldsymbol{y} = \{\boldsymbol{y}_1, \ldots, \boldsymbol{y}_M\}.$$

同样地，用不带下标的 $\boldsymbol{u}$ 表示所有时刻的输入，$\boldsymbol{z}$ 表示所有时刻的观测数据。那么我们说，对机器人状态的估计，从概率学的观点来看，就是已知输入数据 $\boldsymbol{u}$ 和观测数据 $\boldsymbol{z}$ 的条件下，求状态 $\boldsymbol{x}, \boldsymbol{y}$ 的条件概率分布：

$$P(\boldsymbol{x}, \boldsymbol{y}|\boldsymbol{z}, \boldsymbol{u}). \tag{6.4}$$

特别地，当我们不知道控制输入，只有一张张的图像时，即只考虑观测方程带来的数据时，相当于估计 $P(\boldsymbol{x}, \boldsymbol{y}|\boldsymbol{z})$ 的条件概率分布，此问题也称为 SfM，即如何从许多图像中重建三维空间结构[29]。

为了估计状态变量的条件分布，利用贝叶斯法则，有

$$P(\boldsymbol{x}, \boldsymbol{y}|\boldsymbol{z}, \boldsymbol{u}) = \frac{P(\boldsymbol{z}, \boldsymbol{u}|\boldsymbol{x}, \boldsymbol{y}) P(\boldsymbol{x}, \boldsymbol{y})}{P(\boldsymbol{z}, \boldsymbol{u})} \propto \underbrace{P(\boldsymbol{z}, \boldsymbol{u}|\boldsymbol{x}, \boldsymbol{y})}_{\text{似然}} \underbrace{P(\boldsymbol{x}, \boldsymbol{y})}_{\text{先验}}. \tag{6.5}$$

贝叶斯法则左侧称为**后验概率**，右侧的 $P(\boldsymbol{z}|\boldsymbol{x})$ 称为**似然**（Likehood），另一部分 $P(\boldsymbol{x})$ 称为**先验**（Prior）。**直接求后验分布是困难的，但是求一个状态最优估计，使得在该状态下后验概率最大化**，则是可行的：

$$(\boldsymbol{x}, \boldsymbol{y})^*_{\text{MAP}} = \arg\max P(\boldsymbol{x}, \boldsymbol{y}|\boldsymbol{z}, \boldsymbol{u}) = \arg\max P(\boldsymbol{z}, \boldsymbol{u}|\boldsymbol{x}, \boldsymbol{y}) P(\boldsymbol{x}, \boldsymbol{y}). \tag{6.6}$$

请注意贝叶斯法则的分母部分与待估计的状态 $\boldsymbol{x}, \boldsymbol{y}$ 无关，因而可以忽略。贝叶斯法则告诉我们，求解最大后验概率**等价于最大化似然和先验的乘积**。当然，我们也可以说，对不起，我不知道机

器人位姿或路标大概在什么地方，此时就没有了**先验**。那么，可以求解**最大似然估计**（Maximize Likelihood Estimation，MLE）：

$$(\boldsymbol{x}, \boldsymbol{y})^*_{\text{MLE}} = \arg\max P(\boldsymbol{z}, \boldsymbol{u} | \boldsymbol{x}, \boldsymbol{y}). \tag{6.7}$$

直观地讲，似然是指“在现在的位姿下，可能产生怎样的观测数据”。由于我们知道观测数据，所以最大似然估计可以理解成：“**在什么样的状态下，最可能产生现在观测到的数据**”。这就是最大似然估计的直观意义。

6.1.2　最小二乘的引出

那么，如何求最大似然估计呢？我们说，在高斯分布的假设下，最大似然能够有较简单的形式。回顾观测模型，对于某一次观测：

$$\boldsymbol{z}_{k,j} = h\left(\boldsymbol{y}_j, \boldsymbol{x}_k\right) + \boldsymbol{v}_{k,j},$$

由于我们假设了噪声项 $\boldsymbol{v}_k \sim \mathcal{N}\left(\boldsymbol{0}, \boldsymbol{Q}_{k,j}\right)$，所以观测数据的条件概率为

$$P(\boldsymbol{z}_{j,k} | \boldsymbol{x}_k, \boldsymbol{y}_j) = N\left(h(\boldsymbol{y}_j, \boldsymbol{x}_k), \boldsymbol{Q}_{k,j}\right).$$

它依然是一个高斯分布。考虑单次观测的最大似然估计，可以使用**最小化负对数**来求一个高斯分布的最大似然。

我们知道高斯分布在负对数下有较好的数学形式。考虑任意高维高斯分布 $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$，它的概率密度函数展开形式为

$$P\left(\boldsymbol{x}\right) = \frac{1}{\sqrt{(2\pi)^N \det(\boldsymbol{\Sigma})}} \exp\left(-\frac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu})^{\mathrm{T}} \boldsymbol{\Sigma}^{-1} \left(\boldsymbol{x} - \boldsymbol{\mu}\right)\right). \tag{6.8}$$

对其取负对数，则变为

$$-\ln\left(P\left(\boldsymbol{x}\right)\right) = \frac{1}{2}\ln\left((2\pi)^N \det\left(\boldsymbol{\Sigma}\right)\right) + \frac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu})^{\mathrm{T}} \boldsymbol{\Sigma}^{-1} \left(\boldsymbol{x} - \boldsymbol{\mu}\right). \tag{6.9}$$

因为对数函数是单调递增的，所以对原函数求最大化相当于对负对数求最小化。在最小化上式的 $\boldsymbol{x}$ 时，第一项与 $\boldsymbol{x}$ 无关，可以略去。于是，只要最小化右侧的二次型项，就得到了对状态的最大似然估计。代入 SLAM 的观测模型，相当于在求：

$$\begin{aligned}(\boldsymbol{x}_k, \boldsymbol{y}_j)^* &= \arg\max \mathcal{N}(h(\boldsymbol{y}_j, \boldsymbol{x}_k), \boldsymbol{Q}_{k,j}) \\ &= \arg\min\left(\left(\boldsymbol{z}_{k,j} - h\left(\boldsymbol{x}_k, \boldsymbol{y}_j\right)\right)^{\mathrm{T}} \boldsymbol{Q}_{k,j}^{-1} \left(\boldsymbol{z}_{k,j} - h\left(\boldsymbol{x}_k, \boldsymbol{y}_j\right)\right)\right).\end{aligned} \tag{6.10}$$

我们发现，该式等价于最小化噪声项（即误差）的一个二次型。这个二次型称为**马哈拉诺比斯距离**（Mahalanobis distance），又叫**马氏距离**。它也可以看成由 $\boldsymbol{Q}_{k,j}^{-1}$ 加权之后的欧氏距离（二范数），这里 $\boldsymbol{Q}_{k,j}^{-1}$ 也叫作**信息矩阵**，即高斯分布协方差矩阵之逆。

现在我们考虑批量时刻的数据。通常假设各个时刻的输入和观测是相互独立的，这意味着各个输入之间是独立的，各个观测之间是独立的，并且输入和观测也是独立的。于是我们可以对联合分布进行因式分解：

$$P(\boldsymbol{z},\boldsymbol{u}|\boldsymbol{x},\boldsymbol{y}) = \prod_k P(\boldsymbol{u}_k|\boldsymbol{x}_{k-1},\boldsymbol{x}_k)\prod_{k,j} P(\boldsymbol{z}_{k,j}|\boldsymbol{x}_k,\boldsymbol{y}_j), \tag{6.11}$$

这说明我们可以独立地处理各时刻的运动和观测。定义各次输入和观测数据与模型之间的误差：

$$\begin{aligned} \boldsymbol{e}_{\boldsymbol{u},k} &= \boldsymbol{x}_k - f(\boldsymbol{x}_{k-1},\boldsymbol{u}_k) \\ \boldsymbol{e}_{\boldsymbol{z},j,k} &= \boldsymbol{z}_{k,j} - h(\boldsymbol{x}_k,\boldsymbol{y}_j), \end{aligned} \tag{6.12}$$

那么，最小化所有时刻估计值与真实读数之间的马氏距离，等价于求最大似然估计。负对数允许我们把乘积变成求和：

$$\min J(\boldsymbol{x},\boldsymbol{y}) = \sum_k \boldsymbol{e}_{\boldsymbol{u},k}^{\mathrm{T}} \boldsymbol{R}_k^{-1} \boldsymbol{e}_{\boldsymbol{u},k} + \sum_k \sum_j \boldsymbol{e}_{\boldsymbol{z},k,j}^{\mathrm{T}} \boldsymbol{Q}_{k,j}^{-1} \boldsymbol{e}_{\boldsymbol{z},k,j}. \tag{6.13}$$

这样就得到了一个**最小二乘问题**（Least Square Problem），它的解等价于状态的最大似然估计。直观上看，由于噪声的存在，当我们把估计的轨迹与地图代入 SLAM 的运动、观测方程中时，它们并不会完美地成立。这时怎么办呢？我们对状态的估计值进行**微调**，使得整体的误差下降一些。当然，这个下降也有限度，它一般会到达一个**极小值**。这就是一个典型的非线性优化的过程。

仔细观察式 (6.13)，我们发现 SLAM 中的最小二乘问题具有一些特定的结构：

- 首先，整个问题的目标函数由许多个误差的（加权的）二次型组成。虽然总体的状态变量维数很高，但每个误差项都是简单的，仅与一两个状态变量有关。例如，运动误差只与 $\boldsymbol{x}_{k-1},\boldsymbol{x}_k$ 有关，观测误差只与 $\boldsymbol{x}_k,\boldsymbol{y}_j$ 有关。这种关系会让整个问题有一种**稀疏**的形式，我们将在介绍后端的章节中看到。
- 其次，如果使用李代数表示增量，则该问题是**无约束**的最小二乘问题。但如果用旋转矩阵/变换矩阵描述位姿，则会引入旋转矩阵自身的约束，即需在问题中加入 s.t. $\boldsymbol{R}^{\mathrm{T}}\boldsymbol{R} = \boldsymbol{I}$ 且 $\det(\boldsymbol{R}) = 1$ 这样令人头大的条件。额外的约束会使优化变得更困难。这体现了李代数的优势。
- 最后，我们使用了二次型度量误差。误差的分布将影响此项在整个问题中的权重。例如，某次的观测非常准确，那么协方差矩阵就会“小”，而信息矩阵就会“大”，所以这个误

差项会在整个问题中占有较高的权重。我们之后也会看到它存在一些问题，但是目前先不讨论。

现在，我们介绍如何求解这个最小二乘问题，这需要一些**非线性优化的基本知识**。特别地，我们要针对这样一个通用的无约束非线性最小二乘问题，探讨它是如何求解的。在后续几讲中，我们会大量使用本讲的结果，详细讨论它在 SLAM 前端、后端中的应用。

6.1.3　例子：批量状态估计

笔者发现在这里举一个简单的例子会更好一些。考虑一个非常简单的离散时间系统：

$$\begin{aligned} \boldsymbol{x}_k &= \boldsymbol{x}_{k-1} + \boldsymbol{u}_k + \boldsymbol{w}_k, & \boldsymbol{w}_k &\sim \mathcal{N}(0, \boldsymbol{Q}_k) \\ \boldsymbol{z}_k &= \boldsymbol{x}_k + \boldsymbol{n}_k, & \boldsymbol{n}_k &\sim \mathcal{N}(0, \boldsymbol{R}_k) \end{aligned} \tag{6.14}$$

这可以表达一辆沿 x 轴前进或后退的汽车。第一个公式为运动方程，$\boldsymbol{u}_k$ 为输入，$\boldsymbol{w}_k$ 为噪声；第二个公式为观测方程，$\boldsymbol{z}_k$ 为对汽车位置的测量。取时间 $k = 1, \ldots, 3$，现希望根据已有的 $\boldsymbol{v}, \boldsymbol{y}$ 进行状态估计。设初始状态 $\boldsymbol{x}_0$ 已知。下面来推导批量状态的最大似然估计。

首先，令批量状态变量为 $\boldsymbol{x} = [\boldsymbol{x}_0, \boldsymbol{x}_1, \boldsymbol{x}_2, \boldsymbol{x}_3]^{\mathrm{T}}$，令批量观测为 $\boldsymbol{z} = [\boldsymbol{z}_1, \boldsymbol{z}_2, \boldsymbol{z}_3]^{\mathrm{T}}$，按同样方式定义 $\boldsymbol{u} = [\boldsymbol{u}_1, \boldsymbol{u}_2, \boldsymbol{u}_3]^{\mathrm{T}}$。按照先前的推导，我们知道最大似然估计为

$$\begin{aligned} \boldsymbol{x}^*_{\mathrm{map}} &= \arg\max P(\boldsymbol{x}|\boldsymbol{u}, \boldsymbol{z}) = \arg\max P(\boldsymbol{u}, \boldsymbol{z}|\boldsymbol{x}) \\ &= \prod_{k=1}^{3} P(\boldsymbol{u}_k|\boldsymbol{x}_{k-1}, \boldsymbol{x}_k) \prod_{k=1}^{3} P(\boldsymbol{z}_k|\boldsymbol{x}_k), \end{aligned} \tag{6.15}$$

对于具体的每一项，比如运动方程，我们知道：

$$P(\boldsymbol{u}_k|\boldsymbol{x}_{k-1}, \boldsymbol{x}_k) = \mathcal{N}(\boldsymbol{x}_k - \boldsymbol{x}_{k-1}, \boldsymbol{Q}_k), \tag{6.16}$$

观测方程也是类似的：

$$P(\boldsymbol{z}_k|\boldsymbol{x}_k) = \mathcal{N}(\boldsymbol{x}_k, \boldsymbol{R}_k). \tag{6.17}$$

根据这些方法，我们就能够实际地解决上面的批量状态估计问题。根据之前的叙述，可以构建误差变量：

$$\boldsymbol{e}_{\boldsymbol{u},k} = \boldsymbol{x}_k - \boldsymbol{x}_{k-1} - \boldsymbol{u}_k, \quad \boldsymbol{e}_{z,k} = \boldsymbol{z}_k - \boldsymbol{x}_k, \tag{6.18}$$

于是最小二乘的目标函数为

$$\min \sum_{k=1}^{3} \boldsymbol{e}_{\boldsymbol{u},k}^{\mathrm{T}} \boldsymbol{Q}_k^{-1} \boldsymbol{e}_{\boldsymbol{u},k} + \sum_{k=1}^{3} \boldsymbol{e}_{\boldsymbol{z},k}^{\mathrm{T}} \boldsymbol{R}_k^{-1} \boldsymbol{e}_{z,k}. \tag{6.19}$$

此外，这个系统是线性系统，我们可以很容易地将它写成向量形式。定义向量 $\boldsymbol{y} = [\boldsymbol{u}, \boldsymbol{z}]^{\mathrm{T}}$，那么可以写出矩阵 $\boldsymbol{H}$，使得

$$\boldsymbol{y} - \boldsymbol{H}\boldsymbol{x} = \boldsymbol{e} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{\Sigma}). \tag{6.20}$$

那么：

$$\boldsymbol{H} = \left[\begin{array}{cccc} 1 & -1 & 0 & 0 \\ 0 & 1 & -1 & 0 \\ 0 & 0 & 1 & -1 \\ \hline 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{array}\right], \tag{6.21}$$

且 $\boldsymbol{\Sigma} = \mathrm{diag}(\boldsymbol{Q}_1, \boldsymbol{Q}_2, \boldsymbol{Q}_3, \boldsymbol{R}_1, \boldsymbol{R}_2, \boldsymbol{R}_3)$。整个问题可以写成

$$\boldsymbol{x}^*_{\mathrm{map}} = \arg\min \boldsymbol{e}^{\mathrm{T}} \boldsymbol{\Sigma}^{-1} \boldsymbol{e}, \tag{6.22}$$

之后我们将看到，这个问题有唯一的解：

$$\boldsymbol{x}^*_{\mathrm{map}} = (\boldsymbol{H}^{\mathrm{T}} \boldsymbol{\Sigma}^{-1} \boldsymbol{H})^{-1} \boldsymbol{H}^{\mathrm{T}} \boldsymbol{\Sigma}^{-1} \boldsymbol{y}. \tag{6.23}$$

6.2 非线性最小二乘

先来考虑一个简单的最小二乘问题：

$$\min_{\boldsymbol{x}} F(\boldsymbol{x}) = \frac{1}{2} \| f(\boldsymbol{x}) \|_2^2. \tag{6.24}$$

其中，自变量 $\boldsymbol{x} \in \mathbb{R}^n$，$f$ 是任意标量非线性函数 $f(\boldsymbol{x}) : \mathbb{R}^n \mapsto \mathbb{R}$。注意这里的系数 $\frac{1}{2}$ 是无关紧要的，有些文献上带有这个系数，有些文献则不带，它不会影响之后的结论。下面讨论如何求解这样一个优化问题。显然，如果 f 是个数学形式上很简单的函数，那么该问题可以用解析形式来求。令目标函数的导数为零，然后求解 $\boldsymbol{x}$ 的最优值，就和求二元函数的极值一样：

$$\frac{\mathrm{d}F}{\mathrm{d}\boldsymbol{x}} = \boldsymbol{0}. \tag{6.25}$$

解此方程，就得到了导数为零处的极值。它们可能是极大、极小或鞍点处的值，只要逐个比较它们的函数值大小即可。但是，这个方程是否容易求解呢？这取决于 f 导函数的形式。如果 f 为简

单的线性函数，那么这个问题就是简单的线性最小二乘问题，但是有些导函数可能形式复杂，使得该方程可能不容易求解。求解这个方程需要我们知道关于目标函数的**全局性质**，而通常这是不大可能的。对于不方便直接求解的最小二乘问题，我们可以用**迭代**的方式，从一个初始值出发，不断地更新当前的优化变量，使目标函数下降。具体步骤可列写如下：

1. 给定某个初始值 $\boldsymbol{x}_0$。
2. 对于第 k 次迭代，寻找一个增量 $\Delta \boldsymbol{x}_k$，使得 $\|f(\boldsymbol{x}_k + \Delta \boldsymbol{x}_k)\|_2^2$ 达到极小值。
3. 若 $\Delta \boldsymbol{x}_k$ 足够小，则停止。
4. 否则，令 $\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \Delta \boldsymbol{x}_k$，返回第 2 步。

这让求解**导函数为零**的问题变成了一个不断**寻找下降增量** $\Delta \boldsymbol{x}_k$ 的问题，我们将看到，由于可以对 f 进行线性化，增量的计算将简单很多。当函数下降直到增量非常小的时候，就认为算法收敛，目标函数达到了一个极小值。在这个过程中，问题在于如何找到每次迭代点的增量，而这是一个局部的问题，我们只需要关心 f 在迭代值处的局部性质而非全局性质。这类方法在最优化、机器学习等领域应用非常广泛。

接下来，我们考察如何寻找这个增量 $\Delta \boldsymbol{x}_k$。这部分知识实际属于数值优化的领域，我们来看一些广泛使用的结果。

6.2.1　一阶和二阶梯度法

现在考虑第 k 次迭代，假设我们在 $\boldsymbol{x}_k$ 处，想要寻到增量 $\Delta \boldsymbol{x}_k$，那么最直观的方式是将目标函数在 $\boldsymbol{x}_k$ 附近进行泰勒展开：

$$F(\boldsymbol{x}_k + \Delta \boldsymbol{x}_k) \approx F(\boldsymbol{x}_k) + \boldsymbol{J}(\boldsymbol{x}_k)^{\mathrm{T}} \Delta \boldsymbol{x}_k + \frac{1}{2} \Delta \boldsymbol{x}_k^{\mathrm{T}} \boldsymbol{H}(\boldsymbol{x}_k) \Delta \boldsymbol{x}_k. \tag{6.26}$$

其中 $\boldsymbol{J}(\boldsymbol{x}_k)$ 是 $F(\boldsymbol{x})$ 关于 $\boldsymbol{x}$ 的一阶导数〔也叫梯度、**雅可比**（Jacobian）矩阵〕[①]，$\boldsymbol{H}$ 则是二阶导数〔**海塞**（Hessian）矩阵〕，它们都在 $\boldsymbol{x}_k$ 处取值，读者应该在大学本科多元微积分课程中学习过。我们可以选择保留泰勒展开的一阶或二阶项，那么对应的求解方法则称为一阶梯度或二阶梯度法。如果保留一阶梯度，那么取增量为反向的梯度，即可保证函数下降：

$$\Delta \boldsymbol{x}^* = -\boldsymbol{J}(\boldsymbol{x}_k). \tag{6.27}$$

当然这只是个方向，通常我们还要再指定一个步长 λ。步长可以根据一定的条件来计算[30]，在机器学习中也有一些经验性质的方法，但我们不展开谈。这种方法被称为**最速下降法**。它的直观意义非常简单，只要我们沿着反向梯度方向前进，在一阶（线性）的近似下，目标函数必定会下降。

①我们把 $\boldsymbol{J}(\boldsymbol{x})$ 写成列向量，它可以和 $\Delta \boldsymbol{x}$ 进行内积，得到一个标量。

注意，以上讨论都是在第 k 次迭代时进行的，并不涉及其他的迭代信息。所以为了简化符号，后面我们省略下标 k，并认为这些讨论对任意一次迭代都成立。

另外，我们可选择保留二阶梯度信息，此时增量方程为

$$\Delta \boldsymbol{x}^* = \arg\min \left(F(\boldsymbol{x}) + \boldsymbol{J}(\boldsymbol{x})^{\mathrm{T}} \Delta \boldsymbol{x} + \frac{1}{2} \Delta \boldsymbol{x}^{\mathrm{T}} \boldsymbol{H} \Delta \boldsymbol{x} \right). \tag{6.28}$$

右侧只含 $\Delta \boldsymbol{x}$ 的零次、一次和二次项。求右侧等式关于 $\Delta \boldsymbol{x}$ 的导数并令它为零[①]，得到

$$\boldsymbol{J} + \boldsymbol{H} \Delta \boldsymbol{x} = \boldsymbol{0} \Rightarrow \boldsymbol{H} \Delta \boldsymbol{x} = -\boldsymbol{J}. \tag{6.29}$$

求解这个线性方程，就得到了增量。该方法又称为**牛顿法**。

我们看到，一阶和二阶梯度法都十分直观，只要把函数在迭代点附近进行泰勒展开，并针对更新量做最小化即可。事实上，我们用一个一次或二次的函数近似了原函数，然后用近似函数的最小值来猜测原函数的极小值。只要原目标函数局部看起来像一次或二次函数，这类算法就是成立的（这也是现实中的情形）。不过，这两种方法也存在它们自身的问题。最速下降法过于贪心，容易走出锯齿路线，反而增加了迭代次数。而牛顿法则需要计算目标函数的 $\boldsymbol{H}$ 矩阵，这在问题规模较大时非常困难，我们通常倾向于避免 $\boldsymbol{H}$ 的计算。对于一般的问题，一些拟牛顿法可以得到较好的结果，而对于最小二乘问题，还有几类更实用的方法：**高斯牛顿法**和**列文伯格—马夸尔特方法**。

6.2.2 高斯牛顿法

高斯牛顿法是最优化算法中最简单的方法之一。它的思想是将 $f(\boldsymbol{x})$ 进行一阶的泰勒展开。请注意这里不是目标函数 $F(\boldsymbol{x})$ 而是 $f(\boldsymbol{x})$，否则就变成牛顿法了。

$$f(\boldsymbol{x} + \Delta \boldsymbol{x}) \approx f(\boldsymbol{x}) + \boldsymbol{J}(\boldsymbol{x})^{\mathrm{T}} \Delta \boldsymbol{x}. \tag{6.30}$$

这里 $\boldsymbol{J}(\boldsymbol{x})^{\mathrm{T}}$ 为 $f(\boldsymbol{x})$ 关于 $\boldsymbol{x}$ 的导数，为 $n \times 1$ 的列向量。根据前面的框架，当前的目标是寻找增量 $\Delta \boldsymbol{x}$，使得 $\|f(\boldsymbol{x} + \Delta \boldsymbol{x})\|^2$ 达到最小。为了求 $\Delta \boldsymbol{x}$，我们需要解一个线性的最小二乘问题：

$$\Delta \boldsymbol{x}^* = \arg\min_{\Delta \boldsymbol{x}} \frac{1}{2} \left\| f(\boldsymbol{x}) + \boldsymbol{J}(\boldsymbol{x})^{\mathrm{T}} \Delta \boldsymbol{x} \right\|^2. \tag{6.31}$$

这个方程与之前的有什么不一样呢？根据极值条件，将上述目标函数对 $\Delta \boldsymbol{x}$ 求导，并令导数为零。为此，先展开目标函数的平方项：

---

①对矩阵求导不熟悉的同学请参考附录 B。

$$\frac{1}{2}\left\|f(\boldsymbol{x})+\boldsymbol{J}(\boldsymbol{x})^{\mathrm{T}}\Delta\boldsymbol{x}\right\|^2=\frac{1}{2}\left(f(\boldsymbol{x})+\boldsymbol{J}(\boldsymbol{x})^{\mathrm{T}}\Delta\boldsymbol{x}\right)^{\mathrm{T}}\left(f(\boldsymbol{x})+\boldsymbol{J}(\boldsymbol{x})^{\mathrm{T}}\Delta\boldsymbol{x}\right)$$
$$=\frac{1}{2}\left(\|f(\boldsymbol{x})\|_2^2+2f(\boldsymbol{x})\boldsymbol{J}(\boldsymbol{x})^{\mathrm{T}}\Delta\boldsymbol{x}+\Delta\boldsymbol{x}^{\mathrm{T}}\boldsymbol{J}(\boldsymbol{x})\boldsymbol{J}(\boldsymbol{x})^{\mathrm{T}}\Delta\boldsymbol{x}\right).$$

求上式关于 $\Delta\boldsymbol{x}$ 的导数，并令其为零：

$$\boldsymbol{J}(\boldsymbol{x})f(\boldsymbol{x})+\boldsymbol{J}(\boldsymbol{x})\boldsymbol{J}^{\mathrm{T}}(\boldsymbol{x})\Delta\boldsymbol{x}=\boldsymbol{0}.$$

可以得到如下方程组：

$$\underbrace{\boldsymbol{J}(\boldsymbol{x})\boldsymbol{J}^{\mathrm{T}}(\boldsymbol{x})}_{\boldsymbol{H}(\boldsymbol{x})}\Delta\boldsymbol{x}=\underbrace{-\boldsymbol{J}(\boldsymbol{x})f(\boldsymbol{x})}_{\boldsymbol{g}(\boldsymbol{x})}. \tag{6.32}$$

这个方程是关于变量 $\Delta\boldsymbol{x}$ 的**线性方程组**，我们称它为**增量方程**，也可以称为**高斯牛顿方程**（Gauss-Newton equation）或者**正规方程**（Normal equation）。我们把左边的系数定义为 $\boldsymbol{H}$，右边定义为 $\boldsymbol{g}$，那么上式变为

$$\boldsymbol{H}\Delta\boldsymbol{x}=\boldsymbol{g}. \tag{6.33}$$

这里把左侧记作 $\boldsymbol{H}$ 是有意义的。对比牛顿法可见，高斯牛顿法用 $\boldsymbol{J}\boldsymbol{J}^{\mathrm{T}}$ **作为牛顿法中二阶 Hessian 矩阵的近似**，从而省略了计算 $\boldsymbol{H}$ 的过程。**求解增量方程是整个优化问题的核心所在**。如果我们能够顺利解出该方程，那么高斯牛顿法的算法步骤可以写成

1. 给定初始值 $\boldsymbol{x}_0$。
2. 对于第 k 次迭代，求出当前的雅可比矩阵 $\boldsymbol{J}(\boldsymbol{x}_k)$ 和误差 $f(\boldsymbol{x}_k)$。
3. 求解增量方程：$\boldsymbol{H}\Delta\boldsymbol{x}_k=\boldsymbol{g}$。
4. 若 $\Delta\boldsymbol{x}_k$ 足够小，则停止。否则，令 $\boldsymbol{x}_{k+1}=\boldsymbol{x}_k+\Delta\boldsymbol{x}_k$，返回第 2 步。

从算法步骤中可以看到，增量方程的求解占据着主要地位。只要我们能够顺利解出增量，就能保证目标函数能够正确地下降。

为了求解增量方程，我们需要求解 $\boldsymbol{H}^{-1}$，这需要 $\boldsymbol{H}$ 矩阵可逆，但实际数据中计算得到的 $\boldsymbol{J}\boldsymbol{J}^{\mathrm{T}}$ 却只有半正定性。也就是说，在使用高斯牛顿法时，可能出现 $\boldsymbol{J}\boldsymbol{J}^{\mathrm{T}}$ 为奇异矩阵或者病态（ill-condition）的情况，此时增量的稳定性较差，导致算法不收敛。直观地说，原函数在这个点的局部近似不像一个二次函数。更严重的是，就算我们假设 $\boldsymbol{H}$ 非奇异也非病态，如果我们求出来的步长 $\Delta\boldsymbol{x}$ 太大，也会导致我们采用的局部近似式 (6.30) 不够准确，这样一来我们甚至无法保证它的迭代收敛，哪怕是让目标函数变得更大都是有可能的。

尽管高斯牛顿法有这些缺点，但它依然算是非线性优化方面一种简单有效的方法，值得我们学习。在非线性优化领域，相当多的算法都可以归结为高斯牛顿法的变种。这些算法都借助了高斯牛顿法的思想并且通过自己的改进修正其缺点。例如，一些**线搜索方法**（Line Search Method）

加入了一个步长 α，在确定了 $\Delta \boldsymbol{x}$ 后进一步找到 α 使得 $\|f(\boldsymbol{x}+\alpha\Delta\boldsymbol{x})\|^2$ 达到最小，而不是简单地令 $\alpha=1$。

列文伯格—马夸尔特方法在一定程度上修正了这些问题。一般认为它比高斯牛顿法更为健壮，但它的收敛速度可能比高斯牛顿法更慢，被称为**阻尼牛顿法**（Damped Newton Method）。

6.2.3 列文伯格—马夸尔特方法

高斯牛顿法中采用的近似二阶泰勒展开只能在展开点附近有较好的近似效果，所以我们很自然地想到应该给 $\Delta \boldsymbol{x}$ 添加一个范围，称为**信赖区域**（Trust Region）。这个范围定义了在什么情况下二阶近似是有效的，这类方法也称为**信赖区域方法**（Trust Region Method）。在信赖区域里，我们认为近似是有效的；出了这个区域，近似可能会出问题。

那么，如何确定这个信赖区域的范围呢？一个比较好的方法是根据我们的近似模型跟实际函数之间的差异来确定：如果差异小，说明近似效果好，我们扩大近似的范围；反之，如果差异大，就缩小近似的范围。我们定义一个指标 ρ 来刻画近似的好坏程度：

$$\rho=\frac{f(\boldsymbol{x}+\Delta\boldsymbol{x})-f(\boldsymbol{x})}{\boldsymbol{J}(\boldsymbol{x})^{\mathrm{T}}\Delta\boldsymbol{x}}. \tag{6.34}$$

ρ 的分子是实际函数下降的值，分母是近似模型下降的值。如果 ρ 接近于 1，则近似是好的。如果 ρ 太小，说明实际减小的值远少于近似减小的值，则认为近似比较差，需要缩小近似范围。反之，如果 ρ 比较大，则说明实际下降的比预计的更大，我们可以放大近似范围。

于是，我们构建一个改良版的非线性优化框架，该框架会比高斯牛顿法有更好的效果：

1. 给定初始值 $\boldsymbol{x}_0$，以及初始优化半径 μ。
2. 对于第 k 次迭代，在高斯牛顿法的基础上加上信赖区域，求解：

$$\min_{\Delta\boldsymbol{x}_k}\frac{1}{2}\left\|f(\boldsymbol{x}_k)+\boldsymbol{J}(\boldsymbol{x}_k)^{\mathrm{T}}\Delta\boldsymbol{x}_k\right\|^2,\quad \text{s.t.}\quad \|\boldsymbol{D}\Delta\boldsymbol{x}_k\|^2\leqslant\mu, \tag{6.35}$$

   其中，μ 是信赖区域的半径，$\boldsymbol{D}$ 为系数矩阵，将在后文说明。
3. 按式 (6.34) 计算 ρ。
4. 若 $\rho>\frac{3}{4}$，则设置 $\mu=2\mu$。
5. 若 $\rho<\frac{1}{4}$，则设置 $\mu=0.5\mu$。
6. 如果 ρ 大于某阈值，则认为近似可行。令 $\boldsymbol{x}_{k+1}=\boldsymbol{x}_k+\Delta\boldsymbol{x}_k$。
7. 判断算法是否收敛。如不收敛则返回第 2 步，否则结束。

这里近似范围扩大的倍数和阈值都是经验值，可以替换成别的数值。在式 (6.35) 中，我们把增量限定于一个半径为 μ 的球中，认为只在这个球内才是有效的。带上 $\boldsymbol{D}$ 之后，这个球可以看

成一个椭球。在列文伯格提出的优化方法中，把 $\boldsymbol{D}$ 取成单位阵 $\boldsymbol{I}$，相当于直接把 $\Delta\boldsymbol{x}_k$ 约束在一个球中。随后，马夸尔特提出将 $\boldsymbol{D}$ 取成非负数对角阵——实际中通常用 $\boldsymbol{J}^T\boldsymbol{J}$ 的对角元素平方根，使得在梯度小的维度上约束范围更大一些。

无论如何，在列文伯格—马夸尔特优化中，我们都需要解式 (6.35) 那样一个子问题来获得梯度。这个子问题是带不等式约束的优化问题，我们用拉格朗日乘子把约束项放到目标函数中，构成拉格朗日函数：

$$\mathcal{L}(\Delta\boldsymbol{x}_k,\lambda)=\frac{1}{2}\left\|f(\boldsymbol{x}_k)+\boldsymbol{J}(\boldsymbol{x}_k)^{\mathrm{T}}\Delta\boldsymbol{x}_k\right\|^2+\frac{\lambda}{2}\left(\|\boldsymbol{D}\Delta\boldsymbol{x}_k\|^2-\mu\right). \tag{6.36}$$

这里 λ 为拉格朗日乘子。类似于高斯牛顿法中的做法，令该拉格朗日函数关于 $\Delta\boldsymbol{x}$ 的导数为零，它的核心仍是计算增量的线性方程：

$$\left(\boldsymbol{H}+\lambda\boldsymbol{D}^{\mathrm{T}}\boldsymbol{D}\right)\Delta\boldsymbol{x}_k=\boldsymbol{g}. \tag{6.37}$$

可以看到，相比于高斯牛顿法，增量方程多了一项 $\lambda\boldsymbol{D}^T\boldsymbol{D}$。如果考虑它的简化形式，即 $\boldsymbol{D}=\boldsymbol{I}$，那么相当于求解[①]：

$$(\boldsymbol{H}+\lambda\boldsymbol{I})\,\Delta\boldsymbol{x}_k=\boldsymbol{g}.$$

我们看到，一方面，当参数 λ 比较小时，$\boldsymbol{H}$ 占主要地位，这说明二次近似模型在该范围内是比较好的，列文伯格—马夸尔特方法更接近于高斯牛顿法。另一方面，当 λ 比较大时，$\lambda\boldsymbol{I}$ 占据主要地位，列文伯格—马夸尔特方法更接近于一阶梯度下降法（即最速下降），这说明附近的二次近似不够好。列文伯格—马夸尔特方法的求解方式，可在一定程度上避免线性方程组的系数矩阵的非奇异和病态问题，提供更稳定、更准确的增量 $\Delta\boldsymbol{x}$。

在实际中，还存在许多其他的方式来求解增量，例如 Dog-Leg[31] 等方法。我们在这里所介绍的，只是最常见而且最基本的方法，也是视觉 SLAM 中用得最多的方法。实际问题中，我们通常选择高斯牛顿法或列文伯格—马夸尔特方法中的一种作为梯度下降策略。当问题性质较好时，用高斯牛顿。如果问题接近病态，则用列文伯格—马夸尔特方法。

小结

由于不希望本书变成一本让人头疼的数学教科书，所以这里只罗列了最常见的两种非线性优化方案——高斯牛顿法和列文伯格—马夸尔特方法。我们避开了许多数学性质上的讨论。如果

①严谨的读者可能不满意此处的叙述。信赖域原问题的约束条件除了拉格朗日函数求导为零，KKT 条件还会有一些别的约束：$\lambda>0$，且 $\lambda(\|\boldsymbol{D}\Delta\boldsymbol{x}\|^2-\mu)=0$。但是在 L-M 迭代中，我们不妨把它看成在原问题的目标函数上，以 λ 为权重的惩罚项（Argumented Lagrangian）。在每一步迭代后，若发现信赖域条件不满足，或者目标函数增加，就增加 λ 的权重，直到最终满足信赖域条件。所以，理论上对 L-M 算法存在不同的解释，但实际中我们只关心它是否顺利工作。

读者对优化感兴趣，可以进一步阅读专门介绍数值优化的书籍（这是一个很大的课题）[31]。以高斯牛顿法和列文伯格—马夸尔特方法为代表的优化方法，在很多开源的优化库中都已经实现并提供给用户，我们会在下文进行实验。最优化是处理许多实际问题的基本数学工具，不光在视觉 SLAM 中起着核心作用，在类似于深度学习等其他领域，它也是求解问题的核心方法之一（深度学习数据量很大，以一阶方法为主）。我们希望读者能够根据自身能力，去了解更多的最优化算法。

也许你发现了，无论是高斯牛顿法还是列文伯格—马夸尔特方法，在做最优化计算时，都需要提供变量的初始值。你也许会问，这个初始值能否随意设置？当然不能。实际上，非线性优化的所有迭代求解方案，都需要用户提供一个良好的初始值。由于目标函数太复杂，导致在求解空间上的变化难以预测，对问题提供不同的初始值往往会导致不同的计算结果。这种情况是非线性优化的通病：大多数算法都容易陷入局部极小值。因此，无论是哪类科学问题，我们提供初始值都应该有科学依据，例如视觉 SLAM 问题中，我们会用 ICP、PnP 之类的算法提供优化初始值。总之，一个良好的初始值对最优化问题非常重要！

也许读者还会对上面提到的最优化产生疑问：如何求解线性增量方程组呢？我们只讲到了增量方程是一个线性方程，但是直接对系数矩阵进行求逆岂不是要进行大量的计算？当然不是。在视觉 SLAM 算法里，经常遇到 $\Delta \boldsymbol{x}$ 的维度大到好几百或者上千，如果你要做大规模的视觉三维重建，就会经常发现这个维度可以轻易达到几十万甚至更高的级别。要对那么大个矩阵进行求逆是大多数处理器无法负担的，因此存在着许多针对线性方程组的数值求解方法。在不同的领域有不同的求解方式，但几乎没有一种方式是直接求系数矩阵的逆，我们会采用矩阵分解的方法来解线性方程，例如 QR、Cholesky 等分解方法。这些方法通常在矩阵论等教科书中可以找到，我们不多加介绍。

幸运的是，视觉 SLAM 里这个矩阵往往有特定的稀疏形式，这为实时求解优化问题提供了可能性。我们将在第 9 讲中详细介绍它的原理。利用稀疏形式的消元、分解，最后进行求解增量，会让求解的效率大大提高。在很多开源的优化库上，维度为一万多的变量在一般的计算机上就可以在几秒甚至更短的时间内被求解出来，其原因也是用了更加高级的数学工具。视觉 SLAM 算法现在能够实时地实现，也多亏了系数矩阵是稀疏的，如果矩阵是稠密的，恐怕优化这类视觉 SLAM 算法就不会被学界广泛采纳了[32–34]。

6.3 实践：曲线拟合问题

6.3.1 手写高斯牛顿法

接下来，我们用一个简单的例子来说明如何求解最小二乘问题。我们将演示如何手写高斯牛顿法，然后介绍如何使用优化库求解此问题。对于同一个问题，这些实现方式会得到同样的结果，

因为它们的核心算法是一样的。

考虑一条满足以下方程的曲线：

$$y = \exp(ax^2 + bx + c) + w,$$

其中，a, b, c 为曲线的参数，w 为高斯噪声，满足 $w \sim (0, \sigma^2)$。我们故意选择了这样一个非线性模型，使问题不至于太简单。现在，假设我们有 N 个关于 x, y 的观测数据点，想根据这些数据点求出曲线的参数。那么，可以求解下面的最小二乘问题以估计曲线参数：

$$\min_{a,b,c} \frac{1}{2} \sum_{i=1}^{N} \left\| y_i - \exp\left(ax_i^2 + bx_i + c\right) \right\|^2. \tag{6.38}$$

请注意，在这个问题中，待估计的变量是 a, b, c，而不是 x。我们的程序里先根据模型生成 x, y 的真值，然后在真值中添加高斯分布的噪声。随后，使用高斯牛顿法从带噪声的数据拟合参数模型。定义误差为

$$e_i = y_i - \exp\left(ax_i^2 + bx_i + c\right), \tag{6.39}$$

那么，可以求出每个误差项对于状态变量的导数：

$$\begin{aligned} \frac{\partial e_i}{\partial a} &= -x_i^2 \exp\left(ax_i^2 + bx_i + c\right) \\ \frac{\partial e_i}{\partial b} &= -x_i \exp\left(ax_i^2 + bx_i + c\right) \\ \frac{\partial e_i}{\partial c} &= -\exp\left(ax_i^2 + bx_i + c\right) \end{aligned} \tag{6.40}$$

于是 $\boldsymbol{J}_i = \left[\frac{\partial e_i}{\partial a}, \frac{\partial e_i}{\partial b}, \frac{\partial e_i}{\partial c}\right]^{\mathrm{T}}$，高斯牛顿法的增量方程为

$$\left(\sum_{i=1}^{100} \boldsymbol{J}_i (\sigma^2)^{-1} \boldsymbol{J}_i^{\mathrm{T}} \right) \Delta \boldsymbol{x}_k = \sum_{i=1}^{100} -\boldsymbol{J}_i (\sigma^2)^{-1} e_i, \tag{6.41}$$

当然，我们也可以选择把所有的 $\boldsymbol{J}_i$ 排成一列，将这个方程写成矩阵形式，不过它的含义与求和形式是一致的。下面的代码演示了这个过程是如何进行的。

slambook2/ch6/gaussNewton.cpp

```
#include <iostream>
#include <opencv2/opencv.hpp>
#include <Eigen/Core>
#include <Eigen/Dense>
```

```
5
6  using namespace std;
7  using namespace Eigen;
8
9  int main(int argc, char **argv) {
10     double ar = 1.0, br = 2.0, cr = 1.0;          // 真实参数值
11     double ae = 2.0, be = -1.0, ce = 5.0;         // 估计参数值
12     int N = 100;                                  // 数据点
13     double w_sigma = 1.0;                         // 噪声Sigma值
14     double inv_sigma = 1.0 / w_sigma;
15     cv::RNG rng;                                  // OpenCV随机数产生器
16
17     vector<double> x_data, y_data;      // 数据
18     for (int i = 0; i < N; i++) {
19         double x = i / 100.0;
20         x_data.push_back(x);
21         y_data.push_back(exp(ar * x * x + br * x + cr) + rng.gaussian(w_sigma * w_sigma));
22     }
23
24     // 开始Gauss-Newton迭代
25     int iterations = 100;    // 迭代次数
26     double cost = 0, lastCost = 0;  // 本次迭代的cost和上一次迭代的cost
27
28     chrono::steady_clock::time_point t1 = chrono::steady_clock::now();
29     for (int iter = 0; iter < iterations; iter++) {
30
31         Matrix3d H = Matrix3d::Zero();             // Hessian = J^T W^{-1} J in Gauss-Newton
32         Vector3d b = Vector3d::Zero();             // bias
33         cost = 0;
34
35         for (int i = 0; i < N; i++) {
36             double xi = x_data[i], yi = y_data[i];  // 第i个数据点
37             double error = yi - exp(ae * xi * xi + be * xi + ce);
38             Vector3d J; // 雅可比矩阵
39             J[0] = -xi * xi * exp(ae * xi * xi + be * xi + ce);  // de/da
40             J[1] = -xi * exp(ae * xi * xi + be * xi + ce);  // de/db
41             J[2] = -exp(ae * xi * xi + be * xi + ce);  // de/dc
42
43             H += inv_sigma * inv_sigma * J * J.transpose();
44             b += -inv_sigma * inv_sigma * error * J;
45
46             cost += error * error;
47         }
```

```
48
49          // 求解线性方程 Hx=b
50          Vector3d dx = H.ldlt().solve(b);
51          if (isnan(dx[0])) {
52              cout << "result is nan!" << endl;
53              break;
54          }
55
56          if (iter > 0 && cost >= lastCost) {
57              cout << "cost: " << cost << ">= last cost: " << lastCost << ", break." << endl;
58              break;
59          }
60
61          ae += dx[0];
62          be += dx[1];
63          ce += dx[2];
64
65          lastCost = cost;
66
67          cout << "total cost: " << cost << ", \t\tupdate: " << dx.transpose() <<
68            "\t\testimated params: " << ae << "," << be << "," << ce << endl;
69      }
70
71      chrono::steady_clock::time_point t2 = chrono::steady_clock::now();
72      chrono::duration<double> time_used = chrono::duration_cast<chrono::duration<double>>(t2 -
         t1);
73      cout << "solve time cost = " << time_used.count() << " seconds. " << endl;
74      cout << "estimated abc = " << ae << ", " << be << ", " << ce << endl;
75      return 0;
76  }
```

在这个例子中，我们演示了如何对一个简单的拟合问题进行迭代优化。通过自己手写的代码，很容易看清楚整个优化的流程。该程序输出每一步迭代的目标函数值和更新量，如下：

终端输出：

```
1  /home/xiang/Code/slambook2/ch6/cmake-build-debug/gaussNewton
2  total cost: 3.19575e+06,        update: 0.0455771  0.078164 -0.985329       estimated params:
    2.04558,-0.921836,4.01467
3  total cost: 376785,         update:  0.065762  0.224972 -0.962521       estimated params:
   2.11134,-0.696864,3.05215
4  total cost: 35673.6,        update: -0.0670241   0.617616  -0.907497        estimated params:
    2.04432,-0.0792484,2.14465
5  total cost: 2195.01,        update: -0.522767   1.19192 -0.756452       estimated params:
```

```
1.52155,1.11267,1.3882
total cost: 174.853,        update: -0.537502  0.909933 -0.386395       estimated params:
0.984045,2.0226,1.00181
total cost: 102.78,         update: -0.0919666   0.147331 -0.0573675        estimated params:
 0.892079,2.16994,0.944438
total cost: 101.937,        update: -0.00117081  0.00196749 -0.00081055     estimated params:
 0.890908,2.1719,0.943628
total cost: 101.937,        update:   3.4312e-06 -4.28555e-06  1.08348e-06      estimated
params: 0.890912,2.1719,0.943629
total cost: 101.937,        update: -2.01204e-08  2.68928e-08 -7.86602e-09      estimated
params: 0.890912,2.1719,0.943629
cost: 101.937>= last cost: 101.937, break.
solve time cost = 0.000212903 seconds.
estimated abc = 0.890912, 2.1719, 0.943629
```

易见整个问题的目标函数在迭代 9 次之后趋近收敛，更新量趋近于零。最终估计的值与真值接近，函数图像如图 6-1 所示。在笔者的计算机上（笔者计算机的 CPU 是 i7-8700），优化用时约 0.2 毫秒。下面我们尝试使用优化库来完成同样的任务。

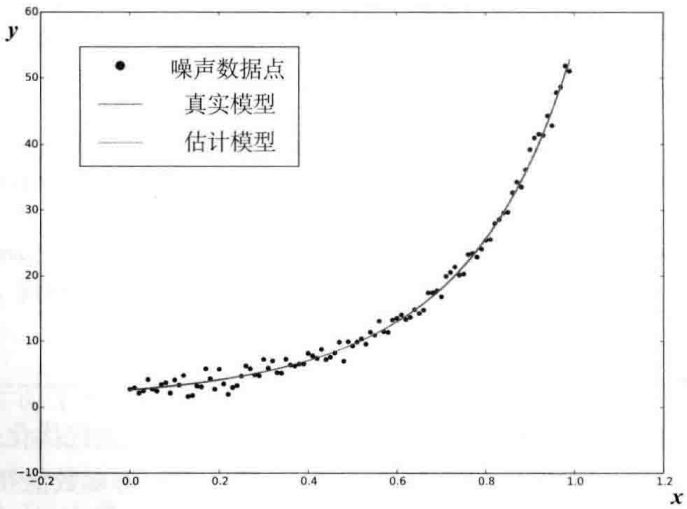


图 6-1 噪声 $\sigma = 1$ 时的曲线拟合结果。真实模型和估计模型非常接近

6.3.2 使用 Ceres 进行曲线拟合

本节向大家介绍两个 C++ 的优化库：来自谷歌的 Ceres 库[35] 及基于图优化的 g2o 库[36]。因为使用 g2o 还需要介绍一点图优化的相关知识，所以我们先来介绍 Ceres，然后介绍一些图优化理论，最后来讲 g2o。因为优化算法在之后的“视觉里程计”和“后端”中都会出现，所以请读者务必掌握优化算法的意义，理解程序的内容。

Ceres 简介

Ceres 是一个广泛使用的最小二乘问题求解库。在 Ceres 中，我们作为用户，只需按照一定步骤定义待解的优化问题，然后交给求解器计算。Ceres 求解的最小二乘问题最一般的形式如下（带边界的核函数最小二乘）：

$$\begin{aligned} &\min_{x} \frac{1}{2} \sum_{i} \rho_i \left(\left\| f_i \left(x_{i_1}, \cdots, x_{i_n} \right) \right\|^2 \right) \\ &\text{s.t. } l_j \leqslant x_j \leqslant u_j. \end{aligned} \tag{6.42}$$

在这个问题中，$x_1, \cdots, x_n$ 为优化变量，又称**参数块**（Parameter blocks），f_i 称为**代价函数**（Cost function），也称为残差块（Residual blocks），在 SLAM 中也可理解为误差项。l_j 和 u_j 为第 j 个优化变量的上限和下限。在最简单的情况下，取 $l_j = -\infty, u_j = \infty$（不限制优化变量的边界）。此时，目标函数由许多平方项经过一个**核函数** $\rho(\cdot)$ 之后求和组成[①]。同样，可以取 ρ 为恒等函数，那么目标函数即为许多项的平方和，我们就得到了无约束的最小二乘问题，和先前介绍的理论是一致的。

为了让 Ceres 帮我们求解这个问题，我们需要做以下几件事：

1. 定义每个参数块。参数块通常为平凡的向量，但是在 SLAM 里也可以定义成四元数、李代数这种特殊的结构。如果是向量，那么我们需要为每个参数块分配一个 double 数组来存储变量的值。
2. 定义残差块的计算方式。残差块通常关联若干个参数块，对它们进行一些自定义的计算，然后返回残差值。Ceres 对它们求平方和之后，作为目标函数的值。
3. 残差块往往也需要定义雅可比的计算方式。在 Ceres 中，你可以使用它提供的“自动求导”功能，也可以手动指定雅可比的计算过程。如果要使用自动求导，那么残差块需要按照特定的写法书写：残差的计算过程应该是一个带模板的括号运算符。这一点我们通过例子来说明。
4. 把所有的参数块和残差块加入 Ceres 定义的 Problem 对象中，调用 Solve 函数求解即可。求解之前，我们可以传入一些配置信息，例如迭代次数、终止条件等，也可以使用默认的配置。

下面，我们来实际操作如何用 Ceres 求解曲线拟合问题，理解优化的过程。

安装 Ceres

为了使用 Ceres，我们需要对它进行编译安装。Ceres 的 GitHub 地址为 `https://github.com/ceres-solver/ceres-solver`，你也可以直接使用本书代码 3rdparty 目录里的 Ceres，这样你将

①核函数的详细讨论见第 9 讲。

和我使用完全一样的版本。

与之前碰到的库一样，Ceres 是一个 cmake 工程。先来安装它的依赖项，在 Ubuntu 系统中可以用 apt-get 安装，主要是谷歌自己使用的一些日志和测试工具：

终端输入：

```
sudo apt-get install liblapack-dev libsuitesparse-dev libcxsparse3 libgflags-dev libgoogle-glog-dev libgtest-dev
```

然后，进入 Ceres 库目录下，使用 cmake 编译并安装它。这个过程我们已经做过很多遍了，此处不再赘述。安装完成后，在 /usr/local/include/ceres 下找到 Ceres 的头文件，并在 /usr/local/lib/ 下找到名为 libceres.a 的库文件。有了这些文件，就可以使用 Ceres 进行优化计算了。

使用 Ceres 拟合曲线

下面的代码演示了如何使用 Ceres 求解同样的问题。

slambook/ch6/ceresCurveFitting.cpp

```
#include <iostream>
#include <opencv2/core/core.hpp>
#include <ceres/ceres.h>
#include <chrono>

using namespace std;

// 代价函数的计算模型
struct CURVE_FITTING_COST {
    CURVE_FITTING_COST(double x, double y) : _x(x), _y(y) {}

    // 残差的计算
    template<typename T>
    bool operator()(
        const T *const abc, // 模型参数，有3维
        T *residual) const {
        // y-exp(ax^2+bx+c)
        residual[0] = T(_y) - ceres::exp(abc[0] * T(_x) * T(_x) + abc[1] * T(_x) + abc[2]);
        return true;
    }

    const double _x, _y;    // x,y数据
};

int main(int argc, char **argv) {
```

```
    double ar = 1.0, br = 2.0, cr = 1.0;          // 真实参数值
    double ae = 2.0, be = -1.0, ce = 5.0;         // 估计参数值
    int N = 100;                                  // 数据点
    double w_sigma = 1.0;                         // 噪声Sigma值
    double inv_sigma = 1.0 / w_sigma;
    cv::RNG rng;                                  // OpenCV随机数产生器

    vector<double> x_data, y_data;      // 数据
    for (int i = 0; i < N; i++) {
        double x = i / 100.0;
        x_data.push_back(x);
        y_data.push_back(exp(ar * x * x + br * x + cr) + rng.gaussian(w_sigma * w_sigma));
    }

    double abc[3] = {ae, be, ce};

    // 构建最小二乘问题
    ceres::Problem problem;
    for (int i = 0; i < N; i++) {
        problem.AddResidualBlock(     // 向问题中添加误差项
            // 使用自动求导，模板参数：误差类型、输出维度、输入维度，维数要与前面struct中一致
            new ceres::AutoDiffCostFunction<CURVE_FITTING_COST, 1, 3>(
                new CURVE_FITTING_COST(x_data[i], y_data[i])
            ),
            nullptr,            // 核函数，这里不使用，为空
            abc                 // 待估计参数
        );
    }

    // 配置求解器
    ceres::Solver::Options options;     // 这里有很多配置项可以填
    options.linear_solver_type = ceres::DENSE_NORMAL_CHOLESKY;  // 增量方程如何求解
    options.minimizer_progress_to_stdout = true;   // 输出到cout

    ceres::Solver::Summary summary;                // 优化信息
    chrono::steady_clock::time_point t1 = chrono::steady_clock::now();
    ceres::Solve(options, &problem, &summary);  // 开始优化
    chrono::steady_clock::time_point t2 = chrono::steady_clock::now();
    chrono::duration<double> time_used = chrono::duration_cast<chrono::duration<double>>(t2 -
     t1);
    cout << "solve time cost = " << time_used.count() << " seconds. " << endl;

    // 输出结果
```

```
    cout << summary.BriefReport() << endl;
    cout << "estimated a,b,c = ";
    for (auto a:abc) cout << a << " ";
    cout << endl;

    return 0;
}
```

程序中需要说明的地方均已加注释。可以看到，我们利用 OpenCV 的噪声生成器生成了 100 个带高斯噪声的数据，随后利用 Ceres 进行拟合。这里演示的 Ceres 用法有如下几项：

1. 定义残差块的类。方法是书写一个类（或结构体），并在类中定义带模板参数的 () 运算符，这样该类就成为了一个**拟函数**（Functor）[①]。这种定义方式使得 Ceres 可以像调用函数一样，对该类的某个对象（比如 a）调用 a<double>() 方法。事实上，Ceres 会把雅可比矩阵作为类型参数传入此函数，从而实现自动求导的功能。
2. 程序中的 double abc[3] 即参数块，而对于残差块，我们对每一个数据构造 CURVE_FITTING_COST 对象，然后调用 AddResidualBlock 将误差项添加到目标函数中。由于优化需要梯度，我们有若干种选择：（1）使用 Ceres 的自动求导（Auto Diff）；（2）使用数值求导（Numeric Diff）[②]；（3）自行推导解析的导数形式，提供给 Ceres。因为自动求导在编码上是最方便的，于是我们使用自动求导。
3. 自动求导需要指定误差项和优化变量的维度。这里的误差是标量，维度为 1；优化的是 a, b, c 三个量，维度为 3。于是，在自动求导类 AutoDiffCostFunction 的模板参数中设定变量维度为 1、3。
4. 设定好问题后，调用 Solve 函数进行求解。你可以在 options 里配置（非常详细的）优化选项。例如，可以选择使用 Line Search 还是 Trust Region、迭代次数、步长，等等。读者可以查看 Options 的定义，看看有哪些优化方法可选，当然默认的配置已经可用于很广泛的问题了。

最后，我们来看看实验结果。调用 build/ceresCurveFitting 查看优化结果：

```
iter      cost      cost_change  |gradient|   |step|    tr_ratio  tr_radius  ls_iter
iter_time  total_time
0  1.597873e+06    0.00e+00    3.52e+06   0.00e+00   0.00e+00  1.00e+04        0    2.10e-05
   7.92e-05
1  1.884440e+05    1.41e+06    4.86e+05   9.88e-01   8.82e-01  1.81e+04        1    5.60e-05
   1.05e-03
2  1.784821e+04    1.71e+05    6.78e+04   9.89e-01   9.06e-01  3.87e+04        1    2.00e-05
   1.09e-03
```

①C++ 术语，带有括号运算符的类在使用括号算符时，就仿佛是一个函数。

②自动求导也是用数值导数实现的，但由于是模板运算，运行更快一些。

```
3  1.099631e+03    1.67e+04    8.58e+03   1.10e+00   9.41e-01  1.16e+05        1    6.70e-05
   1.16e-03
4  8.784938e+01    1.01e+03    6.53e+02   1.51e+00   9.67e-01  3.48e+05        1    1.88e-05
   1.19e-03
5  5.141230e+01    3.64e+01    2.72e+01   1.13e+00   9.90e-01  1.05e+06        1    1.81e-05
   1.22e-03
6  5.096862e+01    4.44e-01    4.27e-01   1.89e-01   9.98e-01  3.14e+06        1    1.79e-05
   1.25e-03
7  5.096851e+01    1.10e-04    9.53e-04   2.84e-03   9.99e-01  9.41e+06        1    1.81e-05
   1.28e-03
solve time cost = 0.00130755 seconds.
Ceres Solver Report: Iterations: 8, Initial cost: 1.597873e+06, Final cost: 5.096851e+01,
Termination: CONVERGENCE
estimated a,b,c = 0.890908 2.1719 0.943628
```

最终的优化值和我们上一节的实验结果基本相同，但运行速度上 Ceres 要相对慢一些。在笔者的计算机上 Ceres 约使用了 1.3 毫秒，这比手写高斯牛顿法慢了约六倍。

希望读者通过这个简单的例子对 Ceres 的使用方法有一个大致了解。它的优点是提供了自动求导工具，使得不必去计算很麻烦的雅可比矩阵。Ceres 的自动求导是通过模板元实现的，在编译时期就可以完成自动求导工作，不过仍然是数值导数。本书大部分时候仍然会介绍雅可比矩阵的计算，因为那样对理解问题更有帮助，而且在优化中更少出现问题。此外，Ceres 的优化过程配置也很丰富，使其适合很广泛的最小二乘优化问题，包括 SLAM 之外的各种问题。

6.3.3　使用 g2o 进行曲线拟合

本讲的第 2 个实践部分将介绍另一个（主要在 SLAM 领域）广为使用的优化库：g2o（General Graphic Optimization，G^2O）。它是一个基于**图优化**的库。图优化是一种将非线性优化与图论结合起来的理论，因此在使用它之前，我们花一点篇幅介绍图优化理论。

图优化理论简介

我们已经介绍了非线性最小二乘的求解方式。它们是由很多个误差项之和组成的。然而，目标函数仅描述了优化变量和许多个误差项，但我们尚不清楚它们之间的**关联**。例如，某个优化变量 x_j 存在于多少个误差项中呢？我们能保证对它的优化是有意义的吗？进一步，我们希望能够直观地看到该优化问题**长什么样**。于是，就牵涉到了图优化。

图优化，是把优化问题表现成**图**的一种方式。这里的**图**是图论意义上的图。一个图由若干个**顶点（Vertex）**，以及连接着这些顶点的**边（Edge）**组成。进而，用**顶点**表示**优化变量**，用**边**表示**误差项**。于是，对任意一个上述形式的非线性最小二乘问题，我们可以构建与之对应的一个**图**。我们可以简单地称它为**图**，也可以用概率图里的定义，称之为**贝叶斯图**或**因子图**。

图 6-2 是一个简单的图优化例子。我们用三角形表示相机位姿节点，用圆形表示路标点，它们构成了图优化的顶点；同时，实线表示相机的运动模型，虚线表示观测模型，它们构成了图优化的边。此时，虽然整个问题的数学形式仍是式 (6.13) 那样，但现在我们可以直观地看到问题的**结构**了。如果希望，也可以做**去掉孤立顶点**或**优先优化边数较多（或按图论的术语，度数较大）的顶点**这样的改进。但是最基本的图优化是用图模型来表达一个非线性最小二乘的优化问题。而我们可以利用图模型的某些性质做更好的优化。

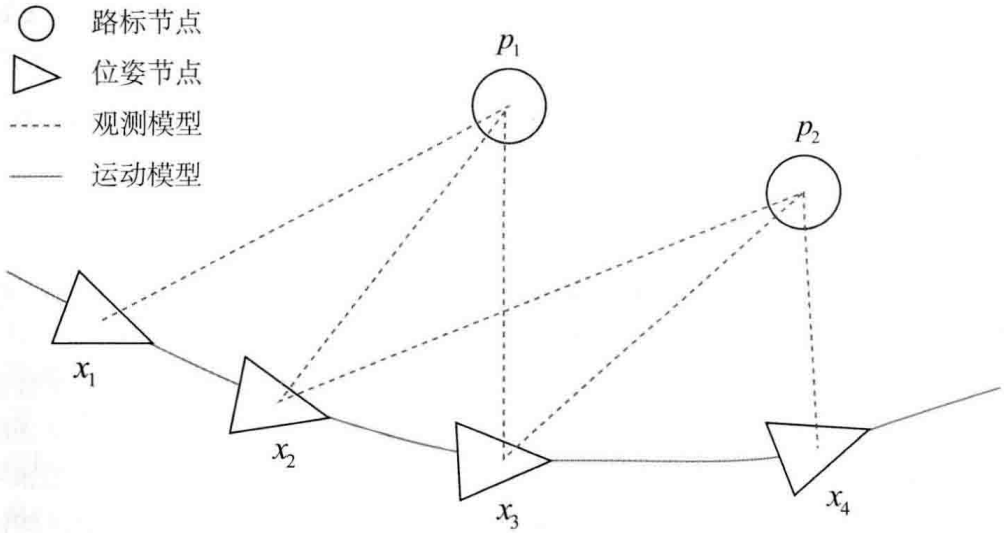


图 6-2 图优化的例子

g2o 是一个通用的图优化库。“通用”意味着你可以在 g2o 里求解任何能够表示为图优化的最小二乘问题，显然包括上面谈的曲线拟合问题。下面我们来演示这个过程。

g2o 的编译与安装

在使用一个库之前，我们需要对它进行编译和安装。读者应该已经体验过很多次这种过程了，它们基本大同小异。关于 g2o，读者可以从 GitHub 下载它：`https://github.com/RainerKuemmerle/g2o`，或从本书提供的第三方代码库中获得。由于 g2o 还在继续更新，所以笔者建议你使用 3rdparty 下的 g2o，以保证版本与笔者的相同。

g2o 也是一个 cmake 工程。我们先来安装它的依赖项（部分依赖项与 Ceres 重合）：

终端输入：

```
sudo apt-get install qt5-qmake qt5-default libqglviewer-dev-qt5 libsuitesparse-dev
libcxsparse3 libcholmod3
```

然后，按照 cmake 的方式对 g2o 进行编译安装即可，这里略去对该过程的说明。安装完成后，

g2o 的头文件将位于/usr/local/g2o 下，库文件位于/usr/local/lib/下。现在，我们重新考虑 Ceres 例程中的曲线拟合实验，在 g2o 中实验一遍。

使用 g2o 拟合曲线

为了使用 g2o，首先要将曲线拟合问题抽象成图优化。这个过程中，只要记住**节点为优化变量，边为误差项**即可。曲线拟合对应的图优化模型可以画成图 6-3 所示的形式。

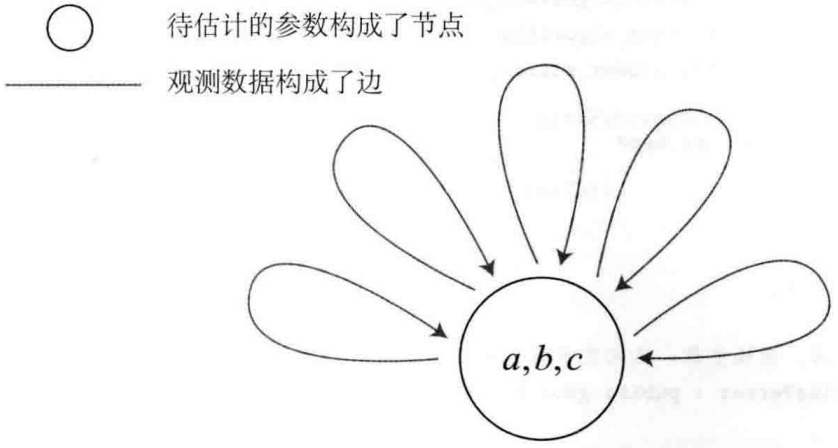


图 6-3　曲线拟合对应的图优化模型（莫名其妙地有些像华为的标志）

在曲线拟合问题中，整个问题只有一个顶点：曲线模型的参数是 a, b, c；而各个带噪声的数据点，构成了一个个误差项，也就是图优化的边。但这里的边与我们平时想的边不太一样，它们是**一元边**（Unary Edge），即**只连接一个顶点**——因为整个图只有一个顶点。所以在图 6-3 中，我们只能把它画成自己连到自己的样子。事实上，图优化中一条边可以连接一个、两个或多个顶点，这主要反映每个误差与多少个优化变量有关。在稍有些玄妙的说法中，我们把它叫作**超边**（Hyper Edge），整个图叫作**超图**（Hyper Graph）[①]。

弄清了这个图模型之后，接下来就是在 g2o 中建立该模型进行优化。作为 g2o 的用户，我们要做的事主要包含以下步骤：

1. 定义顶点和边的类型。
2. 构建图。
3. 选择优化算法。
4. 调用 g2o 进行优化，返回结果。

这部分和 Ceres 是非常相似的，当然程序在书写上会有一些不同。下面演示程序。

①显然笔者并不太喜欢有些故弄玄虚的说法，笔者是个自然主义者。

slambook/ch6/g2oCurveFitting.cpp

```
#include <iostream>
#include <g2o/core/g2o_core_api.h>
#include <g2o/core/base_vertex.h>
#include <g2o/core/base_unary_edge.h>
#include <g2o/core/block_solver.h>
#include <g2o/core/optimization_algorithm_levenberg.h>
#include <g2o/core/optimization_algorithm_gauss_newton.h>
#include <g2o/core/optimization_algorithm_dogleg.h>
#include <g2o/solvers/dense/linear_solver_dense.h>
#include <Eigen/Core>
#include <opencv2/core/core.hpp>
#include <cmath>
#include <chrono>

using namespace std;

// 曲线模型的顶点，模板参数：优化变量维度和数据类型
class CurveFittingVertex : public g2o::BaseVertex<3, Eigen::Vector3d> {
public:
    EIGEN_MAKE_ALIGNED_OPERATOR_NEW

    // 重置
    virtual void setToOriginImpl() override {
        _estimate << 0, 0, 0;
    }

    // 更新
    virtual void oplusImpl(const double *update) override {
        _estimate += Eigen::Vector3d(update);
    }

    // 存盘和读盘：留空
    virtual bool read(istream &in) {}
    virtual bool write(ostream &out) const {}
};

// 误差模型 模板参数：观测值维度，类型，连接顶点类型
class CurveFittingEdge : public g2o::BaseUnaryEdge<1, double, CurveFittingVertex> {
public:
    EIGEN_MAKE_ALIGNED_OPERATOR_NEW

    CurveFittingEdge(double x) : BaseUnaryEdge(), _x(x) {}
```

```
43
44      // 计算曲线模型误差
45      virtual void computeError() override {
46          const CurveFittingVertex *v = static_cast<const CurveFittingVertex *> (_vertices[0]);
47          const Eigen::Vector3d abc = v->estimate();
48          _error(0, 0) = _measurement - std::exp(abc(0, 0) * _x * _x + abc(1, 0) * _x + abc(2,
            0));
49      }
50
51      // 计算雅可比矩阵
52      virtual void linearizeOplus() override {
53          const CurveFittingVertex *v = static_cast<const CurveFittingVertex *> (_vertices[0]);
54          const Eigen::Vector3d abc = v->estimate();
55          double y = exp(abc[0] * _x * _x + abc[1] * _x + abc[2]);
56          _jacobianOplusXi[0] = -_x * _x * y;
57          _jacobianOplusXi[1] = -_x * y;
58          _jacobianOplusXi[2] = -y;
59      }
60
61      virtual bool read(istream &in) {}
62      virtual bool write(ostream &out) const {}
63  public:
64      double _x;  // x值， y值为_measurement
65  };
66
67  int main(int argc, char **argv) {
68      // 省略数据生成部分代码
69      // 构建图优化，先设定g2o
70      typedef g2o::BlockSolver<g2o::BlockSolverTraits<3, 1>> BlockSolverType;  // 每个误差项优
        化变量维度为3，误差值维度为1
71      typedef g2o::LinearSolverDense<BlockSolverType::PoseMatrixType> LinearSolverType; // 线性
        求解器类型
72
73      // 梯度下降方法，可以从GN、LM、DogLeg中选
74      auto solver = new g2o::OptimizationAlgorithmGaussNewton(
75          g2o::make_unique<BlockSolverType>(g2o::make_unique<LinearSolverType>()));
76      g2o::SparseOptimizer optimizer;     // 图模型
77      optimizer.setAlgorithm(solver);   // 设置求解器
78      optimizer.setVerbose(true);       // 打开调试输出
79
80      // 往图中增加顶点
81      CurveFittingVertex *v = new CurveFittingVertex();
82      v->setEstimate(Eigen::Vector3d(ae, be, ce));
```

```
    v->setId(0);
    optimizer.addVertex(v);

    // 往图中增加边
    for (int i = 0; i < N; i++) {
        CurveFittingEdge *edge = new CurveFittingEdge(x_data[i]);
        edge->setId(i);
        edge->setVertex(0, v);                // 设置连接的顶点
        edge->setMeasurement(y_data[i]);      // 观测数值
        edge->setInformation(Eigen::Matrix<double, 1, 1>::Identity() * 1 / (w_sigma * w_sigma
        )); // 信息矩阵：协方差矩阵之逆
        optimizer.addEdge(edge);
    }

    // 执行优化
    cout << "start optimization" << endl;
    chrono::steady_clock::time_point t1 = chrono::steady_clock::now();
    optimizer.initializeOptimization();
    optimizer.optimize(10);
    chrono::steady_clock::time_point t2 = chrono::steady_clock::now();
    chrono::duration<double> time_used = chrono::duration_cast<chrono::duration<double>>(t2 -
     t1);
    cout << "solve time cost = " << time_used.count() << " seconds. " << endl;

    // 输出优化值
    Eigen::Vector3d abc_estimate = v->estimate();
    cout << "estimated model: " << abc_estimate.transpose() << endl;

    return 0;
}
```

在这个程序中，我们从 g2o 派生出了用于曲线拟合的图优化顶点和边：CurveFittingVertex 和 CurveFittingEdge，这实质上扩展了 g2o 的使用方式。这两个类分别派生自 BaseVertex 和 BaseUnaryEdge 类。在派生类中，我们重写了重要的虚函数：

1. 顶点的更新函数：oplusImpl。我们知道优化过程最重要的是增量 $\Delta\boldsymbol{x}$ 的计算，而该函数处理的是 $\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \Delta\boldsymbol{x}$ 的过程。

   读者也许觉得这并不是什么值得一提的事情，因为仅仅是个简单的加法而已，为什么 g2o 不帮我们完成呢？在曲线拟合过程中，由于优化变量（曲线参数）本身位于**向量空间**中，这个更新计算确实就是简单的加法。但是，当优化变量不在向量空间中时，例如 $\boldsymbol{x}$ 是相机位姿，它本身不一定有加法运算。这时，就需要重新定义**增量如何加到现有的估计上**的行为了。按照第 4 讲的解释，我们可能使用左乘更新或右乘更新，而不是直接的加法。

2. 顶点的重置函数：setToOriginImpl。这是平凡的，我们把估计值置零即可。
3. 边的误差计算函数：computeError。该函数需要取出边所连接的顶点的当前估计值，根据曲线模型，与它的观测值进行比较。这和最小二乘问题中的误差模型是一致的。
4. 边的雅可比计算函数：linearizeOplus。这个函数里我们计算了每条边相对于顶点的雅可比。
5. 存盘和读盘函数：read、write。由于我们并不想进行读 / 写操作，所以留空。

定义了顶点和边之后，我们在 main 函数里声明了一个图模型，然后按照生成的噪声数据，往图模型中添加顶点和边，最后调用优化函数进行优化。g2o 会给出优化的结果：

终端输出：

```
start optimization
iteration= 0     chi2= 376785.128234     time= 3.3299e-05     cumTime= 3.3299e-05     edges= 100  schur= 0
iteration= 1     chi2= 35673.566018  time= 1.3789e-05     cumTime= 4.7088e-05     edges= 100  schur= 0
iteration= 2     chi2= 2195.012304   time= 1.2323e-05     cumTime= 5.9411e-05     edges= 100  schur= 0
iteration= 3     chi2= 174.853126    time= 1.3302e-05     cumTime= 7.2713e-05     edges= 100  schur= 0
iteration= 4     chi2= 102.779695    time= 1.2424e-05     cumTime= 8.5137e-05     edges= 100  schur= 0
iteration= 5     chi2= 101.937194    time= 1.2523e-05     cumTime= 9.766e-05  edges= 100  schur= 0
iteration= 6     chi2= 101.937020    time= 1.2268e-05     cumTime= 0.000109928    edges= 100  schur= 0
iteration= 7     chi2= 101.937020    time= 1.2612e-05     cumTime= 0.00012254     edges= 100  schur= 0
iteration= 8     chi2= 101.937020    time= 1.2159e-05     cumTime= 0.000134699    edges= 100  schur= 0
iteration= 9     chi2= 101.937020    time= 1.2688e-05     cumTime= 0.000147387    edges= 100  schur= 0
solve time cost = 0.000919301 seconds.
estimated model: 0.890912   2.1719 0.943629
```

我们使用高斯—牛顿方法进行梯度下降，在迭代了 9 次后得到优化结果，与 Ceres 和手写高斯牛顿法相差无几。从运行速度来看，我们的实验结论是手写快于 g2o，而 g2o 快于 Ceres。这是一个大体符合直觉的经验，通用性和高效性往往是互相矛盾的。但是本实验中 Ceres 使用了自动求导，且求解器配置与高斯牛顿还不完全一致，所以看起来慢一些。

6.4 小结

本节介绍了 SLAM 中经常碰到的一种非线性优化问题：由许多个误差项平方和组成的最小二乘问题。我们介绍了它的定义和求解，并且讨论了两种主要的梯度下降方式：高斯牛顿法和列文伯格—马夸尔特方法。在实践部分中，分别使用了手写高斯牛顿法、Ceres 和 g2o 两种优化库求解同一个曲线拟合问题，发现它们给出了相似的结果。

由于还没有详细谈 Bundle Adjustment，我们在实践部分选择了曲线拟合这样一个简单但有代表性的例子，以演示一般的非线性最小二乘求解方式。特别地，如果用 g2o 来拟合曲线，必须先把问题转换为图优化，定义新的顶点和边，这种做法是有一些迂回的——g2o 的主要目的并不在此。相比之下，Ceres 定义误差项求曲线拟合问题则自然了很多，因为它本身即是一个优化库。然而，在 SLAM 中更多的问题是，一个带有许多个相机位姿和许多个空间点的优化问题如何求解。特别地，当相机位姿以李代数表示时，误差项关于相机位姿的导数如何计算，将是一件值得详细讨论的事。我们将在后续内容中发现，g2o 提供了大量现成的顶点和边，非常便于相机位姿估计问题。而在 Ceres 中，我们不得不自己实现每一个 Cost Function，有一些不便。

在实践部分的两个程序中，我们没有去计算曲线模型关于三个参数的导数，而是利用了优化库的数值求导，这使得理论和代码都更简洁。Ceres 库提供了基于模板元的自动求导和运行时的数值求导，而 g2o 只提供了运行时数值求导这一种方式。但是，对于大多数问题，如果能够推导出雅可比矩阵的解析形式并告诉优化库，就可以避免数值求导中的诸多问题。

最后，希望读者能够适应 Ceres 和 g2o 这些大量使用模板编程的方式。也许一开始会看上去比较吓人（特别是 Ceres 设置残差块的括号运算符，以及 g2o 初始化部分的代码），但是熟悉之后，就会觉得这样的方式是自然的，而且容易扩展。我们将在 SLAM 后端一讲中继续讨论稀疏性、核函数、位姿图等问题。

习题

1. 证明线性方程 $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ 当系数矩阵 $\boldsymbol{A}$ 超定时，最小二乘解为 $\boldsymbol{x} = (\boldsymbol{A}^{\mathrm{T}}\boldsymbol{A})^{-1}\boldsymbol{A}^{\mathrm{T}}\boldsymbol{b}$。
2. 调研最速下降法、牛顿法、高斯牛顿法和列文伯格—马夸尔特方法各有什么优缺点？除了我们举的 Ceres 库和 g2o 库，还有哪些常用的优化库？（你可能会找到一些 MATLAB 上的库。）
3. 为什么高斯牛顿法的增量方程系数矩阵可能不正定？不正定有什么几何含义？为什么在这种情况下解就不稳定了？
4. DogLeg 是什么？它与高斯牛顿法和列文伯格—马夸尔特方法有何异同？请搜索相关的材料[①]。

①例如，http://www.numerical.rl.ac.uk/people/nimg/course/lectures/raphael/lectures/lec7slides.pdf。

5. 阅读 Ceres 的教学材料（`http://ceres-solver.org/tutorial.html`）以更好地掌握其用法。
6. 阅读 g2o 自带的文档，你能看懂它吗？如果还不能完全看懂，请在阅读第 10 讲和第 11 讲之后再回来看。
7.* 请更改曲线拟合实验中的曲线模型，并用 Ceres 和 g2o 进行优化实验。例如，可以使用更多的参数和更复杂的模型。

第2部分

实践应用

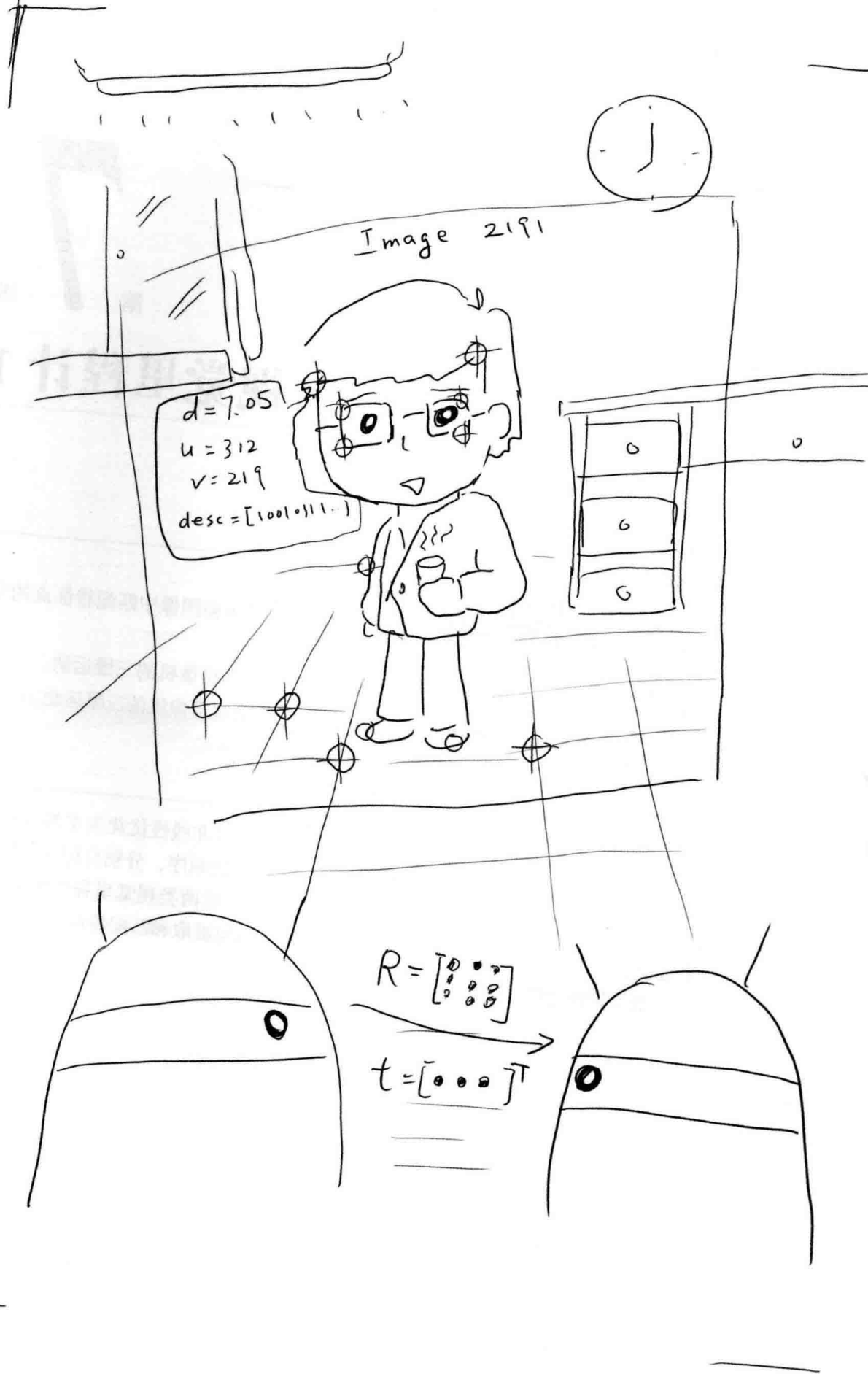
第 7 讲

视觉里程计 1

主要目标

1. 理解图像特征点的意义，并掌握在单幅图像中提取特征点及多幅图像中匹配特征点的方法。
2. 理解对极几何的原理，利用对极几何的约束，恢复图像之间的摄像机的三维运动。
3. 理解 PNP 问题，以及利用已知三维结构与图像的对应关系求解摄像机的三维运动。
4. 理解 ICP 问题，以及利用点云的匹配关系求解摄像机的三维运动。
5. 理解如何通过三角化获得二维图像上对应点的三维结构。

本书前面介绍了运动方程和观测方程的具体形式，并讲解了以非线性优化为主的求解方法。从本讲开始，我们结束基础知识的铺垫而步入正题：按照第 2 讲的顺序，分别介绍视觉里程计、后端优化、回环检测和地图构建 4 个模块。本讲和下一讲主要介绍两类视觉里程计里常用的方法：特征点法和光流法。本讲中，我们将介绍什么是特征点、如何提取和匹配特征点，以及如何根据配对的特征点估计相机运动。


Image 219
d=7.05
u=312
v=219
desc=[1001011...]
R=[...]
t=[...]^T

7.1　特征点法

在第 2 讲中，我们介绍过，一个 SLAM 系统分为前端和后端，其中前端也称为视觉里程计。视觉里程计根据相邻图像的信息估计出粗略的相机运动，给后端提供较好的初始值。视觉里程计的算法主要分为两个大类：**特征点法**和**直接法**。基于特征点法的前端，长久以来（直到现在）被认为是视觉里程计的主流方法。它具有稳定，对光照、动态物体不敏感的优势，是目前比较成熟的解决方案。在本讲中，我们将从特征点法入手，学习如何提取、匹配图像特征点，然后估计两帧之间的相机运动和场景结构，从而实现一个两帧间视觉里程计。这类算法有时也称为两视图几何（Two-view geometry）。

7.1.1　特征点

视觉里程计的核心问题是**如何根据图像估计相机运动**。然而，图像本身是一个由亮度和色彩组成的矩阵，如果直接从矩阵层面考虑运动估计，将会非常困难。所以，比较方便的做法是：首先，从图像中选取比较**有代表性**的**点**。这些点在相机视角发生少量变化后会保持不变，于是我们能在各个图像中找到相同的点。然后，在这些点的基础上，讨论相机位姿估计问题，以及这些点的定位问题。在经典 SLAM 模型中，我们称这些点为**路标**。而在视觉 SLAM 中，路标则是指图像特征（Feature）。

根据维基百科的定义，图像特征是一组与计算任务相关的信息，计算任务取决于具体的应用[37]。简而言之，**特征是图像信息的另一种数字表达形式**。一组好的特征对在指定任务上的最终表现至关重要，所以多年来研究者们花费了大量的精力对特征进行研究。数字图像在计算机中以灰度值矩阵的方式存储，所以最简单的，单个图像像素也是一种“特征”。但是，在视觉里程计中，我们希望**特征点在相机运动之后保持稳定**，而灰度值受光照、形变、物体材质的影响严重，在不同图像间变化非常大，不够稳定。理想的情况是，当场景和相机视角发生少量改变时，算法还能从图像中判断哪些地方是同一个点。所以，仅凭灰度值是不够的，我们需要对图像提取特征点。

特征点是图像里一些**特别的地方**。以图 7-1 为例，我们可以把图像中的角点、边缘和区块都当成图像中有代表性的地方。我们更容易精确地指出，某两幅图像中出现了同一个角点；指出某两幅图像中出现同一个边缘则稍微困难一些，因为沿着该边缘前进，图像局部是相似的；指出某两幅图像中出现同一个区块则是最困难的。我们发现，图像中的角点、边缘相比于像素区块而言更加“特别”，在不同图像之间的辨识度更强。所以，一种直观的提取特征的方式就是在不同图像间辨认角点，确定它们的对应关系。在这种做法中，角点就是所谓的特征。角点的提取算法有很多，例如 Harris 角点[38]、FAST 角点[39]、GFTT 角点[40]，等等。它们大部分是 2000 年以前提出的算法。

然而，在大多数应用中，单纯的角点依然不能满足我们的很多需求。例如，从远处看上去是角点的地方，当相机离近之后，可能就不显示为角点了。或者，当旋转相机时，角点的外观会发

生变化，我们也就不容易辨认出那是同一个角点了。为此，计算机视觉领域的研究者们在长年的研究中设计了许多更加稳定的局部图像特征，如著名的 SIFT[41]、SURF[42]、ORB[43]，等等。相比于朴素的角点，这些人工设计的特征点能够拥有如下性质：

1. 可重复性（Repeatability）：相同的特征可以在不同的图像中找到。
2. 可区别性（Distinctiveness）：不同的特征有不同的表达。
3. 高效率（Efficiency）：同一图像中，特征点的数量应远小于像素的数量。
4. 本地性（Locality）：特征仅与一小片图像区域相关。

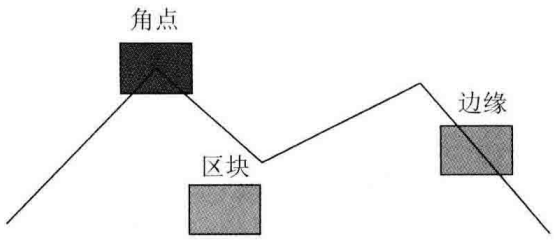


图 7-1 可以作为图像特征的部分：角点、边缘、区块

特征点由**关键点**（Key-point）和**描述子**（Descriptor）两部分组成。例如，当我们说“在一张图像中计算 SIFT 特征点”时，是指“提取 SIFT 关键点，并计算 SIFT 描述子”两件事情。关键点是指该特征点在图像里的位置，有些特征点还具有朝向、大小等信息。描述子通常是一个向量，按照某种人为设计的方式，描述了该关键点周围像素的信息。描述子是按照“**外观相似的特征应该有相似的描述子**”的原则设计的。因此，只要两个特征点的描述子在向量空间上的距离相近，就可以认为它们是同样的特征点。

历史上，研究者们提出过许多图像特征。它们有些很精确，在相机的运动和光照变化下仍具有相似的表达，但相应地计算量较大。其中，SIFT（尺度不变特征变换，Scale-Invariant Feature Transform）当属最为经典的一种。它充分考虑了在图像变换过程中出现的光照、尺度、旋转等变化，但随之而来的是极大的计算量。由于整个 SLAM 过程中图像特征的提取与匹配仅仅是诸多环节中的一个，截至 2016 年，普通计算机的 CPU 还无法实时地计算 SIFT 特征，进行定位与建图，所以在 SLAM 中我们很少使用这种“奢侈”的图像特征。

另一些特征，则考虑适当降低精度和鲁棒性，以提升计算的速度。例如，FAST 关键点属于计算特别快的一种特征点（注意，这里“关键点”的表述，说明它没有描述子），而 ORB（Oriented FAST and Rotated BRIEF）特征则是目前看来非常具有代表性的实时图像特征。它改进了 FAST 检测子[39] 不具有方向性的问题，并采用速度极快的二进制描述子 BRIEF（Binary Robust Independent Elementary Feature）[44]，使整个图像特征提取的环节大大加速。根据作者在论文中所述测试，在同一幅图像中同时提取约 1000 个特征点的情况下，ORB 约花费 15.3 毫秒，SURF 约花费 217.3 毫秒，SIFT 约花费 5228.7 毫秒。由此可以看出，ORB 在保持了特征子具有旋转、尺度不变性的同时，在速度方面提升明显，对于实时性要求很高的 SLAM 来说是一个很好的选择。

大部分特征提取都具有较好的并行性，可以通过 GPU 等设备加速计算。经过 GPU 加速后的 SIFT，就可以满足实时计算的要求。但是，引入 GPU 将带来整个 SLAM 成本的提升。由此带来的性能提升是否足以抵去付出的计算成本，需要系统的设计人员仔细考量。

显然，计算机视觉领域存在大量的特征点种类，我们不可能在书中一一介绍。在目前的 SLAM 方案中，ORB 是质量与性能之间较好的折中，因此，我们以 ORB 为代表介绍提取特征的整个过程。如果读者对特征提取和匹配算法感兴趣，那么我们建议你阅读这方面的相关书籍 [45]。

7.1.2　ORB 特征

ORB 特征由**关键点**和**描述子**两部分组成。它的关键点称为“Oriented FAST”，是一种改进的 FAST 角点，关于什么是 FAST 角点我们将在下文介绍。它的描述子称为 BRIEF。因此，提取 ORB 特征分为如下两个步骤：

1. FAST 角点提取：找出图像中的“角点”。相较于原版的 FAST，ORB 中计算了特征点的主方向，为后续的 BRIEF 描述子增加了旋转不变特性。
2. BRIEF 描述子：对前一步提取出特征点的周围图像区域进行描述。ORB 对 BRIEF 进行了一些改进，主要是指在 BRIEF 中使用了先前计算的方向信息。

下面分别介绍 FAST 和 BRIEF。

FAST 关键点

FAST 是一种角点，主要检测局部像素灰度变化明显的地方，以速度快著称。它的思想是：如果一个像素与邻域的像素差别较大（过亮或过暗），那么它更可能是角点。相比于其他角点检测算法，FAST 只需比较像素亮度的大小，十分快捷。它的检测过程如下（如图 7-2 所示）：

1. 在图像中选取像素 p，假设它的亮度为 I_p。
2. 设置一个阈值 T（比如，I_p 的 20%）。
3. 以像素 p 为中心，选取半径为 3 的圆上的 16 个像素点。
4. 假如选取的圆上有连续的 N 个点的亮度大于 $I_p + T$ 或小于 $I_p - T$，那么像素 p 可以被认为是特征点（N 通常取 12，即 FAST-12。其他常用的 N 取值为 9 和 11，它们分别被称为 FAST-9 和 FAST-11）。
5. 循环以上四步，对每一个像素执行相同的操作。

在 FAST-12 算法中，为了更高效，可以添加一项预测试操作，以快速地排除绝大多数不是角点的像素。具体操作为，对于每个像素，直接检测邻域圆上的第 1, 5, 9, 13 个像素的亮度。只有当这 4 个像素中有 3 个同时大于 $I_p + T$ 或小于 $I_p - T$ 时，当前像素才有可能是一个角点，否则应该直接排除。这样的预测试操作大大加速了角点检测。此外，原始的 FAST 角点经常出现“扎堆”的现象。所以在第一遍检测之后，还需要用非极大值抑制（Non-maximal suppression），在一定区域内仅保留响应极大值的角点，避免角点集中的问题。

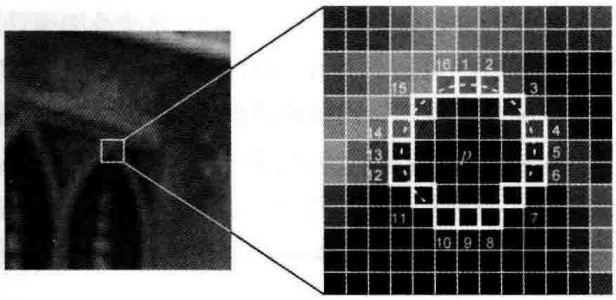


图 7-2 FAST 特征点[39]

FAST 特征点的计算仅仅是比较像素间亮度的差异，所以速度非常快，但它也有重复性不强、分布不均匀的缺点。此外，FAST 角点不具有方向信息。同时，由于它固定取半径为 3 的圆，存在尺度问题：远处看着像是角点的地方，接近后看可能就不是角点了。针对 FAST 角点不具有方向性和尺度的弱点，ORB 添加了尺度和旋转的描述。尺度不变性由构建图像金字塔①，并在金字塔的每一层上检测角点来实现。而特征的旋转是由灰度质心法（Intensity Centroid）实现的。

金字塔是计算图视觉中常用的一种处理方法，示意图如图 7-3 所示。金字塔底层是原始图像。每往上一层，就对图像进行一个固定倍率的缩放，这样我们就有了不同分辨率的图像。较小的图像可以看成是远处看过来的场景。在特征匹配算法中，我们可以匹配不同层上的图像，从而实现尺度不变性。例如，如果相机在后退，那么我们应该能够在上一个图像金字塔的上层和下一个图像金字塔的下层中找到匹配。

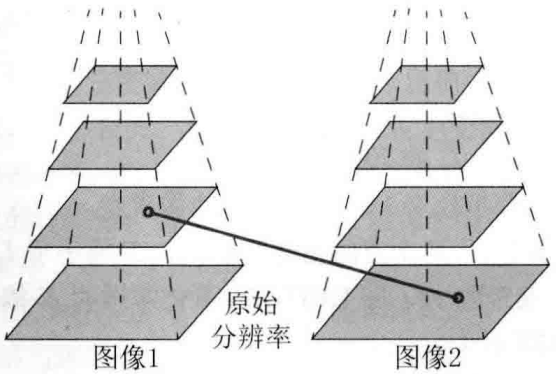


图 7-3 使用金字塔可以匹配不同缩放倍率下的图像

在旋转方面，我们计算特征点附近的图像灰度质心。所谓质心是指以图像块灰度值作为权重的中心。其具体操作步骤如下[46]：

1. 在一个小的图像块 B 中，定义图像块的矩为

①金字塔是指对图像进行不同层次的降采样，以获得不同分辨率的图像。

$$m_{pq} = \sum_{x,y \in B} x^p y^q I(x,y), \quad p,q = \{0,1\}.$$

2. 通过矩可以找到图像块的质心：

$$C = \left(\frac{m_{10}}{m_{00}}, \frac{m_{01}}{m_{00}} \right).$$

3. 连接图像块的几何中心 O 与质心 C，得到一个方向向量 $\overrightarrow{OC}$，于是特征点的方向可以定义为

$$\theta = \arctan(m_{01}/m_{10}).$$

通过以上方法，FAST 角点便具有了尺度与旋转的描述，从而大大提升了其在不同图像之间表述的鲁棒性。所以在 ORB 中，把这种改进后的 FAST 称为 Oriented FAST。

BRIEF 描述子

在提取 Oriented FAST 关键点后，我们对每个点计算其描述子。ORB 使用改进的 BRIEF 特征描述。我们先来介绍 BRIEF。

BRIEF 是一种**二进制**描述子，其描述向量由许多个 0 和 1 组成，这里的 0 和 1 编码了关键点附近两个随机像素（比如 p 和 q）的大小关系：如果 p 比 q 大，则取 1，反之就取 0。如果我们取了 128 个这样的 p,q，则最后得到 128 维由 0、1 组成的向量[44]。BRIEF 使用了随机选点的比较，速度非常快，而且由于使用了二进制表达，存储起来也十分方便，适用于实时的图像匹配。原始的 BRIEF 描述子不具有旋转不变性，因此在图像发生旋转时容易丢失。而 ORB 在 FAST 特征点提取阶段计算了关键点的方向，所以可以利用方向信息，计算旋转之后的“Steer BRIEF”特征使 ORB 的描述子具有较好的旋转不变性。

由于考虑到了旋转和缩放，ORB 在平移、旋转和缩放的变换下仍有良好的表现。同时，FAST 和 BREIF 的组合也非常高效，使得 ORB 特征在实时 SLAM 中非常受欢迎。我们在图 7-4 中展示了一张用 OpenCV 提取 ORB 特征点的结果，下面介绍如何在不同的图像之间进行特征匹配。



图 7-4　OpenCV 提供的 ORB 特征点检测结果

7.1.3 特征匹配

特征匹配（如图 7-5 所示）是视觉 SLAM 中极为关键的一步，宽泛地说，特征匹配解决了 SLAM 中的数据关联问题（data association），即确定当前看到的路标与之前看到的路标之间的对应关系。通过对图像与图像或者图像与地图之间的描述子进行准确匹配，我们可以为后续的姿态估计、优化等操作减轻大量负担。然而，由于图像特征的局部特性，误匹配的情况广泛存在，而且长期以来一直没有得到有效解决，目前已经成为视觉 SLAM 中制约性能提升的一大瓶颈。部分原因是场景中经常存在大量的重复纹理，使得特征描述非常相似。在这种情况下，仅利用局部特征解决误匹配是非常困难的。



图 7-5 两帧图像间的特征匹配

让我们先来看正确匹配的情况，等做完实验再讨论误匹配问题。考虑两个时刻的图像。如果在图像 I_t 中提取到特征点 $x_t^m, m = 1, 2, \ldots, M$，在图像 I_{t+1} 中提取到特征点 $x_{t+1}^n, n = 1, 2, \ldots, N$，如何寻找这两个集合元素的对应关系呢？最简单的特征匹配方法就是**暴力匹配（Brute-Force Matcher）**。即对每一个特征点 x_t^m 与所有的 x_{t+1}^n 测量描述子的距离，然后排序，取最近的一个作为匹配点。描述子距离表示了两个特征之间的**相似程度**，不过在实际运用中还可以取不同的距离度量范数。对于浮点类型的描述子，使用欧氏距离进行度量即可。而对于二进制的描述子（比如 BRIEF 这样的），我们往往使用汉明距离（Hamming distance）作为度量——两个二进制串之间的汉明距离，指的是其**不同位数的个数**。

然而，当特征点数量很大时，暴力匹配法的运算量将变得很大，特别是当想要匹配某个帧和一张地图的时候。这不符合我们在 SLAM 中的实时性需求。此时，**快速近似最近邻（FLANN）**算法更加适合于匹配点数量极多的情况。由于这些匹配算法理论已经成熟，而且实现上也已集成到 OpenCV，所以这里就不再描述它的技术细节了。感兴趣的读者可以参考阅读文献 [47]。

7.2　实践：特征提取和匹配

OpenCV 已经集成了多数主流的图像特征，我们可以很方便地进行调用。下面我们来完成两个实验：第一个实验中，我们演示使用 OpenCV 进行 ORB 的特征匹配；第二个实验中，我们演示如何根据前面介绍的原理，手写一个简单的 ORB 特征。通过手写的过程，读者可以更加清楚地理解 ORB 的计算过程，并类推到其他特征上去。

7.2.1　OpenCV 的 ORB 特征

首先，我们调用 OpenCV 来提取和匹配 ORB。笔者为此实验准备了两张图像，分别是位于 slambook2/ch7/下的 1.png 和 2.png，如图 7-6 所示。它们是来自公开数据集[23] 中的两张图像，我们看到相机发生了微小的运动。本节程序演示如何提取 ORB 特征并进行匹配。在 7.2.2 节中，我们将演示如何用匹配结果估计相机运动。



图 7-6　实验使用的两张图像

下面程序演示了 ORB 的使用方法：

slambook2/ch7/orb_cv.cpp

```
#include <iostream>
#include <opencv2/core/core.hpp>
#include <opencv2/features2d/features2d.hpp>
#include <opencv2/highgui/highgui.hpp>
#include <chrono>

using namespace std;
using namespace cv;

int main(int argc, char **argv) {
    if (argc != 3) {
        cout << "usage: feature_extraction img1 img2" << endl;
```

```
    return 1;
}
//-- 读取图像
Mat img_1 = imread(argv[1], CV_LOAD_IMAGE_COLOR);
Mat img_2 = imread(argv[2], CV_LOAD_IMAGE_COLOR);
assert(img_1.data != nullptr && img_2.data != nullptr);

//-- 初始化
std::vector<KeyPoint> keypoints_1, keypoints_2;
Mat descriptors_1, descriptors_2;
Ptr<FeatureDetector> detector = ORB::create();
Ptr<DescriptorExtractor> descriptor = ORB::create();
Ptr<DescriptorMatcher> matcher = DescriptorMatcher::create("BruteForce-Hamming");

//-- 第一步:检测Oriented FAST角点位置
chrono::steady_clock::time_point t1 = chrono::steady_clock::now();
detector->detect(img_1, keypoints_1);
detector->detect(img_2, keypoints_2);

//-- 第二步:根据角点位置计算BRIEF描述子
descriptor->compute(img_1, keypoints_1, descriptors_1);
descriptor->compute(img_2, keypoints_2, descriptors_2);
chrono::steady_clock::time_point t2 = chrono::steady_clock::now();
chrono::duration<double> time_used = chrono::duration_cast<chrono::duration<double>>(t2 -
 t1);
cout << "extract ORB cost = " << time_used.count() << " seconds. " << endl;

Mat outimg1;
drawKeypoints(img_1, keypoints_1, outimg1, Scalar::all(-1), DrawMatchesFlags::DEFAULT);
imshow("ORB features", outimg1);

//-- 第三步:对两幅图像中的BRIEF描述子进行匹配，使用Hamming距离
vector<DMatch> matches;
t1 = chrono::steady_clock::now();
matcher->match(descriptors_1, descriptors_2, matches);
t2 = chrono::steady_clock::now();
time_used = chrono::duration_cast<chrono::duration<double>>(t2 - t1);
cout << "match ORB cost = " << time_used.count() << " seconds. " << endl;

//-- 第四步:匹配点对筛选
// 计算最小距离和最大距离
auto min_max = minmax_element(matches.begin(), matches.end(),
    [](const DMatch &m1, const DMatch &m2) { return m1.distance < m2.distance; });
```

```
    double min_dist = min_max.first->distance;
    double max_dist = min_max.second->distance;

    printf("-- Max dist : %f \n", max_dist);
    printf("-- Min dist : %f \n", min_dist);

    //当描述子之间的距离大于两倍的最小距离时，即认为匹配有误。但有时最小距离会非常小，所以要设置一个经验值30作为下限
    std::vector<DMatch> good_matches;
    for (int i = 0; i < descriptors_1.rows; i++) {
        if (matches[i].distance <= max(2 * min_dist, 30.0)) {
            good_matches.push_back(matches[i]);
        }
    }

    //-- 第五步:绘制匹配结果
    Mat img_match;
    Mat img_goodmatch;
    drawMatches(img_1, keypoints_1, img_2, keypoints_2, matches, img_match);
    drawMatches(img_1, keypoints_1, img_2, keypoints_2, good_matches, img_goodmatch);
    imshow("all matches", img_match);
    imshow("good matches", img_goodmatch);
    waitKey(0);

    return 0;
}
```

运行此程序（需要输入两个图像位置），将输出运行结果：

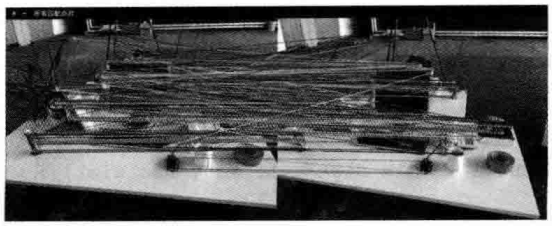
终端输入：

```
% build/orb_cv 1.png 2.png
extract ORB cost = 0.0229183 seconds.
match ORB cost = 0.000751868 seconds.
-- Max dist : 95.000000
-- Min dist : 4.000000
```

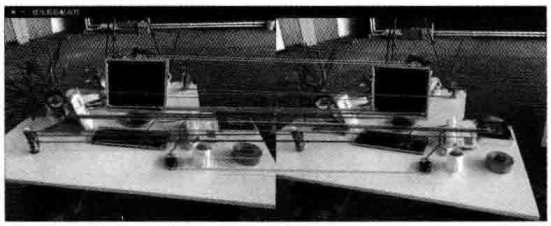
图 7-7 显示了例程的运行结果。我们看到未筛选的匹配中带有大量的误匹配。经过一次筛选之后，匹配数量减少了许多，但大多数匹配都是正确的。这里，筛选的依据是**汉明距离小于最小距离的两倍**，这是一种工程上的经验方法，不一定有理论依据。不过，尽管在示例图像中能够筛选出正确的匹配，但我们仍然不能保证在所有其他图像中得到的匹配都是正确的。因此，在后面的运动估计中，还需要使用去除误匹配的算法。在笔者的机器上，ORB 提取花费了 22.9 毫秒（两张图像），匹配花费了 0.75 毫秒，可见大部分计算量花在了特征提取上。



ORB关键点



未筛选的匹配



筛选后的匹配

图 7-7 特征提取与匹配结果

7.2.2 手写 ORB 特征

下面我们演示手写 ORB 特征的方法。这部分代码比较多，书上只展示核心部分的代码，其余的代码请读者从代码库中获取。

slambook2/ch7/orb_self.cpp（片段）

```
typedef vector<uint32_t> DescType;
// ... 省略图片读取部分代码和测试代码
// compute the descriptor
void ComputeORB(const cv::Mat &img, vector<cv::KeyPoint> &keypoints, vector<DescType> &
descriptors) {
    const int half_patch_size = 8;
    const int half_boundary = 16;
    int bad_points = 0;
    for (auto &kp: keypoints) {
        if (kp.pt.x < half_boundary || kp.pt.y < half_boundary ||
        kp.pt.x >= img.cols - half_boundary || kp.pt.y >= img.rows - half_boundary) {
            // outside
            bad_points++;
            descriptors.push_back({});
            continue;
        }

        float m01 = 0, m10 = 0;
```

```
        for (int dx = -half_patch_size; dx < half_patch_size; ++dx) {
            for (int dy = -half_patch_size; dy < half_patch_size; ++dy) {
                uchar pixel = img.at<uchar>(kp.pt.y + dy, kp.pt.x + dx);
                m01 += dx * pixel;
                m10 += dy * pixel;
            }
        }

        // angle should be arc tan(m01/m10);
        float m_sqrt = sqrt(m01 * m01 + m10 * m10);
        float sin_theta = m01 / m_sqrt;
        float cos_theta = m10 / m_sqrt;

        // compute the angle of this point
        DescType desc(8, 0);
        for (int i = 0; i < 8; i++) {
            uint32_t d = 0;
            for (int k = 0; k < 32; k++) {
                int idx_pq = i * 8 + k;
                cv::Point2f p(ORB_pattern[idx_pq * 4], ORB_pattern[idx_pq * 4 + 1]);
                cv::Point2f q(ORB_pattern[idx_pq * 4 + 2], ORB_pattern[idx_pq * 4 + 3]);

                // rotate with theta
                cv::Point2f pp = cv::Point2f(cos_theta * p.x - sin_theta * p.y, sin_theta * p
                .x + cos_theta * p.y) + kp.pt;
                cv::Point2f qq = cv::Point2f(cos_theta * q.x - sin_theta * q.y, sin_theta * q
                .x + cos_theta * q.y) + kp.pt;
                if (img.at<uchar>(pp.y, pp.x) < img.at<uchar>(qq.y, qq.x)) {
                    d |= 1 << k;
                }
            }
            desc[i] = d;
        }
        descriptors.push_back(desc);
    }

    cout << "bad/total: " << bad_points << "/" << keypoints.size() << endl;
}

// brute-force matching
void BfMatch(
    const vector<DescType> &desc1, const vector<DescType> &desc2, vector<cv::DMatch> &matches
    ) {
```

```
    const int d_max = 40;

    for (size_t i1 = 0; i1 < desc1.size(); ++i1) {
        if (desc1[i1].empty()) continue;
        cv::DMatch m{i1, 0, 256};
        for (size_t i2 = 0; i2 < desc2.size(); ++i2) {
            if (desc2[i2].empty()) continue;
            int distance = 0;
            for (int k = 0; k < 8; k++) {
                distance += _mm_popcnt_u32(desc1[i1][k] ^desc2[i2][k]);
            }
            if (distance < d_max && distance < m.distance) {
                m.distance = distance;
                m.trainIdx = i2;
            }
        }
        if (m.distance < d_max) {
            matches.push_back(m);
        }
    }
}
```

这个演示中我们只展示 ORB 的计算代码和匹配代码。在计算中，我们用 256 位的二进制描述，即对应到 8 个 32 位的 unsigned int 数据，用 typedef 将它表示成 DescType。然后，我们根据前面介绍的原理计算 FAST 特征点的角度，再使用该角度计算描述子。此代码中通过三角函数的原理回避了复杂的 arctan 及 sin、cos 计算，从而达到加速的效果。在 BfMatch 函数中，我们还使用了 SSE 指令集中的_mm_popcnt_u32 函数计算一个 unsigned int 变量中 1 的个数，从而达到计算汉明距离的效果。该段程序的运行结果如下，匹配结果如图 7-8 所示。

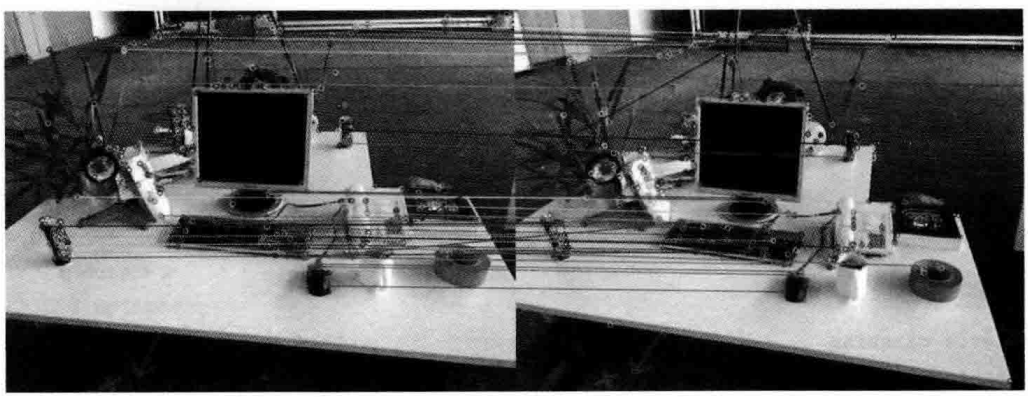


图 7-8 匹配结果

终端输出：

```
bad/total: 43/638
bad/total: 8/595
extract ORB cost = 0.00390721 seconds.
match ORB cost = 0.000862984 seconds.
matches: 51
```

可见，这个程序中，ORB 的提取只需要 3.9 毫秒，匹配只需 0.86 毫秒。我们通过一些简单的算法修改，使对 ORB 的提取加速了 5.8 倍。请读者注意，编译这个程序需要你的 CPU 支持 SSE 指令集（绝大多数现代的家用 CPU 上都已经支持）。如果我们能够对提取特征部分进一步并行化处理，则算法还可以有加速的空间。

7.2.3 计算相机运动

我们已经有了匹配好的点对，接下来，我们要根据点对估计相机的运动。这里由于相机的原理不同，情况发生了变化：

1. 当相机为单目时，我们只知道 2D 的像素坐标，因而问题是根据**两组 2D 点**估计运动。该问题用**对极几何**解决。
2. 当相机为双目、RGB-D 时，或者通过某种方法得到了距离信息，那么问题就是根据**两组 3D 点**估计运动。该问题通常用 ICP 解决。
3. 如果一组为 3D，一组为 2D，即，我们得到了一些 3D 点和它们在相机的投影位置，也能估计相机的运动。该问题通过 **PnP** 求解。

因此，下面几节介绍这三种情形下的相机运动估计。我们将从信息最少的 2D-2D 情形出发，看看它如何求解，求解过程中又遇到哪些麻烦的问题。

7.3 2D-2D：对极几何

7.3.1 对极约束

现在，假设我们从两张图像中得到了一对配对好的特征点，如图 7-9 所示。如果有若干对这样的匹配点，就可以通过这些二维图像点的对应关系，恢复出在两帧之间摄像机的运动。这里“若干对”具体是多少对呢？我们会在下文介绍。下面先来介绍两个图像中的匹配点有什么几何关系。

以图 7-9 为例，我们希望求取两帧图像 I_1, I_2 之间的运动，设第一帧到第二帧的运动为 $\boldsymbol{R}, \boldsymbol{t}$。两个相机中心分别为 O_1, O_2。现在，考虑 I_1 中有一个特征点 p_1，它在 I_2 中对应着特征点 p_2。我们知道两者是通过特征匹配得到的。如果匹配正确，说明它们确实是**同一个空间点在两个成像**

平面上的投影。这里需要一些术语来描述它们之间的几何关系。首先，连线 $\overrightarrow{O_1p_1}$ 和连线 $\overrightarrow{O_2p_2}$ 在三维空间中会相交于点 P。这时 O_1, O_2, P 三个点可以确定一个平面，称为**极平面（Epipolar plane）**。O_1O_2 连线与像平面 I_1, I_2 的交点分别为 e_1, e_2。e_1, e_2 称为**极点（Epipoles）**，O_1O_2 被称为**基线**。我们称极平面与两个像平面 I_1, I_2 之间的相交线 l_1, l_2 为**极线（Epipolar line）**。

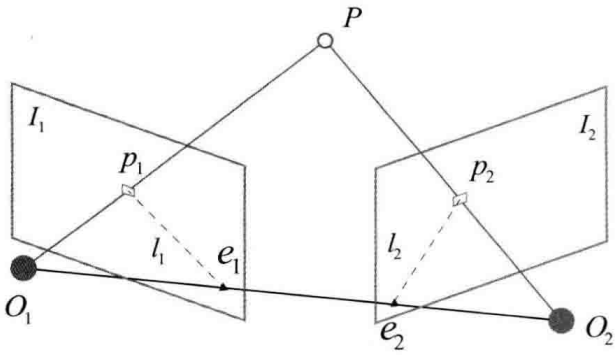


图 7-9 对极几何约束

从第一帧的角度看，射线 $\overrightarrow{O_1p_1}$ 是**某个像素可能出现的空间位置**——因为该射线上的所有点都会投影到同一个像素点。同时，如果不知道 P 的位置，那么当我们在第二幅图像上看时，连线 $\overrightarrow{e_2p_2}$（也就是第二幅图像中的极线）就是 P 可能出现的投影的位置，也就是射线 $\overrightarrow{O_1p_1}$ 在第二个相机中的投影。现在，由于我们通过特征点匹配确定了 p_2 的像素位置，所以能够推断 P 的空间位置，以及相机的运动。要提醒读者的是，**这多亏了正确的特征匹配**。如果没有特征匹配，我们就没法确定 p_2 到底在极线的哪个位置。那时，就必须在极线上搜索以获得正确的匹配，这将在第 12 讲中提到。

现在，我们从代数角度来分析这里的几何关系。在第一帧的坐标系下，设 P 的空间位置为

$$\boldsymbol{P} = [X, Y, Z]^{\mathrm{T}}.$$

根据第 5 讲介绍的针孔相机模型，我们知道两个像素点 $\boldsymbol{p}_1, \boldsymbol{p}_2$ 的像素位置为

$$s_1\boldsymbol{p}_1 = \boldsymbol{K}\boldsymbol{P}, \quad s_2\boldsymbol{p}_2 = \boldsymbol{K}(\boldsymbol{R}\boldsymbol{P} + \boldsymbol{t}). \tag{7.1}$$

这里 $\boldsymbol{K}$ 为相机内参矩阵，$\boldsymbol{R}, \boldsymbol{t}$ 为两个坐标系的相机运动。具体来说，这里计算的是 $\boldsymbol{R}_{21}$ 和 $\boldsymbol{t}_{21}$，因为它们把第一个坐标系下的坐标转换到第二个坐标系下。如果我们愿意，也可以把它们写成李代数形式。

有时，我们会使用齐次坐标表示像素点。在使用齐次坐标时，一个向量将等于它自身乘上任意的非零常数。这通常用于表达一个投影关系。例如，$s_1\boldsymbol{p}_1$ 和 $\boldsymbol{p}_1$ 成投影关系，它们在齐次坐标的意义下是相等的。我们称这种相等关系为**尺度意义下相等**（equal up to a scale），记作：

$$s\boldsymbol{p} \simeq \boldsymbol{p}. \tag{7.2}$$

那么，上述两个投影关系可写为

$$\boldsymbol{p}_1 \simeq \boldsymbol{K}\boldsymbol{P}, \quad \boldsymbol{p}_2 \simeq \boldsymbol{K}(\boldsymbol{R}\boldsymbol{P} + \boldsymbol{t}). \tag{7.3}$$

现在，取：

$$\boldsymbol{x}_1 = \boldsymbol{K}^{-1}\boldsymbol{p}_1, \quad \boldsymbol{x}_2 = \boldsymbol{K}^{-1}\boldsymbol{p}_2. \tag{7.4}$$

这里的 $\boldsymbol{x}_1, \boldsymbol{x}_2$ 是两个像素点的归一化平面上的坐标。代入上式，得

$$\boldsymbol{x}_2 \simeq \boldsymbol{R}\boldsymbol{x}_1 + \boldsymbol{t}. \tag{7.5}$$

两边同时左乘 $\boldsymbol{t}^\wedge$。回忆 $\wedge$ 的定义，这相当于两侧同时与 $\boldsymbol{t}$ 做外积：

$$\boldsymbol{t}^\wedge \boldsymbol{x}_2 \simeq \boldsymbol{t}^\wedge \boldsymbol{R}\boldsymbol{x}_1. \tag{7.6}$$

然后，两侧同时左乘 $\boldsymbol{x}_2^{\mathrm{T}}$：

$$\boldsymbol{x}_2^{\mathrm{T}} \boldsymbol{t}^\wedge \boldsymbol{x}_2 \simeq \boldsymbol{x}_2^{\mathrm{T}} \boldsymbol{t}^\wedge \boldsymbol{R}\boldsymbol{x}_1. \tag{7.7}$$

观察等式左侧，$\boldsymbol{t}^\wedge \boldsymbol{x}_2$ 是一个与 $\boldsymbol{t}$ 和 $\boldsymbol{x}_2$ 都垂直的向量。它再和 $\boldsymbol{x}_2$ 做内积时，将得到 0。由于等式左侧严格为零，乘以任意非零常数之后也为零，于是我们可以把 $\simeq$ 写成通常的等号。因此，我们就得到了一个简洁的式子：

$$\boldsymbol{x}_2^{\mathrm{T}} \boldsymbol{t}^\wedge \boldsymbol{R}\boldsymbol{x}_1 = 0. \tag{7.8}$$

重新代入 $\boldsymbol{p}_1, \boldsymbol{p}_2$，有

$$\boldsymbol{p}_2^{\mathrm{T}} \boldsymbol{K}^{-\mathrm{T}} \boldsymbol{t}^\wedge \boldsymbol{R}\boldsymbol{K}^{-1} \boldsymbol{p}_1 = 0. \tag{7.9}$$

这两个式子都称为**对极约束**，它以形式简洁著名。它的几何意义是 O_1, P, O_2 三者共面。对极约束中同时包含了平移和旋转。我们把中间部分记作两个矩阵：基础矩阵（Fundamental Matrix）$\boldsymbol{F}$ 和本质矩阵（Essential Matrix）$\boldsymbol{E}$，于是可以进一步简化对极约束：

$$\boldsymbol{E} = \boldsymbol{t}^\wedge \boldsymbol{R}, \quad \boldsymbol{F} = \boldsymbol{K}^{-\mathrm{T}} \boldsymbol{E} \boldsymbol{K}^{-1}, \quad \boldsymbol{x}_2^{\mathrm{T}} \boldsymbol{E} \boldsymbol{x}_1 = \boldsymbol{p}_2^{\mathrm{T}} \boldsymbol{F} \boldsymbol{p}_1 = 0. \tag{7.10}$$

对极约束简洁地给出了两个匹配点的空间位置关系。于是，相机位姿估计问题变为以下两步：

1. 根据配对点的像素位置求出 $\boldsymbol{E}$ 或者 $\boldsymbol{F}$。
2. 根据 $\boldsymbol{E}$ 或者 $\boldsymbol{F}$ 求出 $\boldsymbol{R}, \boldsymbol{t}$。

由于 $\boldsymbol{E}$ 和 $\boldsymbol{F}$ 只相差了相机内参，而内参在 SLAM 中通常是已知的[①]，所以实践中往往使用形式更简单的 $\boldsymbol{E}$。我们以 $\boldsymbol{E}$ 为例，介绍上面两个问题如何求解。

7.3.2 本质矩阵

根据定义，本质矩阵 $\boldsymbol{E} = \boldsymbol{t}^\wedge \boldsymbol{R}$。它是一个 3×3 的矩阵，内有 9 个未知数。那么，是不是任意一个 3×3 的矩阵都可以被当成本质矩阵呢？从 $\boldsymbol{E}$ 的构造方式上看，有以下值得注意的地方：

- 本质矩阵是由对极约束定义的。由于对极约束是**等式为零**的约束，所以对 $\boldsymbol{E}$ 乘以任意非零常数后，**对极约束依然满足**。我们把这件事情称为 $\boldsymbol{E}$ 在不同尺度下是等价的。
- 根据 $\boldsymbol{E} = \boldsymbol{t}^\wedge \boldsymbol{R}$，可以证明[3]，本质矩阵 $\boldsymbol{E}$ 的奇异值必定是 $[\sigma, \sigma, 0]^\mathrm{T}$ 的形式。这称为**本质矩阵的内在性质**。
- 另外，由于平移和旋转各有 3 个自由度，故 $\boldsymbol{t}^\wedge \boldsymbol{R}$ 共有 6 个自由度。但由于尺度等价性，故 $\boldsymbol{E}$ 实际上有 5 个自由度。

$\boldsymbol{E}$ 具有 5 个自由度的事实，表明我们最少可以用 5 对点来求解 $\boldsymbol{E}$。但是，$\boldsymbol{E}$ 的内在性质是一种非线性性质，在估计时会带来麻烦，因此，也可以只考虑它的**尺度等价性**，使用 8 对点来估计 $\boldsymbol{E}$——这就是经典的**八点法（Eight-point-algorithm）**[48, 49]。八点法只利用了 $\boldsymbol{E}$ 的线性性质，因此可以在线性代数框架下求解。下面我们来看八点法是如何工作的。

考虑一对匹配点，它们的归一化坐标为 $\boldsymbol{x}_1 = [u_1, v_1, 1]^\mathrm{T}, \boldsymbol{x}_2 = [u_2, v_2, 1]^\mathrm{T}$。根据对极约束，有

$$\begin{pmatrix} u_2, v_2, 1 \end{pmatrix} \begin{pmatrix} e_1 & e_2 & e_3 \\ e_4 & e_5 & e_6 \\ e_7 & e_8 & e_9 \end{pmatrix} \begin{pmatrix} u_1 \\ v_1 \\ 1 \end{pmatrix} = 0. \tag{7.11}$$

我们把矩阵 $\boldsymbol{E}$ 展开，写成向量的形式：

$$\boldsymbol{e} = [e_1, e_2, e_3, e_4, e_5, e_6, e_7, e_8, e_9]^\mathrm{T},$$

那么，对极约束可以写成与 $\boldsymbol{e}$ 有关的线性形式：

$$[u_2 u_1, u_2 v_1, u_2, v_2 u_1, v_2 v_1, v_2, u_1, v_1, 1] \cdot \boldsymbol{e} = 0. \tag{7.12}$$

同理，对于其他点对也有相同的表示。我们把所有点都放到一个方程中，变成线性方程组（u^i, v^i 表示第 i 个特征点，依此类推）：

①在 SfM 研究中则有可能是未知且有待估计的。

$$\begin{pmatrix} u_2^1u_1^1 & u_2^1v_1^1 & u_2^1 & v_2^1u_1^1 & v_2^1v_1^1 & v_2^1 & u_1^1 & v_1^1 & 1 \\ u_2^2u_1^2 & u_2^2v_1^2 & u_2^2 & v_2^2u_1^2 & v_2^2v_1^2 & v_2^2 & u_1^2 & v_1^2 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \\ u_2^8u_1^8 & u_2^8v_1^8 & u_2^8 & v_2^8u_1^8 & v_2^8v_1^8 & v_2^8 & u_1^8 & v_1^8 & 1 \end{pmatrix} \begin{pmatrix} e_1 \\ e_2 \\ e_3 \\ e_4 \\ e_5 \\ e_6 \\ e_7 \\ e_8 \\ e_9 \end{pmatrix} = 0. \tag{7.13}$$

这 8 个方程构成了一个线性方程组。它的系数矩阵由特征点位置构成，大小为 8×9。$\boldsymbol{e}$ 位于该矩阵的零空间中。如果系数矩阵是满秩的（即秩为 8），那么它的零空间维数为 1，也就是 $\boldsymbol{e}$ 构成一条线。这与 $\boldsymbol{e}$ 的尺度等价性是一致的。如果 8 对匹配点组成的矩阵满足秩为 8 的条件，那么 $\boldsymbol{E}$ 的各元素就可由上述方程解得。

接下来的问题是如何根据已经估得的本质矩阵 $\boldsymbol{E}$，恢复出相机的运动 $\boldsymbol{R}, \boldsymbol{t}$。这个过程是由奇异值分解（SVD）得到的。设 $\boldsymbol{E}$ 的 SVD 为

$$\boldsymbol{E} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^{\mathrm{T}}, \tag{7.14}$$

其中 $\boldsymbol{U}, \boldsymbol{V}$ 为正交阵，$\boldsymbol{\Sigma}$ 为奇异值矩阵。根据 $\boldsymbol{E}$ 的内在性质，我们知道 $\boldsymbol{\Sigma} = \mathrm{diag}(\sigma, \sigma, 0)$。在 SVD 分解中，对于任意一个 $\boldsymbol{E}$，存在两个可能的 $\boldsymbol{t}, \boldsymbol{R}$ 与它对应：

$$\begin{aligned} \boldsymbol{t}_1^\wedge &= \boldsymbol{U}\boldsymbol{R}_Z(\frac{\pi}{2})\boldsymbol{\Sigma}\boldsymbol{U}^{\mathrm{T}}, \quad \boldsymbol{R}_1 = \boldsymbol{U}\boldsymbol{R}_Z^{\mathrm{T}}(\frac{\pi}{2})\boldsymbol{V}^{\mathrm{T}} \\ \boldsymbol{t}_2^\wedge &= \boldsymbol{U}\boldsymbol{R}_Z(-\frac{\pi}{2})\boldsymbol{\Sigma}\boldsymbol{U}^{\mathrm{T}}, \quad \boldsymbol{R}_2 = \boldsymbol{U}\boldsymbol{R}_Z^{\mathrm{T}}(-\frac{\pi}{2})\boldsymbol{V}^{\mathrm{T}}. \end{aligned} \tag{7.15}$$

其中，$\boldsymbol{R}_Z(\frac{\pi}{2})$ 表示沿 Z 轴旋转 90° 得到旋转矩阵。同时，由于 $-\boldsymbol{E}$ 和 $\boldsymbol{E}$ 等价，所以对任意一个 $\boldsymbol{t}$ 取负号，也会得到同样的结果。因此，从 $\boldsymbol{E}$ 分解到 $\boldsymbol{t}, \boldsymbol{R}$ 时，一共存在 **4 个**可能的解。

图 7-10 形象地展示了分解本质矩阵得到的 4 个解。我们已知空间点在相机（蓝色线）上的投影（红色点），想要求解相机的运动。在保持红色点不变的情况下，可以画出 4 种可能的情况。不过幸运的是，只有第一种解中 P 在两个相机中都具有正的深度。因此，只要把任意一点代入 4 种解中，检测该点在两个相机下的深度，就可以确定哪个解是正确的了。

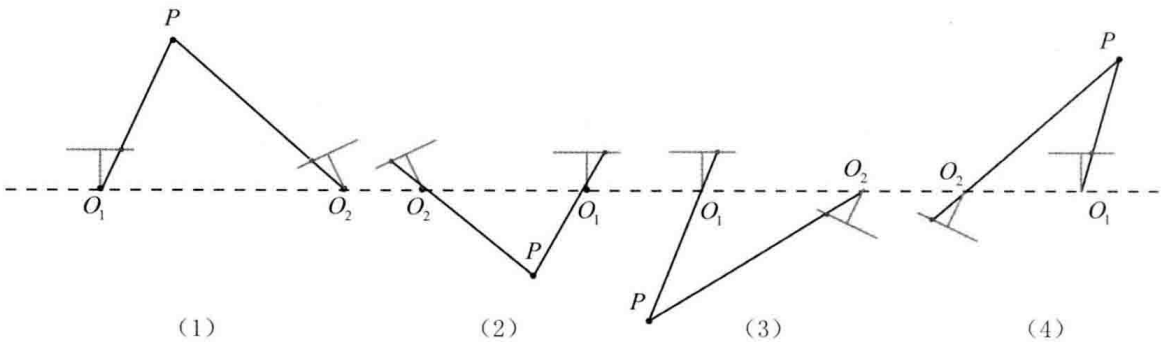


图 7-10 分解本质矩阵得到的 4 个解。在保持投影点（红色点）不变的情况下，两个相机及空间点一共有 4 种可能的情况（见彩插）

如果利用 $\boldsymbol{E}$ 的内在性质，那么它只有 5 个自由度。所以最少可以通过 5 对点来求解相机运动[50, 51]。然而这种做法形式复杂，从工程实现角度考虑，由于平时通常会有几十对乃至上百对的匹配点，从 8 对减至 5 对意义并不明显。为保持简单，我们这里就只介绍基本的八点法。

剩下的一个问题是：根据线性方程解出的 $\boldsymbol{E}$，可能不满足 $\boldsymbol{E}$ 的内在性质——它的奇异值不一定为 $\sigma, \sigma, 0$ 的形式。这时，我们会刻意地把 $\boldsymbol{\Sigma}$ 矩阵调整成上面的样子。通常的做法是，对八点法求得的 $\boldsymbol{E}$ 进行 SVD，会得到奇异值矩阵 $\boldsymbol{\Sigma} = \text{diag}(\sigma_1, \sigma_2, \sigma_3)$，不妨设 $\sigma_1 \geqslant \sigma_2 \geqslant \sigma_3$。取：

$$\boldsymbol{E} = \boldsymbol{U}\text{diag}\left(\frac{\sigma_1 + \sigma_2}{2}, \frac{\sigma_1 + \sigma_2}{2}, 0\right)\boldsymbol{V}^{\mathrm{T}}. \tag{7.16}$$

这相当于是把求出来的矩阵投影到了 $\boldsymbol{E}$ 所在的流形上。当然，更简单的做法是将奇异值矩阵取成 $\text{diag}(1, 1, 0)$，因为 $\boldsymbol{E}$ 具有尺度等价性，所以这样做也是合理的。

7.3.3 单应矩阵

除了基本矩阵和本质矩阵，二视图几何中还存在另一种常见的矩阵：单应矩阵（Homography）$\boldsymbol{H}$，它描述了两个平面之间的映射关系。若场景中的特征点都落在同一平面上（比如墙、地面等），则可以通过单应性进行运动估计。这种情况在无人机携带的俯视相机或扫地机携带的顶视相机中比较常见。由于之前没有提到过单应，因此这里稍微介绍一下。

单应矩阵通常描述处于共同平面上的一些点在两张图像之间的变换关系。设图像 I_1 和 I_2 有一对匹配好的特征点 p_1 和 p_2。这个特征点落在平面 P 上，设这个平面满足方程：

$$\boldsymbol{n}^{\mathrm{T}}\boldsymbol{P} + d = 0. \tag{7.17}$$

稍加整理，得

$$-\frac{\boldsymbol{n}^{\mathrm{T}}\boldsymbol{P}}{d} = 1. \tag{7.18}$$

然后，回顾本节开头的式 (7.1)，得

$$\begin{aligned}\boldsymbol{p}_2 &\simeq \boldsymbol{K}(\boldsymbol{R}\boldsymbol{P}+\boldsymbol{t}) \\ &\simeq \boldsymbol{K}\left(\boldsymbol{R}\boldsymbol{P}+\boldsymbol{t}\cdot(-\frac{\boldsymbol{n}^{\mathrm{T}}\boldsymbol{P}}{d})\right) \\ &\simeq \boldsymbol{K}\left(\boldsymbol{R}-\frac{\boldsymbol{t}\boldsymbol{n}^{\mathrm{T}}}{d}\right)\boldsymbol{P} \\ &\simeq \boldsymbol{K}\left(\boldsymbol{R}-\frac{\boldsymbol{t}\boldsymbol{n}^{\mathrm{T}}}{d}\right)\boldsymbol{K}^{-1}\boldsymbol{p}_1.\end{aligned}$$

于是，我们得到了一个直接描述图像坐标 $\boldsymbol{p}_1$ 和 $\boldsymbol{p}_2$ 之间的变换，把中间这部分记为 $\boldsymbol{H}$，于是：

$$\boldsymbol{p}_2 \simeq \boldsymbol{H}\boldsymbol{p}_1. \tag{7.19}$$

它的定义与旋转、平移及平面的参数有关。与基础矩阵 $\boldsymbol{F}$ 类似，单应矩阵 $\boldsymbol{H}$ 也是一个 3×3 的矩阵，求解时的思路和 $\boldsymbol{F}$ 类似，同样可以先根据匹配点计算 $\boldsymbol{H}$，然后将它分解以计算旋转和平移。把上式展开，得

$$\begin{pmatrix}u_2\\v_2\\1\end{pmatrix} \simeq \begin{pmatrix}h_1 & h_2 & h_3\\h_4 & h_5 & h_6\\h_7 & h_8 & h_9\end{pmatrix}\begin{pmatrix}u_1\\v_1\\1\end{pmatrix}. \tag{7.20}$$

请注意，这里的等号依然是 $\simeq$ 而不是普通的等号，所以 $\boldsymbol{H}$ 矩阵也可以乘以任意非零常数。我们在实际处理中可以令 $h_9=1$（在它取非零值时）。然后根据第 3 行，去掉这个非零因子，于是有

$$\begin{aligned}u_2 &= \frac{h_1u_1+h_2v_1+h_3}{h_7u_1+h_8v_1+h_9} \\ v_2 &= \frac{h_4u_1+h_5v_1+h_6}{h_7u_1+h_8v_1+h_9}.\end{aligned}$$

整理得

$$\begin{aligned}h_1u_1+h_2v_1+h_3-h_7u_1u_2-h_8v_1u_2 &= u_2 \\ h_4u_1+h_5v_1+h_6-h_7u_1v_2-h_8v_1v_2 &= v_2.\end{aligned}$$

这样一组匹配点对就可以构造出两项约束（事实上有三个约束，但是因为线性相关，只取前两个），于是自由度为 8 的单应矩阵可以通过 4 对匹配特征点算出（在非退化的情况下，即这些特征点不能有三点共线的情况），即求解以下的线性方程组（当 $h_9=0$ 时，右侧为零）：

$$\begin{pmatrix} u_1^1 u_2^1 & v_1^1 u_2^1 & u_2^1 & u_1^1 v_2^1 & v_1^1 v_2^1 & v_2^1 & u_1^1 & v_1^1 & 1 \\ u_1^2 u_2^2 & v_1^2 u_2^2 & u_2^2 & u_1^2 v_2^2 & v_1^2 v_2^2 & v_2^2 & u_1^2 & v_1^2 & 1 \\ u_1^3 u_2^3 & v_1^3 u_2^3 & u_2^3 & u_1^3 v_2^3 & v_1^3 v_2^3 & v_2^3 & u_1^3 & v_1^3 & 1 \\ u_1^4 u_2^4 & v_1^4 u_2^4 & u_2^4 & u_1^4 v_2^4 & v_1^4 v_2^4 & v_2^4 & u_1^4 & v_1^4 & 1 \end{pmatrix} \begin{pmatrix} h_1 \\ h_2 \\ h_3 \\ h_4 \\ h_5 \\ h_6 \\ h_7 \\ h_8 \\ h_9 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}. \tag{7.21}$$

这种做法把 $\boldsymbol{H}$ 矩阵看成了向量，通过解该向量的线性方程来恢复 $\boldsymbol{H}$，又称直接线性变换法（Direct Linear Transform，DLT）。与本质矩阵相似，求出单应矩阵以后需要对其进行分解，才可以得到相应的旋转矩阵 $\boldsymbol{R}$ 和平移向量 $\boldsymbol{t}$。分解的方法包括数值法[52, 53] 与解析法[54]。与本质矩阵的分解类似，单应矩阵的分解同样会返回 4 组旋转矩阵与平移向量，同时，可以计算出它们分别对应的场景点所在平面的法向量。如果已知成像的地图点的深度全为正值（即在相机前方），则又可以排除两组解。最后仅剩两组解，这时需要通过更多的先验信息进行判断。通常，我们可以通过假设已知场景平面的法向量来解决，如场景平面与相机平面平行，那么法向量 $\boldsymbol{n}$ 的理论值为 $\mathbf{1}^{\mathrm{T}}$。

单应性在 SLAM 中具有重要意义。当特征点共面或者相机发生纯旋转时，基础矩阵的自由度下降，这就出现了所谓的退化（degenerate）。现实中的数据总包含一些噪声，这时如果继续使用八点法求解基础矩阵，基础矩阵多余出来的自由度将会主要由噪声决定。为了能够避免退化现象造成的影响，通常我们会同时估计基础矩阵 $\boldsymbol{F}$ 和单应矩阵 $\boldsymbol{H}$，选择重投影误差比较小的那个作为最终的运动估计矩阵。

7.4 实践：对极约束求解相机运动

下面，我们来练习如何通过本质矩阵求解相机运动。7.2 节实践部分的程序提供了特征匹配，而本节我们就使用匹配好的特征点来计算 $\boldsymbol{E}, \boldsymbol{F}$ 和 $\boldsymbol{H}$，进而分解 $\boldsymbol{E}$ 得到 $\boldsymbol{R}, \boldsymbol{t}$。整个程序使用 OpenCV 提供的算法进行求解。我们把 7.2 节的特征提取封装成函数，以供后面使用。本节只展示位姿估计部分的代码。

slambook2/ch7/pose_estimation_2d2d.cpp（片段）

```
1 void pose_estimation_2d2d(std::vector<KeyPoint> keypoints_1,
2     std::vector<KeyPoint> keypoints_2,
3     std::vector<DMatch> matches,
```

```
    Mat &R, Mat &t) {
    // 相机内参, TUM Freiburg2
    Mat K = (Mat_<double>(3, 3) << 520.9, 0, 325.1, 0, 521.0, 249.7, 0, 0, 1);

    //-- 把匹配点转换为vector<Point2f>的形式
    vector<Point2f> points1;
    vector<Point2f> points2;

    for (int i = 0; i < (int) matches.size(); i++) {
        points1.push_back(keypoints_1[matches[i].queryIdx].pt);
        points2.push_back(keypoints_2[matches[i].trainIdx].pt);
    }

    //-- 计算基础矩阵
    Mat fundamental_matrix;
    fundamental_matrix = findFundamentalMat(points1, points2, CV_FM_8POINT);
    cout << "fundamental_matrix is " << endl << fundamental_matrix << endl;

    //-- 计算本质矩阵
    Point2d principal_point(325.1, 249.7);  //相机光心, TUM dataset标定值
    double focal_length = 521;      //相机焦距, TUM dataset标定值
    Mat essential_matrix;
    essential_matrix = findEssentialMat(points1, points2, focal_length, principal_point);
    cout << "essential_matrix is " << endl << essential_matrix << endl;

    //-- 计算单应矩阵
    //-- 但是本例中场景不是平面, 单应矩阵意义不大
    Mat homography_matrix;
    homography_matrix = findHomography(points1, points2, RANSAC, 3);
    cout << "homography_matrix is " << endl << homography_matrix << endl;

    //-- 从本质矩阵中恢复旋转和平移信息
    recoverPose(essential_matrix, points1, points2, R, t, focal_length, principal_point);
    cout << "R is " << endl << R << endl;
    cout << "t is " << endl << t << endl;
}
```

该函数提供了从特征点求解相机运动的部分，然后，我们在主函数中调用它，就能得到相机的运动：

slambook2/ch7/pose_estimation_2d2d.cpp（片段）

```
int main( int argc, char** argv ){
    if (argc != 3) {
```

```
        cout << "usage: pose_estimation_2d2d img1 img2" << endl;
        return 1;
    }
    //-- 读取图像
    Mat img_1 = imread(argv[1], CV_LOAD_IMAGE_COLOR);
    Mat img_2 = imread(argv[2], CV_LOAD_IMAGE_COLOR);
    assert(img_1.data && img_2.data && "Can not load images!");

    vector<KeyPoint> keypoints_1, keypoints_2;
    vector<DMatch> matches;
    find_feature_matches(img_1, img_2, keypoints_1, keypoints_2, matches);
    cout << "一共找到了" << matches.size() << "组匹配点" << endl;

    //-- 估计两张图像间运动
    Mat R, t;
    pose_estimation_2d2d(keypoints_1, keypoints_2, matches, R, t);

    //-- 验证E=t^R*scale
    Mat t_x =
        (Mat_<double>(3, 3) << 0, -t.at<double>(2, 0), t.at<double>(1, 0),
        t.at<double>(2, 0), 0, -t.at<double>(0, 0),
        -t.at<double>(1, 0), t.at<double>(0, 0), 0);
    cout << "t^R=" << endl << t_x * R << endl;

    //-- 验证对极约束
    Mat K = (Mat_<double>(3, 3) << 520.9, 0, 325.1, 0, 521.0, 249.7, 0, 0, 1);
    for (DMatch m: matches) {
        Point2d pt1 = pixel2cam(keypoints_1[m.queryIdx].pt, K);
        Mat y1 = (Mat_<double>(3, 1) << pt1.x, pt1.y, 1);
        Point2d pt2 = pixel2cam(keypoints_2[m.trainIdx].pt, K);
        Mat y2 = (Mat_<double>(3, 1) << pt2.x, pt2.y, 1);
        Mat d = y2.t() * t_x * R * y1;
        cout << "epipolar constraint = " << d << endl;
    }
    return 0;
}
```

我们在函数中输出了 $\boldsymbol{E}, \boldsymbol{F}$ 和 $\boldsymbol{H}$ 的数值，然后验证了对极约束是否成立，以及 $\boldsymbol{t}^\wedge \boldsymbol{R}$ 和 $\boldsymbol{E}$ 在非零数乘下等价的事实。现在，调用此程序即可看到输出结果：

终端输入：

```
% build/pose_estimation_2d2d 1.png 2.png
-- Max dist : 95.000000
```

```
-- Min dist : 4.000000
一共找到了79组匹配点
fundamental_matrix is
[4.844484382466111e-06, 0.0001222601840188731, -0.01786737827487386;
-0.0001174326832719333, 2.122888800459598e-05, -0.01775877156212593;
0.01799658210895528, 0.008143605989020664, 1]
essential_matrix is
[-0.0203618550523477, -0.4007110038118445, -0.03324074249824097;
0.3939270778216369, -0.03506401846698079, 0.5857110303721015;
-0.006788487241438284, -0.5815434272915686, -0.01438258684486258]
homography_matrix is
[0.9497129583105288, -0.143556453147626, 31.20121878625771;
0.04154536627445031, 0.9715568969832015, 5.306887618807696;
-2.81813676978796e-05, 4.353702039810921e-05, 1]
R is
[0.9985961798781875, -0.05169917220143662, 0.01152671359827873;
0.05139607508976055, 0.9983603445075083, 0.02520051547522442;
-0.01281065954813571, -0.02457271064688495, 0.9996159607036126]
t is
[-0.8220841067933337;
-0.03269742706405412;
0.5684264241053522]

t^R=
[0.02879601157010516, 0.5666909361828478, 0.04700950886436416;
-0.5570970160413605, 0.0495880104673049, -0.8283204827837456;
0.009600370724838804, 0.8224266019846683, 0.02034004937801349]
epipolar constraint = [0.002528128704106625]
epipolar constraint = [-0.001663727901710724]
epipolar constraint = [-0.0008009088410884102]
......
```

从程序的输出结果可以看出，对极约束的满足精度约在 10^{-3} 量级。根据前面的讨论，分解得到的 $\boldsymbol{R}, \boldsymbol{t}$ 一共有 4 种可能性。不过，OpenCV 会替我们检测角点的深度是否为正，从而选出正确的解。

讨论

从演示程序中可以看到，输出的 $\boldsymbol{E}$ 和 $\boldsymbol{F}$ 之间相差了相机内参矩阵。虽然它们在数值上并不直观，但可以验证它们的数学关系。从 $\boldsymbol{E}, \boldsymbol{F}$ 和 $\boldsymbol{H}$ 都可以分解出运动，不过 $\boldsymbol{H}$ 需要假设特征点位于平面上。对于本实验的数据，这个假设是不好的，所以我们主要用 $\boldsymbol{E}$ 分解运动。

值得一提的是，由于 $\boldsymbol{E}$ 本身具有尺度等价性，它分解得到的 $\boldsymbol{t}, \boldsymbol{R}$ 也有一个尺度等价性。而

$\boldsymbol{R} \in \mathrm{SO}(3)$ 自身具有约束，所以我们认为 $\boldsymbol{t}$ 具有一个**尺度**。换言之，在分解过程中，对 $\boldsymbol{t}$ 乘以任意非零常数，分解都是成立的。因此，我们通常把 $\boldsymbol{t}$ 进行**归一化**，让它的长度等于 1。

尺度不确定性

对 $\boldsymbol{t}$ 长度的归一化，直接导致了**单目视觉的尺度不确定性**。例如，程序中输出的 $\boldsymbol{t}$ 第一维约为 0.822。这个 0.822 究竟是指 0.822 米还是 0.822 厘米，我们是没法确定的。因为对 $\boldsymbol{t}$ 乘以任意比例常数后，对极约束依然是成立的。换言之，在单目 SLAM 中，对轨迹和地图同时缩放任意倍数，我们得到的图像依然是一样的。这在第 2 讲中就已经向读者介绍过了。

在单目视觉中，我们对两张图像的 $\boldsymbol{t}$ 归一化相当于**固定了尺度**。虽然我们不知道它的实际长度是多少，但我们以这时的 $\boldsymbol{t}$ 为单位 1，计算相机运动和特征点的 3D 位置。这被称为单目 SLAM 的**初始化**。在初始化之后，就可以用 3D–2D 计算相机运动了。初始化之后的轨迹和地图的单位，就是初始化时固定的尺度。因此，单目 SLAM 有一步不可避免的**初始化**。初始化的两张图像必须有一定程度的平移，而后的轨迹和地图都将以此步的平移为单位。

除了对 $\boldsymbol{t}$ 进行归一化，另一种方法是令初始化时所有的特征点平均深度为 1，也可以固定一个尺度。相比于令 $\boldsymbol{t}$ 长度为 1 的做法，把特征点深度归一化可以控制场景的规模大小，使计算在数值上更稳定。

初始化的纯旋转问题

从 $\boldsymbol{E}$ 分解到 $\boldsymbol{R}, \boldsymbol{t}$ 的过程中，如果相机发生的是纯旋转，导致 $\boldsymbol{t}$ 为零，那么，得到的 $\boldsymbol{E}$ 也将为零，这将导致我们无从求解 $\boldsymbol{R}$。不过，此时我们可以依靠 $\boldsymbol{H}$ 求取旋转，但仅有旋转时，我们无法用三角测量估计特征点的空间位置（这将在下文提到），于是，另一个结论是，**单目初始化不能只有纯旋转，必须要有一定程度的平移**。如果没有平移，单目将无法初始化。在实践中，如果初始化时平移太小，会使得位姿求解与三角化结果不稳定，从而导致失败。相对地，如果把相机左右移动而不是原地旋转，就容易让单目 SLAM 初始化。因而，有经验的 SLAM 研究人员，在单目 SLAM 情况下经常选择让相机进行左右平移以顺利地进行初始化。

多于 8 对点的情况

当给定的点数多于 8 对时（例如，例程找到了 79 对匹配），我们可以计算一个最小二乘解。回忆式 (7.13) 中线性化后的对极约束，我们把左侧的系数矩阵记为 $\boldsymbol{A}$：

$$\boldsymbol{A}\boldsymbol{e} = \boldsymbol{0}. \tag{7.22}$$

对于八点法，$\boldsymbol{A}$ 的大小为 8×9。如果给定的匹配点多于 8，则该方程构成一个超定方程，即不一定存在 $\boldsymbol{e}$ 使得上式成立。因此，可以通过最小化一个二次型来求：

$$\min_{\boldsymbol{e}} \|\boldsymbol{A}\boldsymbol{e}\|_2^2 = \min_{\boldsymbol{e}} \boldsymbol{e}^{\mathrm{T}} \boldsymbol{A}^{\mathrm{T}} \boldsymbol{A} \boldsymbol{e}. \tag{7.23}$$

于是就求出了在最小二乘意义下的 $\boldsymbol{E}$ 矩阵。不过，当可能存在误匹配的情况时，我们会更倾向于使用**随机采样一致性（Random Sample Concensus，RANSAC）**来求，而不是最小二乘。RANSAC 是一种通用的做法，适用于很多带错误数据的情况，可以处理带有错误匹配的数据。

7.5　三角测量

之前两节我们使用对极几何约束估计了相机运动，也讨论了这种方法的局限性。在得到运动之后，下一步我们需要用相机的运动估计特征点的空间位置。在单目 SLAM 中，仅通过单张图像无法获得像素的深度信息，我们需要通过**三角测量（Triangulation）（或三角化）**的方法估计地图点的深度，如图 7-11 所示。

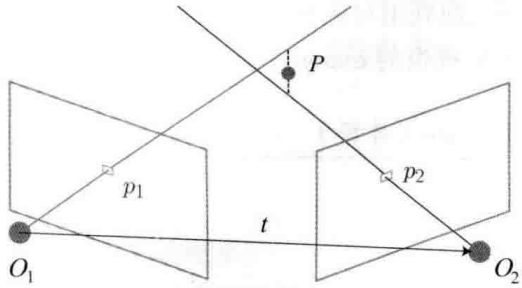


图 7-11　通过三角测量的方法获得地图点的深度

三角测量是指，通过不同位置对同一个路标点进行观察，从观察到的位置推断路标点的距离。三角测量最早由高斯提出并应用于测量学中，它在天文学、地理学的测量中都有应用。例如，我们可以通过不同季节观察到的星星的角度，估计它与我们的距离。在 SLAM 中，我们主要用三角化来估计像素点的距离。

和上一节类似，考虑图像 I_1 和 I_2，以左图为参考，右图的变换矩阵为 $\boldsymbol{T}$。相机光心为 O_1 和 O_2。在 I_1 中有特征点 p_1，对应 I_2 中有特征点 p_2。理论上，直线 O_1p_1 与 O_2p_2 在场景中会相交于一点 P，该点即两个特征点所对应的地图点在三维场景中的位置。然而由于噪声的影响，这两条直线往往无法相交。因此，可以通过最小二乘法求解。

按照对极几何中的定义，设 $\boldsymbol{x}_1, \boldsymbol{x}_2$ 为两个特征点的归一化坐标，那么它们满足：

$$s_2\boldsymbol{x}_2 = s_1\boldsymbol{R}\boldsymbol{x}_1 + \boldsymbol{t}. \tag{7.24}$$

现在已知 $\boldsymbol{R}, \boldsymbol{t}$，我们想求解两个特征点的深度 s_1, s_2。从几何上看，可以在射线 O_1p_1 上寻找 3D 点，使其投影位置接近 $\boldsymbol{p}_2$。同理，也可以在 O_2p_2 上找，或者在两条线的中间找。不同的策略对应着不同的计算方式，当然它们大同小异。例如，我们希望计算 s_1，那么先对上式两侧左乘一个 $\boldsymbol{x}_2^\wedge$，得

$$s_2 \boldsymbol{x}_2^\wedge \boldsymbol{x}_2 = 0 = s_1 \boldsymbol{x}_2^\wedge \boldsymbol{R} \boldsymbol{x}_1 + \boldsymbol{x}_2^\wedge \boldsymbol{t}. \tag{7.25}$$

该式左侧为零，右侧可看成 s_2 的一个方程，可以根据它直接求得 s_2。有了 s_2，s_1 也非常容易求出。于是，我们就得到了两帧下的点的深度，确定了它们的空间坐标。当然，由于噪声的存在，我们估得的 $\boldsymbol{R}, \boldsymbol{t}$ 不一定精确使式 (7.25) 为零，所以更常见的做法是求最小二乘解而不是直接的解。

7.6 实践：三角测量

7.6.1 三角测量代码

下面，我们演示如何根据之前利用对极几何求解的相机位姿，通过三角化求出上一节特征点的空间位置。我们调用 OpenCV 提供的 triangulation 函数进行三角化。

slambook2/ch7/triangulation.cpp（片段）

```
void triangulation(
    const vector<KeyPoint> &keypoint_1,
    const vector<KeyPoint> &keypoint_2,
    const std::vector<DMatch> &matches,
    const Mat &R, const Mat &t,
    vector<Point3d> &points) {
    Mat T1 = (Mat_<float>(3, 4) <<
        1, 0, 0, 0,
        0, 1, 0, 0,
        0, 0, 1, 0);
    Mat T2 = (Mat_<float>(3, 4) <<
        R.at<double>(0, 0), R.at<double>(0, 1), R.at<double>(0, 2), t.at<double>(0, 0),
        R.at<double>(1, 0), R.at<double>(1, 1), R.at<double>(1, 2), t.at<double>(1, 0),
        R.at<double>(2, 0), R.at<double>(2, 1), R.at<double>(2, 2), t.at<double>(2, 0)
    );

    Mat K = (Mat_<double>(3, 3) << 520.9, 0, 325.1, 0, 521.0, 249.7, 0, 0, 1);
    vector<Point2f> pts_1, pts_2;
    for (DMatch m:matches) {
        // 将像素坐标转换至相机坐标
        pts_1.push_back(pixel2cam(keypoint_1[m.queryIdx].pt, K));
        pts_2.push_back(pixel2cam(keypoint_2[m.trainIdx].pt, K));
    }

    Mat pts_4d;
    cv::triangulatePoints(T1, T2, pts_1, pts_2, pts_4d);
```